

successful algorithmic trading

中文版
(简校版)

作者：Michael L.Hall-Moore

翻译：CIIA 橘猫

序 言

本序言是译者添加，旨在方便读者在阅读前，能了解此书。

本书适合的人群是量化初学者，对金融或算法有一定了解想深入学习，并想实现交易的人。

和国内市面上的量化书籍不太一样，本书更注重实操性，当然这种实操性是指针对国外市场，但对国内想要从事或已经从事此行业的人有一定的借鉴作用。这就是使得本书有一定的阅读门槛，要求读者熟悉 `Python` 的操作以及金融市场基本知识，最好对机器学习的算法内容也有一定的了解。

说一说本书的优缺点，首先就是上面提到的国内很少有这种注重实操的量化书籍，书中有详细的代码介绍。其次本书的逻辑性很强，能够把解决问题贯彻始终。

再说一下本书存在的一些问题，主要是本书的一些方法不适用于国内环境，尤其是有些程序安装上过于复杂，读者可以自行安装不用考虑本书的方法。其次是受限于篇幅问题，有些算法的数学方法解释不够清楚，会使一些读者比较茫然，这里就建议读者在网上进行相关内容的查询，以详细的了解算法的内核。

还有吐槽一下本书的书名，个人感觉这个书名 `Flag` 立的有点大，不如叫《量化交易入门》

最后说明一下， 本书的翻译全无牟利之想，仅供大家学习参考。 本人水平有限能力一般，本书也没怎么校对，请轻喷。

Part1 介绍算法交易

第 1 章

本书简介

1.1 QuantStart 简介

QuantStart 由 Michael Halls-Moore 于 2010 年创立，旨在帮助初级量化分析师（QA）在艰难的经济环境中找到工作。从那时起，该网站已发展成为量化金融的重要资源。该网站现在专注于算法交易，还会讨论 Python 和 C++ 中的量化开发。自 2010 年 3 月以来，QuantStart 已帮助超过 20 万名访客提高了他们的量化金融技能。您可以发送电子邮件至 mike@quantstart.com 联系 QuantStart。

1.2 这本书说了什么？

成功的算法交易是为了教授具备基本的编程技能的个人交易者和金融交易专业人士如何使用 Python 编程语言创建完全自动化的强大的算法交易系统并且盈利。

本书描述了算法交易系统的本质，如何获取和金融数据，回测以及如何实现执行系统。本书的设计非常实用，本书中有 Python 代码示例，用于演示算法交易的原理和实践。

1.3 本书适应人群

本书是为个人交易者和专业人士编写的，对编程有一些基本的了解，并希望学习如何将计算机语言和库应用于算法交易。它专为喜欢自学并可以通过实际案例学习的人而设计。本书面向对编程和交易实现感兴趣的人，因为我相信算法交易的成功来自于完全理解实现细节。

专业的量化交易者也会发现有用内容，接触新库和实现方法可能会有助于优化原有框架或实现更准确的回测。

1.4 阅读本书前要掌握的技能

这本书相对独立，但需要熟悉交易基础知识。本书不需要强大的编程背景，但是要基本熟悉编程语言。应该了解基本编程概念，例如变量声明，流控制（if-else）和循环（for / while）。还有一些交易策略利用的统计机器学习技术。

此外，投资组合/策略优化部分广泛使用搜索和优化算法。虽然对数学的深入理解并非绝对必要，但如果学习过的话可以很容易地理解这些算法在实践层面是如何工作的。如果这些材料对您来说是新知识，请查看 **QuantStart** 阅读名单。

1.5 软件/硬件要求

Python 中的量化交易应用程序可以在 Windows，Mac OSX 或 Linux 中开发。本书与操作系统无关，因此最好使用您熟悉的任何系统。但我建议使用 Mac OSX 或 Linux（我使用 Ubuntu），因为我发现安装和数据管理要简单得多。

为了编写 Python 程序，您只需要使用文本编辑器（最好使用语法高

亮显示)。在 Windows 上我倾向于使用 Notepad ++。在 Mac OSX 上，我使用了 SublimeText。在 Ubuntu 上我倾向于使用 emacs，但当然，你可以使用 vim。

本书中的代码将在 Python 版本 2.7.x 和 Python 3.4.x 下运行。在硬件方面，您可能需要至少 1GB 的 RAM，但更多的总是更好。您还需要使用相对较新的 CPU 和大容量硬盘存储来存储历史数据，当然这取决于您打算交易的频率。对于较小的数据组正常一块 200Gb 的硬盘就足够了，1TB 对于储存 tick 级别数据很有用。

1.6 书籍结构

本书旨在创建一系列从构思到自动执行的算法交易策略。随后的内容如下。

- 为什么用算法交易？ - 讨论使用系统/算法方法交易的好处，而不是主观相关的交易方法。此外，还探讨了算法交易采用的不同方法。
- 交易系统开发 - 从假设到实时交易和持续评估，涵盖了开发算法交易系统的过程。
- 交易系统设计 - 涵盖形成算法交易系统的实际组件。特别是信号生成，风险管理，绩效衡量，仓位管理/杠杆，投资组合优化和实盘执行。
- 交易系统环境 - 所有 Python 及相关软件的程序安装
- 数据管理 - 执行并获取历史数据，清洗并存储在本地数据库系统中。

- 时间序列分析 - 各种时间序列方法用于预测，均值反转，动量和波动率识别。这些统计方法后来构成了交易策略的基础。
- 优化 - 讨论优化/搜索算法，并考虑它们如何应用于策略优化的示例。
- 绩效评估 - 详细描述了各种风险/回报和其他绩效指标的实施。
- 风险管理 - 概述了影响算法交易系统的各种风险来源，并提供了降低风险的方法。
- 交易策略实施 - 提供基于统计计量和技术指标的交易策略示例，以及如何优化此类策略组合的详细信息。
- 执行 - 讨论连接到经纪商，创建基于事件的自动交易系统和监控/弹性工具。

1.7 本书未涵盖的内容

这不是一本关于主观交易的初学者书，也不是一本充满“技术分析”交易策略的书。如果您没有进行任何交易经验，我建议您阅读 QuantStart 阅读清单上的一些书籍。

它也不是 Python 教程书，对 Python 的熟悉重要。虽然每个例子都尽力引入 Python 代码，有些内容可以再次查阅 QuantStart 阅读列表。

1.8 何处获得帮助

寻求帮助的最佳位置是 QuantStart.com 上的文章列表，可在 QuantStart.com/articles 找到，或者通过 mike@quantstart.com 与我联系。我写了 140 多篇关于量化金融（特别是算法交易）的文章，所以您可以通过阅读其中的一些来了解。我还想说谢谢你购买这本书，并

在我写更多内容时帮助支持我一非常感谢。 祝你的算法策略好运！

现在进行一些交易.....

第 2 章

什么是算法交易？

此处所定义的算法交易是使用自动化系统来执行交易，其通过算法以预定方式执行而没有任何人为干预。 后者的很重要，算法策略是在交易开始之前设计的并且在没有人类交易者自行决定的情况下执行。

在本书中，“算法交易”是指自动化，系统化和量化交易的实践，为了本文的目的，这些实践都将被视为同义词。 在金融行业中，“算法交易”通常是指用于优化较大交易订单成本的一类执行算法（如成交量加权平均价格，VWAP）。

2.1 概述

算法交易与主观交易有很大不同。在本节中，将概述算法交易的优点和缺点。

2.1.1 优点

与主观交易方法相比，算法交易具有许多优势。

历史回测评估

创建自动化交易的最重要优势在于，可以根据历史数据测算其绩效，可以试图代表未来的市场数据。整个过程称为回测，我将在本书中进行深入讨论。回测确定策略的（先前）统计特性，从而深入了解策略

在未来是否可能有利可图。

效率

算法交易比主观交易的方法更有效。使用全自动交易系统，个人或团队无需持续监控市场的价格或新闻。这为交易策略的开发者腾出时间进行更多研究，因此，效率的提升使得更多策略部署到投资组合中。此外，自动化风险管理和头寸调整过程，考虑到系统的稳定，就必须动态地自动调节杠杆和风险因素，直接应对市场动态的实时性。这在主观交易的世界中是不可能的，因为交易者无法持续计算风险，并且需要休息。

无主观操作

自动交易系统的主要优点之一是（理论上）没有主观交易影响。是指在交易时或在某个位置修改交易。在进行主观交易时，恐惧和贪婪可能是压倒性的动力。在系统交易的背景下，主观选择的交易很少能提高策略的表现。

话虽这么说，自动交易策略可能由于交易规则变化或其他外部因素不再有利可图。在这种情况下，需要判断修改策略的参数或将其退出。当然请注意，此过程仍然没有干扰个别交易。

比较研究

系统策略提供有关历史和当前业绩的统计信息。特别是可以确定净值增长，风险（以各种形式），交易频率和无数其他指标。可以在各种策略之间进行同类比较，从而可以最佳地分配资金。与主观交易中仅

设置跟踪损益（P&L）信息，掩盖潜在风险的情况形成了对比。

更高的交易频率

这是上面讨论的效率优势的必然结果。在自动化交易中，使得在许多市场上以更高频率运行的策略成为可能。实际上，一些最有利可图的交易策略在极限订单簿的超高频内运行。这些策略对于人类来说根本不可能实现。

2.1.2 缺点

虽然算法交易的优点很多，但也存在一些缺点。

资金要求

算法交易通常需要比用于主观交易要有更大的资金基础，是因为很少有经纪商支持自动交易接口而不需要大额账户的。零售自动化交易领域中最多的经纪商是盈透证券，其账户余额为 10,000 美元。情况正在缓慢变化，特别是当其他经纪商允许通过 FIX 协议直接连接时。此外，在某些保证金情况下，证券交易委员会定义的日内交易者要求在任何时候都要求至少维持 25,000 美元的账户净值。这些问题将在实现部分详细讨论。

此外，获得日内量化策略数据，特别是如果使用期货合约行情，对零售经纪商来说并不便宜。常见的零售日内行情价格在每月 300-500 美元的范围内。根据延迟需求不同，可能需要在交换机中共同定位服务器，这会增加每月的成本。对于日内交易者而言，这不一定是个问题，但值得考虑。还有一些辅助工具，例如更强大的网络速度和购买强大（因而昂贵的）高性能计算机。

编程/专业知识

虽然存在某些自动化算法交易平台，例如 Quantopian, QuantConnect 和 TradeStation，它们可以减少大部分编程难度，但有些系统尚未（截至撰写本文时）支持实时执行。在这种情况下，TradeStation 显然是个例外。因此，算法交易者需要在编程和科学建模方面相对熟练。

我试图展示各种各样的策略，其基础是以一种易于理解的方式。但是，如果您拥有数字建模技能，那么您可能会发现使用建模部分中的统计时间序列方法更容易。本书所演示的大多数技术已经在外部 Python 库中实现，这为我们节省了大量的开发工作。因此，我们生成算法交易系统时将“减少”将我们的数据分析和执行库联系的内容。

2.2 科学方法

本书中的交易策略设计完全基于科学方法的原理。科学方法的过程始于基于观察假设方法。在交易的背景下，一个例子是“SPDR Gold Shares ETF (GLD) 和 Market Vectors Gold Miners ETF (GDX) 之间是否存在关系？”。这可以形成解释观察到的行为。在这种情况下，假设可能是“GLD 和 GDX 之间的差异是否具有均值回归行为？”。如果是没有均值回归行为，即价差是随机游走。

在提出假设之后，科学家应该反驳假设错误并证明确实存在均值回归行为。为了实现这一点，必须定义预测。回到 GLD-GDX 示例，预测是表示两个 ETF 的扩散的时间序列是固定的。为了证明或反驳该假设，预测需要进行测试。在 GLD-GDX 的情况下，这意味着应用统计平稳性测试，例如 Augmented Dickey-Fuller, Hurst Exponent 和

Variance-Ratio Tests（在后续章节中详细描述）。

测试程序的结果将提供关于是否可以在一定置信水平下拒绝假设的统计答案。如果假设无法被证伪，这意味着两个 **ETF** 之间没有可辨别的关系，则该假设仍然可能（部分）为真。还可以测试更大的数据集，包含附加信息（例如影响价格的第三个 **ETF**）。这是分析过程。

使用科学方法进行交易策略设计的主要优点是，如果策略在前一个盈利期后“分解”，则可以重新审视初始假设并重新评估它，从而可能导致新的假设导致继续获得策略的盈利能力。

这与数据挖掘或黑箱方法形成鲜明对比，其中大量参数或“指示符”应用于时间序列。如果这样的“策略”最初是有利可图的，然后绩效恶化，很难确定原因。它经常导致策略可能暂时盈利但随后策略的指标或参数进一步下降。在这种情况下，策略通常被放弃，“研究”的过程再次继续。

在本书中，所有的交易策略都将采用观察假设的方法。

2.3 为何选择 Python？

以上部分概述了算法交易和科学方法的好处。现在是时候将注意力转向我们交易系统的实施语言。在本书中，我选择了 **Python**。**Python** 是一种专为提高开发速度而设计的高级语言。它拥有各种各样的库，几乎可以满足任何计算任务，也在资产管理和投资银行领域得到更广泛的应用。

以下是我选择 **Python** 作为交易系统研究和实施语言的原因：

- 学习 - 与其他语言（如 C++）相比，Python 非常容易学习。只需几周时间（有些人说几天！），您就可以非常高效地使用 Python。

- 库 - 使用 Python 的主要原因是它带有一个惊人的库数组，这大大减少了实现的时间以及将错误引入代码的可能性。特别是，我们将使用 NumPy（向量化操作），

SciPy（优化算法），pandas（时间序列分析），statsmodel（统计建模），scikit-learn（统计/机器学习），IPython（交互式开发）和 matplotlib（可视化）。

- 开发速度 - Python 在开发速度较高

有些评论说这就像写“伪代码”。像 IPython 这样的工具的交互性使得策略研究极其迅速，同时又不牺牲鲁棒性。

- 执行速度 - 虽然不如 C++ 快，但 Python 提供了大量优化的科学计算组件（通过矢量化）。如果执行速度成为一个问题，可以利用 Cython 并获得类似于 C 的执行速度，仅因代码复杂性会略有增加。

- 交易执行 - 大型经纪商存在 Python 插件，例如 Interactive Brokers（IBPy）。此外，Python 可以在必要时轻松使用 FIX 协议。

- 成本/许可 - Python 是免费的，开源的和跨平台的。它将在 Windows，Mac OSX 或 Linux 上运行愉快。虽然 Python 几乎适用于几乎所有形式的算法交易，但它无法在超高频交易（UHFT）领域与 C（或低级语言）竞争。

但是，这些类型的策略远远超出了本书的范围！

2.4 个人交易者仍然可以与机构竞争吗？

作为在散户层面实践的初始算法交易者，通常会质疑是否仍有可能与大型机构量化基金竞争。在本节中，有人会认为，由于制度监管环境的性质，组织结构和维持投资者关系的需要，这些资金存在一些与个人算法交易者无关的缺点。

对资金施加的资本和监管限制导致某些可预测的行为，这些行为能够被个人交易者利用。“大笔资金”推动了市场，因此人们可以想出许多策略来利用这些运动。其中一些策略将在后面的章节中讨论。现在将概述个人算法交易者对许多大型基金所享有的比较优势。

2.4.1 交易优势

个人算法交易者可以通过多种方式在交易过程中与基金竞争，但也有一些优劣：

- 规模 - 个人算法交易者在较小的市场中拥有更大的相对空间，可以在这些空间中产生显着的回报，但机构资金不能。
- 拥挤交易 - 基金可能会受到“技术转移”的影响。保密协议和非竞争协议缓解了这一问题，但仍导致许多量化基金“追逐同一笔交易”。异想天开的投资者情绪和“下一个热门事件”加剧了这个问题。个人算法交易者并不局限于遵循相同的策略，因此可能与较大的基金保持不相关。
- 市场影响 - 在高流动性，非场外交易市场中发挥作用时，个人账户的低资金规模会大幅降低本身对市场影响。
- 杠杆 - 个人交易者，根据其法律设置，受到保证金/杠杆法规的约束。私人投资基金并没有这样的劣势，尽管它们同样受到风险管理

的限制。

- 流动性 - 普通零售算法交易者无法接触到大宗经纪商。他们必须与 Interactive Brokers 等零售经纪公司“合作”。因此，某些工具的流动性减少。交易订单路径也不太清晰，是会导致策略绩效与回测不同。

- 新闻信息 - 个人交易者最大的劣势可能是无法从其主要经纪或信贷机构获得新闻信息。个人交易者必须利用非传统来源，如聚会小组，博客，论坛获取的金融期刊。

2.4.2 风险管理

与较大的量化基金相比，个人算法交易员通常采用不同的风险管理方法。在同样的风险的环境下，“小而敏捷”通常是有利的。

至关重要的是，除了他们自己施加的风险管理预算之外，没有风险管理预算，也没有合规或风险管理部门强制执行监督。这使得个人交易者可以自定义或任意选风险建模方法，而无需遵循“行业标准”（隐含的投资者要求）。

然而，另一种观点认为，这种灵活性可能导致个人算法交易者在风险管理方面变得“草率”。风险问题可以出现在回测和执行过程中，而不考虑整体投资组合风险。尽管“深度思考”可能适用于 alpha 模型（策略），但风险管理可能无法达到相似的考虑水平。

2.4.3 投资者关系

外部投资者是个人交易者和大型基金之间的主要区别。

- 薪酬结构 - 在个人交易环境中，交易者只关心绝对回报。没有高水

位标记，也没有资本配置规则需要遵循。个人交易者也能够承受更大波动的股票曲线，因为没有人观察他们的表现，也没有人从账户中赎回资金。

- 法规 and 报告 - 除了税收之外，个人交易者的监管报告限制几乎没有。此外，无需发送客户月度业绩报告或“装扮”投资组合。这节省了大量时间。

- 基准比较 - 基金不仅与同行进行比较，还与“行业基准”进行比较。例如，对于长期美国股票基金，投资者希望看到的回报超过标准普尔 500 指数。个人交易者的执行方式不同于将其策略与基准进行比较。

- 绩效费用 - 对于一个成功的量化基金，作为个人交易者运营自己的投资组合的缺点是没有管理费和绩效费用。个人交易是没有“2 和 20”管理费和绩效费用的！

2.4.4 技术

个人算法交易者具有显着优势的一个领域是为交易系统选择技术路径。交易者不仅可以选择“最适合工作的工具”，而且不用担心遗留系统问题或公司 IT 框架。较新的语言（如 Python 或 R）现在拥有构建端到端回测，执行，风险和项目组合管理系统的软件包，其代码行（LOC）远远少于 C++ 等更冗长的语言所需。 。

但是，这种灵活性需要付出代价。要么必须自己构建框架，要么自己构建框架将全部或部分内容外包给供应商。这在时间和成本两者方面都很昂贵。此外，交易者必须调试交易系统的所有方面 - 这是一个漫长而且潜在的艰苦过程。所有研究和服务器等必须直接从交易利润

中支付，因为没有管理费用来支付费用。

总之，可以看出个人算法交易者比较大的量化基金具有显着的比较优势。存在有许多方法可以利用这些优势。后面的章节将讨论利用这些差异的一些策略。

Part II

交易系统

第 3 章

成功的回测

算法回测需要许多领域的知识，包括心理学，数学，统计学，软件开发和市场/交易微观结构。我不可能在一章中涵盖所有这些主题，所以我将把它们分成两到三个小块。我们将在本节讨论什么？我将首先定义回测，然后我将描述它是如何实现的基础知识。然后，我将阐述我们在前几章中提到的问题。

在随后的章节中，我们将探讨在其他地方经常被提及或忽略的策略实施的细节。我们还将考虑如何通过包含交易所的特性来使回测过程更加真实。然后我们将交易成本以及如何在回测设置中正确建模。我们将最终讨论我们的测试结果，并最终提供常见量化策略的详细示例。

让我们首先讨论回测什么以及为什么我们应该在算法交易中实现它。

3.1 为什么回测策略？

算法交易与其他类型的投资类别不同，因为我们可以通过丰富的数据更可靠地对过去业绩的未来业绩的预期。执行此操作的过程称为回测。

简单来说，通过将特定策略算法暴露于历史数据流来执行回溯测试，这会导致一组交易信号。每笔交易（我们在这里指的是两个信号的“往返”）都会产生相关的利润或亏损。在策略回测期间累积此损

益将导致总利润和损失（也称为“损益”或“PnL”）。这就是回测思想的本质，“细节决定成败”！

回测算法策略的关键是什么？

- 过滤 - 如果您回忆起前一章关于策略识别的内容，我们在初始研究阶段的目标是建立一个策略框架，然后过滤掉任何不符合某些标准的策略。回测为我们提供了另一种过滤机制，因为我们可以消除不符合我们性能需求的策略。

- 建模 - 回测允许我们（安全地！）测试某些市场现象的新模型，例如交易成本，订单，延迟，流动性或其他市场微观结构问题。

- 优化 - 虽然策略优化充满了问题，但回测允许我们通过修改与该策略相关的参数的数量或值并重新计算其性能来提高策略的绩效。

- 验证 - 我们的策略通常来自外部，通过我们的策略框架。虽然我们很少能够识别策略生成的个别信号，但我们可以用绩效指标，例如夏普比率和回撤特征，可以确保策略没有被错误地实现。

回测为算法交易提供了许多优势。但并不总是能够直接回撤策略。

一般而言，随着策略交易的频率增加，正确地模拟市场和交易微观结构效应变得更加困难。这导致不太可靠的回测，因此要对所选策略进行较为复杂的评估。这是一个特殊的问题，实现系统是策略性能的关键，就像超高频算法一样。

不幸的是，回测充满了各种类型的问题，我们现在将深入讨论它们。

3.2 回测偏差

有许多偏见会影响策略的绩效。不幸的是，这些偏见倾向于夸大绩效

而不是贬低绩效。因此，您应该始终考虑将回测作为策略实际绩效的理想上限。消除算法交易的偏差几乎是不可能的，因此我们的工作就是尽可能地最小化它们，以便对我们的算法策略做出明智的决策。

我想讨论四个主要偏见：优化偏差，前向偏差，幸存者偏差和认知偏差。

3.2.1 优化偏差

这可能是所有回测测偏差中最隐蔽的。它涉及调整或引入额外的交易参数，直到回测数据集的策略性能非常有吸引力。但是，一旦实时策略的表现可能会有明显的不同。这种偏差的另一个名称是“曲线过拟合”或“数据窥探偏差”。

优化偏差很难消除，因为算法策略通常涉及许多参数。在这种情况下，“参数”可以是输入/输出标准，回测时间段，平均周期（即移动平均平滑参数）或波动率测量频率。通过将参数数量保持在最小值并增加回测数据数量，可以最小化优化偏差。事实上，人们也必须小心后者，因为较旧的数据可能受制于先前的制度（例如监管环境），因此可能与您当前的策略实现无关。

一种有助于缓解这种偏差的方法是进行敏感度分析。这意味着逐步改变参数并绘制绩效表。在考虑所有其他因素的情况下，参数选择的合理的基本推理应该导致更平滑的参数表面。如果您有一个非常跳跃的绩效表，通常意味着参数不反映现象（译者按：这里面讲的是要测试策略的泛化能力即在不同或相邻数据组下的策略表现）。有关于

多维优化算法的大量文献，它是一个高度活跃的研究领域。我不会在这里详述，但保持这些内容在你的脑海里，尤其是当你发现一个梦幻般的回测的策略时！

3.2.2 前向偏差

当未来数据意外地包含在模拟中实际上不可用的数据点时，将向后测试系统引入预测偏差。如果我们按时间顺序运行后测并且我们达到时间点 N ，那么如果包含任何点 $N + k$ 的数据，则会出现前向偏差，其中 $k > 0$ 。前向偏差可能非常微妙。以下是如何引入前向偏差的三个示例：

- 技术错误 - 代码中的数组/向量通常具有迭代器或索引变量。这些索引的错误偏移可以通过在 $N + k$ 处并入非零 k 的数据而导致前向偏差。（译者：数据错误，尤其是未清洗的数据造成的错误）

参数计算 - 发生前向偏差的另一个常见例子

计算最佳策略参数，例如两个时间序列之间的线性回归。如果整个数据集（包括未来数据）用于计算回归系数，并因此追溯应用于交易策略以用于优化目的，那么将来的数据被合并并且存在前向偏差。（译者：未来函数）

- 最大值/最小值 - 某些交易策略在任何时间段都使用极值，例如在 OHLC 数据中加入高价或低价。但是，由于这些最大值/最小值只能在一个时间段结束时计算，因此如果使用这些值，则会引入前向偏差。在使用它们的任何交易策略中，始终有必要将开仓设置在 `close`

的下一个 tick。（译者：使用了一些理想值）

与优化偏差一样，必须非常小心以避免其引入。这通常是交易策略在“实时交易”中表现不佳的主要原因。

3.2.3 幸存者偏差

幸存者偏差是一种特别危险的现象，可能会导致某些策略类型的性能显着下降。当在不包括特定时间点选择的先前资产的完整范围的数据集上测试策略时，会发生这种情况，但仅考虑那些“幸存”到当前时间的数据集。

例如，考虑在 2001 年市场崩盘之前和之后对随机选择的股票进行策略测试。一些科技股破产了，而其他科技股则设法维持下去甚至繁荣。如果我们将这一策略仅限于那些通过市场下跌期活下去的公司股票，我们会引入幸存者偏差，因为他们已经向我们证明了他们的成功。事实上，这只是前向偏差的另一个具体情况，因为未来的信息正被纳入过去的分析中。

在策略回测中，有两种主要方法可以减少幸存者偏差：

- 幸存者无偏见数据集 - 在股票数据的情况下，可以购买包含已删除退市公司的数据集，尽管它们并不便宜且仅倾向于由机构公司使用。特别的是，雅虎财经数据不免费剔除幸存者偏差，而这是许多个人算法交易者常用的。人们还可以交易不存在生存偏差的资产类别，例如某些大宗商品期货（及其衍生品）。
- 使用更多近期数据 - 在股票的情况下，利用更新的数据集可以减少选择的股票选择权重于“幸存者”的可能性，因为在较短的时间段内

整体股票退市的可能性较小。人们还可以通过从当前点开始收集数据来开始构建个人幸存者 - 无偏差数据集。3 - 4 年后，您将拥有一套可靠的股票数据，可进一步用于回测的策略。

3.2.4 认知偏差

在量化交易的背景下，不经常讨论这种特殊现象。但是，对于更多的主观交易方法，我们进行了广泛的讨论。在 5 年或更长时间内创建回测时，很容易查看向上趋势的权益曲线，计算复合年回报，夏普比率甚至回撤特征并对结果感到满意。例如，该策略可能具有 25% 的最大相对回撤和 4 个月的最大回撤持续时间。对于动量策略而言，这是典型的。直截了当地说服自己很容易忍受这样的损失期，因为总体情况是乐观的。然而，在实践中它要难得多！（译者：【捂脸】）

如果在回测中发生 25% 或更多的历史回撤，那么很可能您会在实时交易中看到类似的回撤期。这些回撤期在心理上难以忍受。我已经在机构环境中亲眼观察过这种情况，并且它并不令人愉快 - 即使后测表明会出现这样的时期。我之所以称之为“偏见”的原因在于，在下跌期间，通常会取消成功的策略停止交易，因此与回测相比会导致显著的表现不佳。因此，即使策略本质上是算法，心理因素仍然会对盈利能力产生重大影响。需要注意的是，如果您在回测中看到一定百分比和持续时间的下降，那么您应该期望它们在实时交易环境中发生，并且需要坚持下去才能再次获得盈利。

3.3 交易问题

3.3.1 订单类型

算法交易者必须做出的一个选择是如何以及何时利用可用的不同交易订单类型。这种选择通常属于执行系统的范畴，但我们在这里考虑它，因为它可以极大地影响策略回测性能。可以执行两种类型的订单：市价订单和限价订单。

无论可用价格如何，市价单立即执行交易。因此，作为市价订单执行的大比交易通常会得到不确定的成本价，因为另一侧的每个后续限价订单都被交易（直到挂出的资金用尽）。市价订单被认为是激进的订单，因为它们几乎肯定会被交易，尽管成本可能未知。

限价订单为策略确定交易执行的最差（基本）价格提供了一种机制，但需要注意的是交易可能无法部分或完全交易。限价订单被视为被动订单，能够执行的价格才进行交易。交易所的限价订单集合称为限价订单簿，它本质上是一定数量和价格的买卖订单。

在回测时，必须对正确使用市价或限价订单进行建模。特别是对于高频策略，如果因市价和限价订单的影响没有准确建模，则会导致回测明显优于实时交易。

3.3.2 价格数据整理

当以开盘 - 高 - 低 - 收盘（OHLC）数字形式使用日常数据时，特别是对股票而言，回测策略存在特殊问题。请注意，这正是雅虎财经提供的的数据形式，这是个人算法交易者非常常见的数据来源！

廉价或免费的数据集，通常来自不同的交易所的综合价格容易产生幸存者偏差（我们已经在上面讨论过）。这意味着由于区域交易所的小订单，数据的极值点（即开盘价，收盘价，最高价和最低价）非常

容易受到“偏离”价值的影响。此外，这些值有时可能是尚未从数据集中删除的数据错误。

这意味着，如果您的交易策略特别广泛使用任何 OHLC 价格，则回测效果可能与实时表现不同，因为订单可能会根据您的经纪商和您可用的访问权限发送到不同的交易所。解决这些问题的唯一方法是利用更高频率的数据或直接从单个交易所获取数据，而不是更便宜的综合数据。

3.3.3 外汇交易和 ECN

实施外汇策略回测比股票策略更难实现。外汇交易发生在多个场所和电子交易平台（ECN）。在一个交易场所实现的买/卖价格可能与另一交易场所的买/卖价格大不相同。必须非常小心地利用您在回测中交易的特定交易所的价格信息，而不是来自多个交易场所的合并数据，这有助于使用更加准确的数据。

外汇市场的另一个特质是经纪商没有义务与每个交易参与者分享交易价格/规模，因为这是他们的专有信息。因此，在您的回测中使用买卖报价更为合适，并且要非常小心经纪人/交易所之间交易成本的变化。

3.4 交易成本

实施交易模型时最常见的初学者错误之一是忽略（或严重低估）交易成本对策略的影响。虽然通常假设交易成本仅反映经纪人佣金，但事实上还有许多其他方式可以在交易模型上产生成本。必须考虑的三种主要成本包括：

3.4.1 佣金

算法交易策略产生的最直接的交易成本形式是佣金和费用。所有策略都需要某种形式的交易，直接或通过经纪中介（“经纪人”）。这些服务会在每笔交易中产生增量成本，称为佣金。

经纪人通常提供许多服务，尽管量化算法只使用交易席位，因此，每笔交易的经纪佣金往往很小。经纪商代为收取其他费用，比如清算和结算交易所产生的费用。此外还有地方或国家政府征收的税收。例如，在英国，股票交易要收取印花税。由于佣金，费用和税收通常是固定的，因此在回测引擎中实施它们相对简单（见下文）。

3.4.2 滑点

滑点是交易策略系统决定交易的时间与交易实际进行交易的时间之间的价格差异。滑点是交易成本的重要组成部分，可以使一个很有利策略成为表现不佳的策略。滑点是标的资产波动，交易系统与交易所之间的延迟以及正在执行的策略类型的函数。

具有较高波动性的资产有更大的滑点，因此交易信号和执行之间的价格可能会有很大差异。延迟定义为信号生成和执行点之间的时间差。更高频率的策略对延迟问题更敏感，并且这种延迟时间改进对盈利能力产生重大影响。交易策略的类型也很重要。动量策略平均受到滑点的影响更大，因为他们试图购买已经在预测方向上移动的工具。对于均值回归策略则相反，因为这些策略正朝着反方向交易。

3.4.3 市场影响

市场影响是交易者因交易（和资产）影响到市场而产生的成本。相对

低流动性市场里面，交易占当前市场买/卖的大部分，大订单可能会大幅推动市场。为了解决这个问题，当新增流动性到达交易所时，大块交易被分解成更小的“块”，这些“块”定期交易。另一方面，对于高流动性工具，如 S&P500 E-Mini 指数期货合约，低交易量交易不太可能大幅调整“当前价格”。

低流动性资产的特点是价差更大，是限价订单簿上（译者按：买卖档行情）当前买入价和卖出价之间的差额，此差价是与交易相关的额外交易成本。点差是总交易成本的一个非常重要的组成部分 - 无数的英国点差交易公司证明了这一点，其很多广告表达了大量交易品种的点差“紧张”。

3.5 回测与现实

总之，为了生成逼真的回测，需要考虑一系列因素。过度拟合，糟糕的数据清理，错误的交易成本处理，市场制度变化和交易限制的问题往往导致回测性能与实时策略部署大不相同。

因此，必须非常清楚未来的表现不太可能直接与历史表现相匹配。当我们在本书末尾实现一个事件驱动的回测试引擎时，我们将进一步详细讨论这些问题

第 4 章

自动化交易

自动化交易是让策略自动生成执行信号的过程，这些信号在没有任何人为干预的情况下发送给经纪商。这是最纯粹的算法交易策略，因为它最大限度地减少了人为干预引起的问题。这是我们在本书中最常考虑的系统形式。

4.1 回测平台

策略回测的编程可选类型很多。解决方案范围从完全集成的机构级复杂软件到编程语言（如 C++，Python 和 R），几乎所有内容都必须从头开始编写（或者获得合适的“插件”）。作为量化交易者，我们感兴趣的是能够“拥有”我们的交易软件与我们的开发方法的速度和可靠性之间的平衡。以下是软件选择的关键考虑因素：

- 编程技巧 - 环境的选择在很大程度上取决于您编写软件的能力。我认为使用外包供应商软件对您的长期 PnL 有不利的影响。这是因为您无法在供应商软件中修复外部错误或特定的下行风险，如果您对“技术堆栈”有更多控制权，则可以轻松解决这些问题。您还需要一个能够在生产环境，库的可用性和执行速度之间取得适当平衡的环境。
- 执行能力/经纪商交互 - 某些回测软件，如 Tradestation，直接与经纪商绑定。我并不喜欢这种方法，因为降低交易成本通常是获得更高夏普比率的重要因素。如果你被捆绑到一个特定的经纪商（并且

Tradestation “强迫”你这样做），那么如果有需要你很难过渡到新软件（或新的经纪人）。盈透提供了一个优秀的 API，尽管它有一个稍微迟钝的界面。（译者按：だいじょうぶ？确定这不是个软广告？）

- 自定义 - 像 MATLAB 或 Python 这样的环境在创建算法策略时为您提供了极大的灵活性，因为它们可以为您提供几乎任何可想象的数学运算和出色的库，也可以在必要时进行广泛的自定义。

- 策略复杂性 - 某些软件并不适用于繁重的数字运算或复杂数学函数。Excel 就是这样一款软件，虽然它对于处理简单的策略是很简单的，但它无法真正快速地处理大量资产或更复杂的算法。

- 偏差最小化 - 特定的软件或数据是否更适合交易？如果您想确保自己创建所有功能准确性。

- 开发速度 - 人们不应该花费数月和数月来开发回测引擎。原型设计应该只需要几周时间。确保您的软件开发不会在很大程度上妨碍您的进度。

- 执行速度 - 如果您的策略完全依赖于执行的及时性（如在 HFT / UHFT 中），那么将需要诸如 C 或 C++ 之类的语言。但是，您要了解诸如 Linux 内核优化和 FPGA 使用情况，这超出了本书的范围。

- 成本 - 您可以使用算法交易策略编程的许多软件环境都是完全免费且开源的。事实上，许多对冲基金都在其整个算法交易软件中都使用开源软件。此外，Excel 和 MATLAB 都相对便宜可以自由选择。

不同的策略需要不同的软件包。HFT 和 UHFT 策略将用 C / C++ 编写。

目前，这些策略通常在 GPU 和 FPGA 上执行。相反的，软件/经纪商

的“一体化”特性，在 TradeStation 中易于实施低频方向性股票策略。

4.1.1 编程

在测试策略时，好的编程语言中定制开发回测平台可提供最大的灵活性。相反，供应商开发的集成回测平台将始终对回测的执行方式做出假设。可用编程语言的选择是多种多样的，在开发之前，哪种语言适合，这一点并不好确定。

一旦将策略编入系统，就必须以特定方式对其进行回测，即量化交易者要确信其策略未来表现将反映其过去表现。通常有两种形式的回测系统用于测试该模型。从广义上讲，它们被划分为研究回测和事件驱动回测。

4.1.2 研究工具

通常首先考虑回测工具的简单形式，即研究工具。该研究工具用于快速确定策略是否具有可操作性。这些工具通常不能对交易成本，填补价格，卖空限制，交易所，风险管理以及前一章中概述的许多其他问题做出合理假设。常用的研究工具包括 MATLAB, R, Python 和 Excel。软件包提供向量化运算，从而使得策略在执行速度和实现（较少的代码行）的研究阶段快速实现，这是非常有用的。以便于快速，迭代的方式测试多种策略，组合。

虽然这些工具通常用于回测和执行，但这样的研究环境通常不适合于以更高频率（亚分钟）的日内交易的策略。因为这些工具通常不具备连接到实时市场数据供应商服务器，或者可以更直接的方式与经纪 API 连接。（译者按：这里面说的有点问题）

尽管存在这些执行上的缺点，但研究工具在专业量化领域中被大量使用。 它们是所有策略构想的“第一环”，然后在现实的回测环境中进行更严格的检查（译者按：这里应指实盘交易）。

4.1.3 事件驱动的回测

一旦策略被认为适合进一步研究，就必须以更现实的方式进行测试。这种实现是试图解决前一章中描述的大多数（如果不是全部）问题。理想的情况是能够使用相同的交易代码进行历史回测和实时执行。可以使用事件驱动方式来实现。

事件驱动系统广泛用于软件工程中，通常用于处理基于窗口的操作系统内的图形用户界面（GUI）输入。 它们也是算法交易的理想选择。这些系统通常用高性能语言编写，如 C++，C# 和 Java。

考虑一种自动交易策略与实时市场交易和经纪商相关的情况（这两者可能是同一个）（译者按：有些是由经纪商撮合交易，这和交易所类型不同有关）。 新的市场信息将被发送到系统，触发和事件以产生新的交易信号，从而产生执行事件。 因此，这样的系统处于连续循环中，等待接收事件并适当地处理它们。

可以生成一些组件，例如历史数据处理程序和经纪商模拟器，可以模仿它们的实时处理。这使得回测策略更像实时执行的形式进行。这种系统的缺点在于它们比简单的研究工具设计和实现复杂得多。因此，“开发时间”更长。 他们更容易出错并需要合理的编程知识，并且在某种程度上需要软件开发的方法。（译者按：回测更像是简单脚本，这种就很庞杂了）

4.1.4 延迟

在工程术语中，延迟定义为模拟和响应之间的时间间隔。我们通常将指代执行信号的生成与从执行执行的代理接收信息之间的往返时间延迟。

这种延迟在低频率的日间策略中很少出现问题，因为在延迟期间可能的价格变动不会在很大程度上影响策略。不幸的是，高频策略不适如此。在高频率下，延迟变得很重要，所以我们的目标是尽可能减少延迟，以尽量减少滑点，如前一章所述。

减少延迟涉及最优化算法交易系统与执行订单的最终交易之间的“距离”。这可能涉及缩短系统之间的物理距离（减少网络布线的传输时间），减少网络硬件中的处理（在 HFT 策略中很重要）或选择具有更优秀的设备的经纪商。

作为“因特网距离”（即两个服务器之间的网络距离）的函数，减少等待时间是呈指数级增加的。因此，对于高频交易者，必须在延迟减少的支出与最小化滑点的增益之间达成折衷。这些问题将在下面的主机托管部分中讨论。

4.1.5 语言选择

已经概述了语言选择的一些问题。现在我们将考虑各种编程语言的优缺点。我将语言大致分为高性能/更难开发和低性能/更容易开发。这些是主观术语，有些人会根据他们的背景不同而产生异议。

编程自定义回测环境最重要的一个方面是程序员熟悉所使用的工具。这可能是一个比开发速度更重要的标准。但是，对于那些对编程语言领域不熟悉的人来说，下面的内容应该阐明算法交易中常用的内容。

C ++, C # 和 Java

C ++, C # 和 Java 都是通用的面向对象编程语言的例子。也就是说，它们可以在没有相应 IDE 的情况下使用，都是跨平台的（可以在 Windows, Mac OSX 或 Linux 上运行），有广泛的，几乎任何可以想象的任务的库范围和拥有快速执行速度。

如果需要很高的执行速度，那么 C ++（或 C）可能是最佳选择。它为管理内存和优化执行速度提供了最大的灵活性。这种灵活性需要付出代价，众所周知，C ++很难学好，并且经常会导致细微的错误，开发时间可能比其他语言花费更长的时间。

C # 和 Java 类似，它们都要求所有组件都是对象，除了浮点数和整数等基本数据类型。，它们与 C ++的不同之处在于它们都执行自动垃圾回收。这意味着在销毁对象时不必手动取消分配内存。垃圾回收增加了性能开销，它使开发速度更快。因为它们具有本机 GUI 功能，数值分析库和快速执行速度，这些语言都是开发回测不错选择。

就个人而言，我会利用 C ++来创建一个事件驱动的交易实现器，它需要极快的执行速度，例如 HFT。只有当我觉得 Python 事件驱动系统成为瓶颈时才会这样，因为后一种语言将是我对这种系统的首选。

MATLAB, R 和 Python

MATLAB 是用于数值计算的商业集成开发环境（IDE）。它在学术，工程和金融领域得到了广泛的认可。它具有大量用于许多科学计算任务的数值库，并具有快速执行速度，所开发的任何算法都受向量化或并行化的影响。**MATLAB** 使其对预有限的个人交易者不那么有吸引力。**MATLAB** 通常直接嵌入执行到经纪商，如盈透证券。

R 不是通用编程语言，而是更多的统计脚本环境。它是免费的，开源的，跨平台的，包含大量可自由使用的统计软件包，用于执行极其先进的分析。**R** 在量化对冲基金行业中被广泛用于统计/策略研究。虽然可以将 **R** 连接到经纪商，但 **R** 不太适合这项任务，应该更多地将其作为研究工具。此外，除非操作是向量化的，否则它执行速度较慢。

我在这个标题下写了 **Python**，位于 **MATLAB**，**R** 和上述通用语言之间。它是免费的，开源的和跨平台的，它因为需要解释器与直接编译比，使得它本身比 **C++** 慢。尽管如此，它还包含各种库，用于执行几乎任何可以想象的任务，从科学计算到低级 **Web** 服务器设计。特别是，它包含 **NumPy**，**SciPy**，**pandas**，**matplotlib** 和 **scikit-learn**，提供了一个强大的数值研究环境，在向量化时，与编译语言执行速度相当。此外，它还拥有成熟的封装接口，可以连接经纪公司。这使得它成为创建事件驱动的回测和实时执行环境的“一站式商店”，而无需涉及其他语言。执行速度对于日内交易者按时间分钟交易而言是足够的。最后，与 **C++** 等低级语言相比，非常容易学习。出于这些原因，我们在本书中广泛使用了 **Python**。

4.1.6 集成开发环境

IDE 这个术语在算法交易中有多重含义。软件开发人员使用它来表示允许使用语法突出显示，文件浏览，调试和代码执行功能进行编程的 GUI。 算法交易者使用它来表示完整的集成回测/交易环境，包括历史或实时数据下载，图表，统计评估和实时执行。 出于我们的目的，我使用该术语来表示不是通用编程语言的任何环境（通常是基于 GUI 的），例如 C++ 或 Python。 例如，MATLAB 被认为是一个 IDE。

Excel

虽然一些更纯粹的 Quant 可能会看不起 Excel，但我发现它对于结果的“健壮性检查”非常有用。事实上所有数据都是直接可用的，而不是隐藏在对象后面，因此可以直接实现非常基本的信号/滤波器策略。 经纪商，如盈透证券，也允许 DDE 插件，允许 Excel 接收实时市场数据和执行交易订单。

尽管易于使用，但对于任何一定规模的数据或数值计算 Excel 都非常慢。 我只是在与其他策略进行开发时使用它进行错误检查，并确保我们避免前向偏差，由于软件的电子表格特性，这在 Excel 中很容易看到。

如果您对编程语言感到不适应并且正在执行日间策略，那么 Excel 可能是最佳选择。

商业/零售回测软件

个人图表市场（译者按：比如通达信，同花顺），“技术分析”和回测软件极具竞争力。 这些软件提供的功能包括价格的实时图表，丰

富的技术指标，定制的回测测试和自动执行。

一些供应商提供一体化解决方案，例如 TradeStation。 TradeStation 是一家在线经纪公司，生产交易软件（也称为 TradeStation），提供跨多个资产类别的自动化订单执行。目前他们不提供自动执行的直接 API，而是必须通过软件下订单。 这与拥有更多精简图表/交易界面（Trader WorkStation）的盈透证券形成鲜明对比，但同时提供其专有的实时市场/订单执行 API 和 FIX 接口

另一个非常受欢迎的平台是 MetaTrader，它用于外汇交易以“Expert Advisors”的形式。 这些是使用专有语言（easy language）编写的自定义脚本，可用于自动交易。 我对 TradeStation 或 MetaTrader 没有多少经验，所以我不会花太多时间讨论它们的优点。

如果您不熟悉软件开发并希望处理大量细节，则此类工具非常有用。然而，使用这样的系统会牺牲很多灵活性，而且你经常与特定的经纪商绑定在一起。

基于 Web 的工具

目前流行的两种基于网上的回测系统是 Quantopian（<https://www.quantopian.com/>）和 QuantConnect（<https://www.quantconnect.com/>）。前者使用 Python（和 ZipLine，见下文），而后者使用 C#。两者都提供了丰富的历史数据。Quantopian 目前支持盈透证券交互进行实时交易，而 QuantConnect 则致力于实时交易。

开源回测软件

除商业产品外，还有用于回测软件的开源替代方案。

Algo-Trader 是一家总部位于瑞士的公司，系统提供开源和商业许可。从我可以收集的内容来看，这个产品似乎已经非常成熟，他们拥有许多机构客户，该系统有完整的历史回测组件和复杂的事件处理，并与盈透证券相关联。企业版提供了更高性能的功能。

Marketcetera 提供了一个回测系统，可以绑定许多其他语言，例如 Python 和 R，以便利用您可能已编写的代码。“**StrategyStudio**”提供了编写回测代码以及优化执行算法的功能，并从历史回测过渡到实时交易。

ZipLine 是支持上述 **Quantopian** 服务的 Python 库。这是一个完全由事件驱动的回测环境，目前支持美国股市。我没有使用 **ZipLine**，但其他人认为它是一个很好的工具。虽然仍有许多方面需要改进，但团队一直致力于该项目，因此它得到了很积极的维护。

还有一些您可能研究 **Github / Google Code** 上面的托管项目。我没有花费大量时间来调查它们。此类项目包括 **OpenQuant**（<http://code.google.com/p/openquant/>），**TradeLink**（<https://code.google.com/p/tradelink/>）和 **PyAlgoTrade**（<http://gbeced.github.io>）/ pyalgotrade/）。

机构回测软件

个人算法交易者通常不会使用机构级的回测系统，例如 **Deltix** 和 **QuantHouse**。购买软件通常远远超出个人算法交易者预算。话虽如此，这种软件被量化基金，自营交易公司，家族办公室等广泛使用。

这种系统的好处是显而易见的。 它们为单个品种或产品组合的数据采集，策略开发，历史回测和实时执行提供了一体化解决方案， 这些平台已经进行了广泛的测试，并且有大量的“现场”使用，因此被认为是强大的。

这些系统是事件驱动的，因此回测环境通常可以很好地模拟实时环境。 该系统还支持优化的执行算法，这些算法试图最小化交易成本。 我不得不承认除了一些粗略的概述之外，我对 Deltix 或 QuantHouse 没有多少经验。 话虽这么说，单单是预算使软件与多数个人交易者无缘，所以我不会详述这些系统。

4.2 服务器托管

现在已经赘述完算法交易的软件环境。现在是时候将注意力转向实施我们策略的硬件的实施。

个人交易者可能会在交易时间在家里执行他们的策略，开启他们的 PC，连接到经纪人，初始化交易软件，然后让算法在白天自动执行。相反，具有资产管理（AUM）的专业量化基金将拥有专用特定位置的服务器，以尽可能减少延迟，以执行其高速策略。

4.2.1 家用台式机

最简单的硬件部署方法是简单地通过宽带（或类似）连接，家庭台式计算机连接到经纪商来实现算法策略。

虽然这种方法很容易上手，但它确实存在许多缺点。 除非由 UPS 备份，否则台式机主要会出现电源故障。 此外，家庭互联网连接也受到 ISP 的影响。 在交易的关键时刻可能发生断电或互联网连接故障，

使算法交易者处于无法平掉未平仓头寸境地。

其次，由于操作系统的可靠性，必须偶尔重启台式机（译者按：国内要每天都重启），这意味着该策略受到一定程度的间接人工干预（译者按：非交易时段重启即可，是完全不会影响的）。如果这发生在交易时间之外，问题就会减轻。但是，如果计算机在交易时间内需要重新启动，则问题类似于断电。未平仓的头寸可能仍有风险。

计算机组件故障也会导致同样的“停机”问题。硬盘，显示器（译者按：作者你是认真的？显示器不被这个锅啊！）或主板故障可能会发生。出于所有这些原因，我不愿意推荐用家庭电脑进行算法交易。如果您决定采用这种方法，请确保备用计算机和备用互联网连接（如 3G dongle）可用于在停机情况下启动。

4.2.2 VPS

从家用台式机的下一级是利用虚拟专用服务器（VPS）。VPS 是一种远程服务器系统，通常称为“云”服务。它们比相应的专用服务器便宜得多，因为 VPS 实际上是更大服务器的分区，您可以使用虚拟隔离操作系统环境。CPU 负载在多个服务器之间共享，系统一部分 RAM 分配给 VPS。

常见的 VPS 提供商包括 Amazon EC2 和 Rackspace Cloud。它们提供具有低 RAM 和基本 CPU 的入门级系统，以及企业级高 RAM，高 CPU 服务器。对于大多数个人零售交易者而言，入门级系统足以满足低频日内或日间策略和较小的历史数据数据（译者：这里面指的历史数据数据是指策略运算缓存所需的数据，而非我们理解的回测数据）。

基于 VPS 的系统的优势包括全天候可用性（具有一定的实际停机时间！），更强大的监控功能，易于“插件”的附加服务，如文件存储或托管数据库以及灵活的架构。缺点包括系统费用，因为专用硬件每个性能变得便宜得多，托管远离交易所，以及处理故障等情况。

此外，通过选择 VPS /云提供商并不总能提高延迟。您的位置可能比云提供商的数据中心更接近特定的金融交易所。通过选择一家专门为算法交易服务的，在交易所或交易所附近提供 VPS 的公司，可以稍微减轻这种影响，但这些公司的成本可能高于亚马逊或 Rackspace 等“传统” VPS 提供商。

4.2.3 交易服务器

为了获得最佳的延迟最小化和最快的系统，有必要直接在交易数据中心设置一个专用服务器（或一组服务器）。对于几乎所有个人算法交易者来说，这是一个非常昂贵的选择（除非他们的资本非常充足）。它确实是专业量化基金或经纪业务的领域。

如上所述，更现实的选择是从位于交易所附近的提供商处购买 VPS 系统。我不会过分关注交换机托管，因为这个主题有点超出了本书的范围。

第 5 章

策略思想源泉

在本章中，我想向您介绍我自己确定有利可图的算法交易策略方法。我们将讨论如何查找，评估和选择此类策略。我将解释如何确定策略与个人偏好匹配，因为它涉及策略绩效，如何确定回测历史数据的使用，如何冷静地评估交易策略以及最后如何进行回测阶段和策略实施。

5.1 识别您自己的交易个人偏好

为了成为一个成功的交易者 - 无论是在主观交易上还是在量化交易上 - 都有必须诚实回答一些的问题。交易可能为您提供了惊人的赔钱速度（译者按：作者皮这一下很快开心啊），因此有必要“了解您自己”，因为有必要了解您所选择的策略。

我想说交易中最重要因素是了解自己的个性。特别是主观交易和算法交易都需要相当程度的纪律，耐心和理性。由于您让算法为您执行交易，因此有必要解决在执行时不干扰策略的问题。这可能非常困难，特别是在长期回撤期间。然而，许多策略在回测中被证明是高利润的，但可能通过简单的干扰来破坏。如果你想进入算法交易的世界，你将受到理性测试，为了获得成功，有必要解决这些困难！

下一个要考虑的问题是时间。你是全职工作还是兼职工作？你

每天在家工作还是长途通勤？ 这些问题将有助于确定您应该寻求的策略的频率。对于那些全职工作的人来说，日内期货策略可能不合适（至少在它完全自动化之前！）。 您的时间限制也将决定策略的方法。如果您的策略经常交易并且依赖于昂贵的新闻源（例如彭博终端），那么您想办公室成功运行此策略要切合实际！ 对于那些有很多时间的人，或者自动化策略，您可以研究一种更技术性的高频交易（HFT）策略。

我的看法是有必要对您的交易策略进行持续研究，以维持持续盈利的投资组合。 很少有策略永远处于“under the radar”。因此，分配给交易的大部分时间都在进行持续研究上。 问问自己是否准备好这样做，这可能是强劲的盈利能力或缓慢的亏损之间的差异。

您还需要考虑您的交易本金，普遍接受的量化策略的理想最低金额是 50,000 美元（英国我们约 35,000 英镑）。如果我是我，我需要更大的金额开始，可能接近 100,000 美元（约 70,000 英镑）。 这是因为对于中高频策略而言，交易成本可能非常昂贵，并且必须有足够的资金来稀释它们。 如果您考虑从低于 10,000 美元开始，那么您将需要限制自己使用低频策略，交易一两个交易品种，因为交易成本会极大的影响您的回报（译者按：国内外可能有所差异，如果差别不大，交易所不会考虑返佣（国内期货），而经纪商手续费都压得很低）。 盈透经纪公司是拥有编程技能的最友好的经纪商之一，由于其 API，其零售账户最低为 10,000 美元。

编程技能是创建自动算法交易策略的重要因素。熟悉 C ++, Java,

C#，Python 或 R 等编程语言，您可以自己创建端到端数据存储，回测引擎和执行系统。这具有许多优点，其中最主要的是能够完全了解交易设备的所有方面。它还允许您探索更高频率的策略，因为您将完全控制“技术堆栈”。虽然这意味着您可以测试自己的软件并消除错误，但这也意味着花费更多的时间来编写基础架构，而不是实施策略，至少在您的算法交易生涯的早期阶段。您可能会发现自己熟悉 Excel 或 MATLAB 交易，并可以外包其他组件的开发。但是我不建议这样做，特别是要从事高频交易的人。

您需要问自己你希望通过算法交易实现什么。您是否对一般收益感兴趣，您希望从您的交易账户中获取收益多少？或者，您是否对长期资本收益感兴趣并且能够在不需要提取资金的情况下进行交易？是否提取也将决定您的策略频率，盈利提取将需要更高频率的交易策略，波动性更小（即更高的夏普比率）。长期交易者可以承受较低的交易频率。

5.2 算法交易理念源泉

尽管普遍存在不同的看法，在公共领域找到有利可图的交易策略实际上非常简单。从来没有比现在更容易获得交易思想。学术金融期刊，预印本论文，交易博客，交易论坛，每周交易杂志和专业文本提供了数千种交易策略，以此为基础构建您的想法。

我们作为量化交易研究人员的目标是建立一个策略框架，为我们提供一系列持续的交易思路。理想情况下，我们希望创建一种有条理的方法，评估和实施我们的策略。该方法的目标是产生一定数量的新

想法，并为我们提供一个框架，这些想法以最理性的方法考虑问题。我们必须非常小心，不要让认知偏见影响我们的决策。这就像对一种资产类别优先于另一种资产类别（黄金和其他贵金属）一样简单，因为它们被认为更特别。我们的目标始终是找到持续盈利的策略，并给予积极的期望。资产类别的选择应基于其他考虑因素，例如交易资金约束，经纪费用以及杠杆能力。

5.2.1 书单

如果您完全不熟悉交易策略和金融市场的概念，那么首先要看的是已有的教科书。经典理论提供了各种更简单，更直接的想法，用于熟悉量化交易。这是我推荐给那些刚接触量化交易的人的书单，在列表中按照复杂程度排序的。

金融市场介绍

以下列表详细介绍了资本市场如何运作并描述现代电子交易的书籍。

- Glen Arnold 撰写的 “*Financial Times Guide to the Financial Markets*” — 本书专为金融市场新手设计，对于深入了解所有市场参与者非常有用。它为我们提供了一个市场列表，我们可以在这些市场中形成算法交易策略。

- Trading and Exchanges: Market Microstructure for Practitioners*—这是一本描述关于金融市场参与者及其运作方式极为丰富的书。它提供了交易方式的重要细节，参与者的各种动机以及市场的监管方式。虽然有些人可能认为它是 “dry reading”，但我认为理解这些概念对于一个优秀的算法交易者绝对是必要的。

Algorithmic Trading and DMA: An introduction to direct access trading strategies—约翰逊的书

更倾向于市场的技术方面。 它讨论了订单类型，最佳执行算法，接受算法交易的交易类型以及更复杂的策略。 与哈里斯上面的书一样，它详细解释了电子交易市场的运作方式，我也相信这些知识是实施自动化策略的必要先决条件。

量化交易

下面一组书直接涉及算法/量化交易。 他们概述了一些基本概念，并描述了可以实施的特定策略。

- **Quantitative Trading: How to Build Your Own Algorithmic Trading Business**——Ernest Chan 的第一本书是量化交易策略的“初学者指南”。 虽然它对策略思想并不重视，但它确实提供了如何建立交易业务逻辑的框架，包括风险管理模块和实施工具。 如果您对算法交易完全陌生，这本书是一本很好的书。 这本书主要是使用 **MATLAB**。
- **Algorithmic Trading: Winning Strategies and Their Rationale** by Ernest Chan[6] - Chan 的第二本书对策略思想非常重视。 它基本上是从上一本书中断的地方开始，并更新市场条件的内容。 本书讨论了日间和日间频率的均值回归和基于动量的策略。 它还简要介绍了高频交易。 与前一本书一样，它广泛使用 **MATLAB** 代码。
- **Inside The Black Box: The Simple Truth About Quantitative and High-Frequency Trading, 2nd Ed** by Rishi Narang—Narang 的书概述了定量对冲基金所使用的交易系统的组成部分，包括 **alpha** 生成器，风险管理，投资组合优化和交易成本。 第二版详细介绍了高频交易技术。
- **Volatility Trading** by Euan Sinclair[16] - Sinclair 的书专注于波动率建

模/预测和旨在利用这些模型进行期权策略交易。如果您打算以量化方式交易期权，那么本书将提供许多研究思路。

5.2.2 互联网

在通过经典文章获得算法交易的基础之后，可以从互联网上获得额外的策略思想。量化金融博客和交易论坛等都提供了丰富的想法来测试。

但需要注意的是：许多互联网的交易策略都依赖于技术分析概念。技术分析涉及利用基本信号分析指标和行为心理学来确定资产价格的趋势或反转模式。

尽管在整个交易领域非常受欢迎，但技术分析在量化金融界被认为存在争议。有些人认为，就预测能力而言，并不比最好算命或研究玄学！实际上，有成功的个人在交易中广泛使用技术分析。

作为具有更复杂的数学和统计工具箱的 quant，我们可以轻松评估这种“TA-based”策略的有效性我们能够做出由数据分析和假设检验驱动的决策，而不是基于情感考虑或先入为主的决策。

Quant 博客

我推荐以下量化博客，以获得有关算法交易的良好交易思路和理念：

- MATLAB Trading - <http://matlab-trading.blogspot.co.uk/>
- Quantitative-Trading(ErnestChan) - <http://epchan.blogspot.com>
- Quantivity - <http://quantivity.wordpress.com>
- Quantopian - <http://blog.quantopian.com>
- Quantpedia - <http://quantpedia.com>

平台

在过去几年中，主题内容被聚合然后讨论已成为时尚。我阅读了以下平台：

- Quantocracy - <http://www.quantocracy.com>
- Quant News - <http://www.quantnews.com>
- Algo rading Sub-Reddit - <http://www.reddit.com/r/algotrading>

论坛

寻找其他策略思想的下一个地方是交易论坛。不要被更多“技术分析”导向的策略所拖累。这些策略仅是提供可以进行统计测试的好主意：

- Elite Trader Forums - <http://www.elitetrader.com>
- Nuclear Phynance - <http://www.nuclearphynance.com>
- QuantNet - <http://www.quantnet.com>
- Wealth Lab - <http://www.wealth-lab.com/Forum>
- Wilmott Forums - <http://www.wilmott.com>

5.2.3 期刊文献

如果您在评估简单策略方面有了一些经验，现在是时候看一下更复杂的学术内容了。如果没有高额的费用用以订阅或一次性买断，一些学术期刊将难以访问。如果您是大学的学生或校友，您应该能够访问其中一些金融期刊。否则，您可以查看预打印网站，这些网站是正在进行同行评审的学术论文的最新草稿的互联网存储库。由于我们只对能够成功复制，回测和获取盈利能力的策略感兴趣，因此同行评审对我们而言并不重要。

学术策略的主要缺点是它们往往可能过时、需要昂贵的历史数据、低流动性资产类别的交易或没有费用，滑点或差价。不清楚交易策略是用市价订单，限价订单还是包含止损订单等来执行的。因此，尽可能自己复制策略，对其进行回测后再并添加实际交易是绝对必要的，同时要考虑您希望交易的资产类别的多个方面的成本。

以下是您可以从以下方面获取思路且流行的预打印网站和金融期刊的列表：

- arXiv - <http://arxiv.org/archive/q-fin>
- SSRN - <http://www.ssrn.com>
- Journal of Investment Strategies
-<http://www.risk.net/type/journal/source/journalof-investment-strategies>
- Journal of Computational Finance
-<http://www.risk.net/type/journal/source/journalof-computational-finance>
- Mathematical Finance
-<http://onlinelibrary.wiley.com/journal/10.1111/%28ISSN%291467-9965>

5.2.4 独立研究

如何形成自己的量化策略？这通常需要（但不限于）以下一个或多个类别的专业知识：

- 市场微观结构 - 特别是对于超频率的策略，可以利用市场微观结构，即了解订单动态以产生盈利能力。不同的市场将具有各种技术

限制，法规，市场参与者，这些都可以进行开发特定策略。这是一个非常复杂的领域，个人从业者将难以在这个领域竞争，特别是因为竞争者中包括具有强大技术能力的充足资金的量化对冲基金。

- 基金结构 - 投资基金，如养老基金，私人投资合伙企业(对冲基金)，商品交易顾问和共同基金受到严格监管及其庞大资金规模限制。因此，某些一致的行为可以被那些更敏捷的人利用。例如，大型基金因其规模而受到产能限制。因此，如果他们需要迅速出售一定数量的证券，他们将不得不拆单它以避免“推动市场”。尖端算法可以在基金结构套利的过程中利用诸如此类的特性。

机器学习/人工智能 - 机器学习算法近年来在金融市场中变得越来越普遍。分类器（例如 **Naive-Bayes** 等人）非线性函数匹配器（神经网络）和优化算法（遗传算法）都被用于预测资产走势或优化交易策略。如果您具有此领域的背景知识，您可能会对特定算法如何应用于某些市场有所了解。

通过每周甚至每天持续分析上述资料，您可以从各种来源获得一致的策略。下一步是筛选这些策略，以便最大限度地减少对可能无利可图的策略进行回测，防止浪费时间。

5.3 评估交易策略

第一个也是最重要的考虑因素是您是否真正理解了这个策略。您能够简明扼要地解释策略，还是需要一系列风险揭示和无穷无尽的参数列表？此外，该策略在现实中是否具有良好和坚实的理论基础？例如，您是否可以指出某些可能让您可以利用的模式行为作为理由？

这种行为是否会因政策而改变，例如严格监管环境导致策略中断？

策略是否依赖于复杂的统计或数学规则？ 它是否适用于任何金融时间序列，还是特定的有利可图的金融资产类别？ 在评估新的交易策略时，您应该不断考虑这些因素，否则您可能会浪费大量时间来尝试回测并优化无利可图的策略。

一旦确定您了解策略的筛选原则，您需要确定它是否符合您前面提到的风险回报水平。策略在性能特征方面会有很大差异。 某些人风险回报水平可以忍受更长时间的回撤，或者愿意接受更大的回报风险。 尽管我们作为 quant 尝试消除尽可能多的认知偏差，并且应该能够冷静地评估策略，但是偏见总是会蔓延。因此，我们需要一种一致的方法来评估策略的表现。 以下是我通过以下方式判断潜在新策略的标准列表：

- 方法论 - 策略原理是基于均值回归，市场中性，趋势的策略，是否依赖于难以理解的复杂统计或机器学习技术，是否需要掌握统计学的博士学位？

这些技术是否会引入大量参数，这是否会导致优化偏差？ 该策略是否可能抵御政策变化（即金融市场潜在的新监管）？

- 夏普比率 - 夏普比率更深入的表现了策略的收益/风险比率。它量化了您可以通过多大承受的波动率而产生的收益曲线能达到多少回报。 当然，我们需要确定这些回报和波动率（即标准差）的周期和频率。 例如，更高频率的策略将需要更高的标准差采样率和更短的总体测试时间段。

•杠杆 - 策略是否需要很高的杠杆才能实现盈利？该策略是否需要使用杠杆衍生品合约（期货，期权，掉期）才能获得回报？这些杠杆合约可能具有很大的波动特征，因此很容易导致追加保证金。你有忍受这种波动的交易资金和勇气吗？

•频率 - 策略的频率与您的技术水平（以及技术专业知识），夏普比率和总体交易成本水平密切相关。考虑所有其他问题，更高频率的策略需要更多资金，更复杂，更难以实施。但是，这种策略基于您的复杂的回测引擎并且没有错误，那么它们的夏普比率通常会高得多。

•波动率 - 波动率与策略的“风险”密切相关，夏普比率表现这一点。如果未做对冲，标的资产又是较高波动率通常会导致收益曲线的波动性增加，从而导致夏普比率变小。我当然假设正波动率大致等于负波动率。而一些策略可能具有更大的下行波动性，您需要了解这些属性。

•盈亏比，平均盈利/亏损 - 策略的盈亏比和平均盈亏特征会有所不同。即使亏损交易的数量超过获胜交易的数量，也可以有一个非常有利可图的策略。动量策略往往具有这种特征，因为它们依赖于少量的“大热门”才能获利。均值反转策略往往相反，更多是交易中的“赢家”，但亏损时也可能非常严重。

•最大回撤 - 最大回撤是该策略的权益曲线上最大的总值 - 低值百分比下降。众所周知，动量策略会受到长期回撤的影响（由于一系列亏损交易）。许多交易者将在长期回撤期间放弃，即使历史回测表明这是该策略是“一切照旧”。在停止交易策略之前，您需要确定可以

接受的亏损百分比（以及在什么时间段内）。这是一个非常私人化的决定，因此必须仔细考虑。

- 策略容量 - 在个人层面，除非您参与低流动性金融产品（如小型股票）交易，否则您不必过多关注容量对自己策略的影响。容量决定了策略的可扩展性，随着资本资金对策略的增加。许多较大的对冲基金都面临着严峻的容量问题。

- 参数 - 某些策略（尤其是机器学习社区中的策略）需要大量参数。策略所需的每个额外参数都使其更容易受到过拟合（也称为“曲线拟合”）的影响。您应该尝试使用尽可能少的参数来制定策略，或者确保您有足够数量的数据来测试您的策略。

- 比较基准 - 几乎所有策略（除非被称为“绝对回报”）都是根据某些绩效基准来衡量的。基准通常是一个指数，描述了该策略交易品种的大量样本。如果该策略交易大盘美国股票，那么标准普尔 500 指数将成为衡量您的策略的自然基准。您将听到术语“alpha”和“beta”，适用于此类策略。

请注意，我们尚未讨论该策略的实际回报。为什么是这样？单单说收益回报实际上只为我们提供了策略有效性的有限信息。它们不会让您深入了解杠杆率，波动性，基准或资金要求。因此，很少仅根据回报判断策略。在查看回报之前，请始终考虑策略的风险属性。

在这个阶段，从您的脑海中发现的许多策略将被否定，因为它们将无法

满足您的收益要求，杠杆约束，最大回撤或波动率偏好。现在可

以考虑剩余的策略进行回测。但是，在此之前，有必要考虑一个最终标准 - 可用于测试这些策略的历史数据。

5.4 获取历史数据

如今，各品种的历史数据需求是十分巨大的，要求也很高。为了保持竞争力，买方（基金，柜台）和卖方（经纪商/交易商）都在其技术上投入巨资。必须考虑其重要性。特别是我们对及时性，准确性和存储有一定要求。我们将在本书后面的章节中讨论历史数据。

在上一节中，我们建立了一个策略标准，允许我们根据自己的个人情况来筛选某些策略。在本节中，我们将根据自己的需求获取历史数据过滤更多的策略。我们主要考虑因素（特别是在个人从业者层面）是数据的成本，存储需求和您的技术专业水平。我们还会讨论不同类型的可用数据以及每种类型的数据我们要考虑的不同因素。

让我们首先讨论可用数据的类型以及我们需要考虑的关键问题，并理解我们将在本书深入探讨这些问题：

- 基本数据 - 包括有关宏观经济的数据，如利率，通胀数据，公司行为（派股，派息），SEC 文件，公司账户，收益数据，农作物报告，气象数据等。这些数据经常被用来从根本上对公司或其他资产进行估值，即通过某种预期未来现金流量的方式来预测价格。一些基本数据可从政府网站免费获得。但其他长期历史基础数据可能非常昂贵。存储要求通常不是特别大，除非同时对数千家公司进行研究。

- 新闻数据 - 新闻数据通常具有定性性质。它包括文章，博客文章，微博帖子（“推特”）和社论。通常机器学习（如分类器）用于舆情

分析。通过订阅媒体渠道，这些数据通常也是免费或廉价的。较新的“**NoSQL**”文档存储数据库可以存储此类非结构化定性数据。

- **资产价格数据** - 这是 **quant** 的传统数据分析领域。由资产价格的时间序列组成。股票，固定收益产品（债券），商品和外汇价格都属于这一类。对于较简单的资产类别（例如股票），每日历史数据通常很容易获得。但是，一旦要获得准确性和清洁度很高并消除了统计偏差的数据，就会变得昂贵。此外，时间序列数据通常具有显着的存储要求，尤其是在考虑日内数据时。

- **金融工具** - 股票，债券，期货和更为不同的衍生品期权具有截然不同的特征和参数。因此，没有“一刀切”的数据库结构可以容纳它们。必须非常谨慎地设计各种金融工具的数据库结构。

- **频率** - 数据频率越高，成本和存储要求就越高。对于低频策略，每日数据通常就足够了。对于高频策略，可能有必要获得特定细粒度的交易订单簿数据。为此类数据实现存储的技术技术要求很高，仅适用于具有强大编程/技术背景的人员。

- **基准** - 策略通常会与基准进行比较。这通常表现为其他的金融时间序列（基准）。对于股票而言，这通常是国家股票基准，例如标准普尔 500 指数（美国）或 **FTSE100**（英国）。对于固定收益基金，与一篮子债券或固定收益产品进行比较是有用的。“无风险利率”（即适当的利率）也是另一个被广泛接受的基准。所有资产类别都拥有基准，因此如果您希望从外部获得对策略的比较，则有必要根据您的特定策略对此进行研究。

- 技术 - 金融数据存储中心背后的技术很复杂。但是，它通常以数据库集群引擎为中心，例如关系数据库管理系统（RDBMS），如 MySQL，SQL Server，Oracle 或文档存储引擎（即“**NoSQL**”）。这可以通过“业务逻辑”应用程序代码访问，外部工具可以访问数据库（如 MATLAB，R 或 Excel）。通常，这种业务逻辑是用 C ++，Java 或 Python 编写的。您还需要在某个地方托管此数据，当然可以储存在您自己的个人计算机上，也可以通过 Internet 服务器远程托管。近年来，亚马逊网络服务等产品使这一过程变得更加简单和便宜，但仍需要大量的技术专业知识才能以安全的方式实现。

可以看出，一旦通过框架识做出策略，就有必要评估特定历史数据集的可用性，成本，复杂性和实现细节。您可能会发现有必要仅根据历史数据考虑因素否定策略。这是一个很大的工程，很多大型基金雇佣博士团队就是为了确保定价准确和及时。不要低估为回测和创建强大的数据中心的难度！

但我想说，付出一定代价，许多回测平台可以自动为您提供这些数据。因此，您可以完全专注于策略实现和优化。像 TradeStation 这样的工具就拥有这种能力。但是，我个人的观点是尽可能自己做，并避免将部分代码外包给软件供应商。我更喜欢高频率的策略，因为它们具有更具吸引力的夏普比率，但它们通常与技术紧密结合，其中高级优化至关重要。

Part III

数据平台开发

第 6 章

软件安装

本章将详细讨论如何安装算法交易环境。操作系统选择被视为必要的第一步，概述了三个主要选择。我们选择 **Linux** 作为首选系统（尤其是 **Ubuntu**），并且 **Python** 安装了所有必需的库。

软件包/库的安装经常被其他书籍所忽略，但我个人认为它可能成为许多人的绊脚石所以我已经花了整整一章。不幸的是，现实是该章将在它发布的那一刻变得过时。因为会出现更新版本的操作系统，和不断更新软件包。因此，有些具体的实施细节可能会过时。

如果您在安装或使用这些软件包时遇到问题，请务必检查已安装的版本并在必要时进行升级。如果您仍有问题，请随时给我发电子邮件 mike@quantstart.com，我会尽力帮助您。

6.1 操作系统选择

决定算法交易平台时的第一个事是操作系统。在某种意义上，这将由主要编程语言或连接到经纪商的方式决定。目前，大多数软件，特别是开源软件，都是跨平台的，因此选择受到的限制较少。

6.1.1 Microsoft Windows

Windows 可能是许多算法交易者的“默认”选项。这是大家非常熟悉，尽管有很多批评的声音，但在某些环境下依然相当强大。

Windows 8 并没有得到很好的应用，但是之前的版本 Windows 7 是一个可靠的操作系统。算法交易行业中的某些工具仅在 Windows 上运行，特别是 IQFeed 服务器，这是从 DTN IQFeed 下载 tick 数据所必需的。此外，Windows 是 Microsoft .NET 框架的本机平台，使用 C++ 和 C# 编写大量金融软件。

如果您不希望使用 Windows，则有时可以使用 WINE 模拟器

(<http://www.winehq.org/>) 在基于 UNIX 的系统下运行基于 Windows 的软件。

6.1.2 Mac OSX

Mac OSX 结合了 Windows 的图形易用性（有人说它大大改进了它！）与 UNIX 系统（FreeBSD）的健壮性。虽然我使用 MacBook Air 进行所有“日常”工作，例如网络/电子邮件和开发 QuantStart 网站，但我发现在 Mac 下安装基于 Python 进行算法研究是非常痛苦。

Mac OSX 的软件包非常分散，Homebrew 和 MacPorts 是主要的竞争者。由于专有的编译过程（使用 XCode），但从源代码安装很棘手。在撰写本文时，我尚未在 MacBook 上成功安装 NumPy, SciPy 和 pandas！但是，如果您可以在 Mac OSX 上导航 Python 安装的问题，它可以为算法研究提供良好的环境。由于 IB Trader Workstation 是基于 Java 的，因此在 Mac OSX 上运行没有问题。

6.1.3 Linux

“Linux”是指一组免费的 UNIX 发行版，如 Cent OS, Debian 和 Ubuntu。我不想详细介绍每个发行版的优点/缺点，而是集中讨论基于 Debian

的发行版。特别是我将考虑将 Ubuntu Desktop 作为算法交易环境。
aptitude 包管理使得安装很简单，必要的底层库变得简单。此外，为 Python 创建虚拟环境非常简单，可以将您的算法交易代码与其他 Python 应用程序隔离开来。我从未在 Ubuntu 系统上安装 Python 环境时遇到任何（重大）问题，因此我选择此作为进行交易的主要环境。如果您想在完全提交之前给 Ubuntu 一个例子，例如通过双启动，那么就可以使用 VirtualBox (<https://www.virtualbox.org/>) 来安装它。我有一个关于 QuantStart 的详细指南

(<http://www.quantstart.com/articles/Installing-a-DesktopAlgorithmic-Trading-Research-Environment-using-Ubuntu-Linux-and-Python>)，它描述了这个过程。

6.2 在 Ubuntu Linux 上安装 Python 环境

在本节中，我们将讨论如何使用 Ubuntu Desktop Linux 和 Python 编程语言为算法交易策略研究建立一个强大，高效和交互式的开发环境。我们将利用此环境进行所有后续算法交易实现。

为了创建研究环境，我们将安装以下软件工具，所有这些工具都是开源的，可以免费下载：

- Ubuntu Desktop Linux - 操作系统
- Python - 核心编程环境
- NumPy / SciPy - 用于快速，高效的向量化/矩阵计算
- IPython - 用于使用 Python 进行可视化交互式开发
- matplotlib - 用于数据的图形可视化

- pandas - 用于数据分析和时间序列分析
- scikit-learn - 用于机器学习和人工智能算法
- IbPy - 使用 Interactive Brokers API 进行交易

这些工具与合适的 MySQL 证券数据库相结合，将使我们能够创建快速的交互式策略研究和回测环境。Pandas 专为“数据分析”而设计，可以非常有效地导入和清理时间序列数据。NumPy / SciPy 在运行时可以使系统得到极好的优化。IPython / matplotlib（以及下面描述的 qtconsole）允许交互式可视化结果和快速迭代。scikit-learn 让我们将机器学习技术应用于我们的策略以进一步提高性能。

6.2.1 Python

最新版本的 Ubuntu，在撰写本文时为 13.10，仍然使用 Python 2.7.x 版本系列。虽然正在进行 3.3.x 的过渡，但大多数库与 2.7.x 分支完全兼容。因此我选择将其用于算法交易。算法可能会迅速发展，但在几年内 3.3.x 可能是主要的分支。我们现在开始安装 Python 环境。任何全新的 Ubuntu Linux 系统要做的第一件事就是更新和升级软件包。前者告诉 Ubuntu 有关可用的新软件包，而后者实际上执行的是使用较新版本替换旧软件包的过程。在终端会话中运行以下命令，系统将提示您输入密码：

```
sudo apt-get -y update  
sudo apt-get -y upgrade
```

请注意，-y 前缀告诉 Ubuntu 您要对所有是/否问题接受“是”。

“sudo”是一个 Ubuntu / Debian Linux 命令，允许以管理员权限执行其他命令。安装软件包，我们需要对计算机进行“root 访问”，因

此必须使用“sudo”。

一旦这两个更新命令都成功执行，我们需要安装编译所有软件所需的 Python 开发包和编译器。 请注意，我们正在安装 build-essential，其中包含 GCC 编译器和 LAPACK 线性代数库，以及 pip，它是 Python 包管理系统：

```
sudo apt-get install python-pip python-dev python2.7-dev \
build-essential liblapack-dev libblas-dev
```

下一步是安装 Python 数值和数据分析库。

6.2.2 NumPy, SciPy 和 Pandas

一旦安装了必要的软件包，我们就可以继续通过 Python 软件包管理器 pip 安装 NumPy。pip 将下载该软件包的 zip 文件，然后从源代码中为我们编译它。 请记住，编译需要一些时间，可能需要 10 分钟或更长时间，具体取决于您的 CPU：

```
sudo pip install numpy
```

一旦安装了 NumPy，我们需要在继续之前检查它是否有效。 如果您查看终端，您将看到您的用户名，后面是您的计算机名称。 在我的情况下，它是 mhallsmoore @ algobox，然后是提示符。 在提示符下键入 python，然后尝试导入 NumPy。 我们将通过计算列表的平均值来测试它的工作原理：

```
mhallsmoore@algobox:~$ python
Python 2.7.4 (default, Sep 26 2013, 03:20:26)
[GCC 4.7.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> from numpy import mean
>>> mean([1,2,3])
2.0
>>> exit()
```

现在已经成功安装了 NumPy，我们想要安装称为 SciPy 的 Python Scientific 库。它有一些自己的包依赖，包括 ATLAS 库和 GNU Fortran 编译器，必须先安装它们：

```
sudo apt-get install libatlas-base-dev gfortran
```

我们现在准备好用 pip 安装 SciPy。这需要很长时间才能编译，也许 10-20 分钟，具体取决于 CPU 速度：

```
sudo pip install scipy
```

SciPy 现已安装。在计算整数列表的标准差，我们将以类似于 NumPy 的方式测试它：

```
mhallsmoore@algobox:~$ python
Python 2.7.4 (default, Sep 26 2013, 03:20:26)
[GCC 4.7.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import scipy
>>> from scipy import std
>>> std([1,2,3])
0.81649658092772603
>>> exit()
```

本节的最后一项任务是安装 pandas 数据分析库。我们在此阶段不需要任何其他依赖项：

```
sudo pip install pandas
```

我们现在可以像以前一样测试 pandas：

```
>>> from pandas import DataFrame
>>> pd = DataFrame()
>>> pd
Empty DataFrame
Columns: []
Index: []
>>> exit()
```

现在已经安装了基础数值和科学计算库，我们将安装统计和机器学习

库，statsmodels 和 scikit-learn。

6.2.3 Statsmodels 和 Scikit-Learn

安装过程和以前一样，使用 pip 安装软件包：

```
sudo pip install statsmodels
sudo pip install scikit-learn
```

两个库都要测试：

```
mhallsmoore@algobox:~$ python
Python 2.7.4 (default, Sep 26 2013, 03:20:26)
[GCC 4.7.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> iris
..
..
'petal width (cm)']}]
>>>
```

现在已经安装了两个统计库，我们可以安装可视化和开发工具，

IPython 和 matplotlib。

6.2.4 PyQt, IPython 和 Matplotlib

第一个任务是为 matplotlib 安装依赖包，Python 图形库。由于 matplotlib 是一个 Python 包，我们不能使用 pip 来安装底层库来处理 PNG，JPEG 和 freetype 字体，所以我们需要 Ubuntu 来安装它们：

```
sudo apt-get install libpng-dev libjpeg8-dev libfreetype6-dev
```

现在我们可以安装 matplotlib：

```
sudo pip install matplotlib
```

本节的最后一项任务是安装 IPython。这是一个交互式 Python 解释器，与使用标准 Python 控制台相比，它提供了更加简化的工作流程。在后面的章节中，我们将强调 IPython 在算法交易开发中的全部用处：

```
sudo pip install ipython
```

虽然 IPython 本身就非常有用，但它可以通过 `qtconsole` 来提供更强大的功能，`qtconsole` 提供了内嵌 `matplotlib` 可视化的功能。但是，要使其正常运行需要更多的工作。

首先，我们需要安装 Qt 库：

```
sudo apt-get install libqt4-core libqt4-gui libqt4-dev
```

`qtconsole` 有一些额外的依赖包，即 ZMQ 和 Pygments 库：

```
sudo apt-get install libzmq-dev  
sudo pip install pyzmq  
sudo pip install pygments
```

通过键入以下命令来测试 IPython 是很简单的：

```
ipython qtconsole --pylab=inline
```

要测试 IPython，可以通过键入以下命令生成简单的绘图。请注意，这里已经包含了您不需要键入的 IPython 编号输入/输出：

```
In [1]: x=np.array([1,2,3])  
  
In [2]: plot(x)  
Out[2]: [<matplotlib.lines.Line2D at 0x392a1d0>]
```

这应该显示 `matplotlib` 图。关闭 IPython 让我们继续安装。

6.2.5 IbPy 和交易平台

盈透经纪商是零个人算法交易者使用的主要经纪商之一，因为其相对较低的最低账户余额要求（10,000 美元）和（相对）直接的 API。在本节中，我们将安装 IbPy 和 Trader Workstation，我们稍后将使用它来执行自动交易执行。

我想强调的是，我们不会进行任何实时交易！我们只是安装一些软

件，让我们试用一个“模拟账户”，让它以“实时”的方式为市场模拟器提供历史的数据。注：我与 Interactive Brokers 没有任何关系。我以前在专业的基金公司中使用它们，所以熟悉它们的软件。

IbPy 是一个围绕基于 Java 的 Interactive Brokers API 编写的 Python 包装器。它使得在 Python 中开发算法交易系统的问题稍微少一些。它将用作随后与 Interactive Brokers 进行的所有通信的基础。另一种方法是使用 FIX 协议，但我们不会在本书中考虑这种方法。

由于 IBPy 是在 GitHub 源代码版本控制网站上维护的，因此作为 git 存储库，我们需要安装 git。这由以下方式处理：

```
sudo apt-get install git-core
```

一旦安装了 git，就必须创建一个子目录来存储 IBPy。它只需放在主目录下：

```
mkdir ~/ibapi
```

下一步是通过'git clone'命令行下载 IBPy：

```
cd ~/ibapi
git clone https://github.com/blampe/IbPy
```

最后一步是进入 IbPy 目录并使用 Python setuptools 进行安装：

```
cd ~/ibapi/IbPy
python setup.py.in install
```

这样就完成了 IBPy 的安装。下一步是安装 Trader Workstation。在撰写本文时，有必要遵循此链接（IB），它将您直接带到 Interactive Brokers 的 Trader Workstation 下载页面。选择您要使用的平台。在这个例子中，我选择了 UNIX 下载，可以在这里找到（IB Unix 下载）。

在该链接中，它将描述该过程的其余部分，但我将在此复制以获得它。

下载的文件名为 `unixmacosx_latest.jar`。 打开文件：

```
jar xf unixmacosx_latest.jar
```

然后切换到 `IBJts` 目录并加载 TWS：

```
cd IBJts
java -cp jts.jar:total.2013.jar -Xmx512M -XX:MaxPermSize=128M jclient.LoginFrame .
```

这将显示 `Trader Workstation` 登录屏幕。 如果您选择用户名“`edemo`”和密码“`演示用户`”，您将登录系统。

这样就完成了在 `Python` 和 `Ubuntu` 下安装完整的算法交易环境。 下一阶段是开始收集和存储我们策略需要的历史数据。

第 7 章

金融数据存储

在算法交易中，焦点通常投向整个交易系统的 **alpha** 模型组件。在风险管理和投资组合构建系统过滤之前，该组件生成交易信号。因此，算法交易员通常会花费大部分研究时间来改进 **alpha** 模型，以便在将策略部署到投入生产前优化一个或多个指标。

但是，**alpha** 模型只能与输入数据的数据一样。这个概念的特点是计算机科学格言“垃圾进，垃圾出”。准确，及时的数据用于提供给 **alpha** 模型绝对至关重要。否则结果最好是差或者便是最坏完全不错误。当系统实时部署时，将导致严重的策略性能不佳。

在本章中，我们将讨论有关获取数据和为算法策略回测系统及最终交易实现引擎提供及时准确数据的问题。特别是我们将研究如何获取金融数据以及如何存储它。后续章节将讨论如何清理它以及如何导出它。在金融行业中，这种类型的数据服务被称为证券数据库。

7.1 证券数据库

证券数据库是一个涵盖很广的数据库，存储各资产类别各种金融工具的基本面，价格和交易数据。它以统一的方式提供对此信息的访问，以供各部门使用，例如风险管理，清算/结算和自营交易。在大型公司中，以下是一些公司要存储数据基于不同金融产品如下：

- 股票

- 股票期权
- 指数
- 外汇
- 利率
- 期货
- 商品
- 债券 - 政府和企业
- 衍生品 - 期权，掉期

证券数据库通常由开发人员和数据专家组成的团队确保金融机构内的高可用性。虽然这在大公司，个人层面或小型基金中都是必要的。但证券管理人要用到的比实际上简单得多。实际上，虽然大型证券管理人使用昂贵的企业数据库和分析系统，经过优化开源软件也能提供相同级别的功能。

7.2 金融数据集

对于个算法交易者或小型量化基金，最常见的数据库是股票，指数，期货（主要是商品或固定收益）和外汇（外汇）的当日和日内历史价格。为了简化此讨论，我们将仅关注股票，ETF 和股票指数的日终（EOD）数据。后面的部分将讨论添加更高频率数据，另类资产类别和衍生数据，这些数据具有更高级的要求。

股票的 EOD 数据很容易获得。有许多服务可通过网络 API 免费获得：

- Yahoo Finance - <http://finance.yahoo.com>
- Google Finance - <https://www.google.com/finance>

- QuantQuote - <https://www.quantquote.com> (S&P500 EOD data only)
- EODData - <http://eoddata.com> (requires registration)

手动下载单个证券的历史数据非常简单，但如果需要每天下载许多股票，则会变得非常耗时。因此，我们证券组件的一个重要组成部分将自动更新数据集。

另一个问题是回测期。我们要选择多长时间使用我们的数据进行回测？这将特定于您的交易策略的要求，但是所有策略的都存在某些问题。最常见的是市场变化，其特点往往是新的监管环境，波动性更高/更低的时期或长期单边趋势市场。例如，长期的短期趋势跟踪/动量策略可能在 2000-2003 或 2007-2009 期间表现良好。然而，从 2003 年到 2007 年或 2009 年到现在，这将是艰难的时期。

我的经验是尽可能多地获取数据，特别是对于便宜的 EOD 数据。而且仅仅储存在主数据中存在数据，并不意味着必须使用它。因为较大的数据库意味着更长的查询时间（见下文），但拥有更多样本的好处解决任何策略评估问题。

与金融数据一样，要注意数据错误，例如不正确的高/低价格或幸存者偏差，我在前面的章节中详细讨论过。

7.3 存储格式

存储金融数据有三种主要方式。它们都具有不同的访问，性能和结构性能。我们将依次分析每一个。

7.3.1 平面文件存储

最简单的金融数据数据存储以及您可能从任何数据供应商接收数据

的方式是平面文件格式。平面文件通常使用逗号分隔变量（CSV）格式，它将二维数据矩阵存储为一系列行，列数据通过分隔符分隔（通常是逗号，空格，）。对于 EOD 定价数据，每行代表交易日的 OHLC（即交易期间的开盘价，最高价，最低价和收盘价）。

平面文件的优点是它们足够简单和大量压缩以便存档或下载的能力。主要缺点在于它们缺乏查询能力以及跨大型数据集的迭代性能差。

SQLite 和 Excel 通过提供某些查询功能来缓解这些问题。

7.3.2 文档存储/ NoSQL

文档存储/ NoSQL 数据库虽然不是一个新概念，但由于它们被用于谷歌，Facebook 和 Twitter 等“规模网络”公司，近年来已经获得了显着的突出地位。它们与 RDBMS 系统有很大不同，因为没有表模式的概念。相反，有集合和文档，它们分别是与表和记录最接近的类比。存在广泛的文档存储分类，其讨论完全在本章之外！一些比较受欢迎包括 MongoDB，Cassandra 和 CouchDB。

金融应用程序中的文档存储主要适用于基础数据或单元数据。金融资产的基本数据有多种形式，例如公司行为，收益报表，SEC 文件等。因此，NoSQL DB 的无模式特性非常适合。但是，NoSQL DB 并不适用于高分辨定价数据等时间序列，因此我们不会在本章中进一步分析它们。

7.3.3 关系型数据库管理系统

关系型数据库管理系统（RDBMS）利用关系型模型来存储数据。这些数据库特别适合于财务数据，因为不同的“对象”（例如交易，数

据源，价格）可以分成表格，并在它们之间定义关系。

RDBMS 使用结构化查询语言（SQL）来对金融数据执行复杂的数据查询。RDBMS 的主要包括 Oracle，MySQL，SQLServer 和 PostgreSQL。

RDBMS 的主要优点是安装简单，平台独立，易于查询，易于与测试集成以及高性能（尽管有些人认为后者并非如此！）。它们的缺点通常是由于操作复杂和实现所述性能的困难，而没有关于如何存储 RDBMS 数据的介绍。此外，它们具有半刚性模式，因此通常必须修改数据以适应此类设计。这与 NoSQL 数据存储区不同，NoSQL 数据存储区没有架构。

对于本书中所有未来的历史价格代码实现，我们将使用 MySQL RDBMS。它是免费的，开源的，跨平台的，高度可靠的，并且其大规模的充分验证其可靠性，这使其成为量化工作的明智选择。

7.4 历史数据结构

在计算机科学领域进行了大量的理论和学术研究，以实现数据存储的最优设计。但我们不会涉及太多细节，因为很容易迷失在细枝末节中！相反，我将提出一个构建股票主数据的通用模式，您可以根据自己的应用程序修改它。

第一项任务是定义我们的需求，这些需求是最终将映射到数据库中的表的财务数据的元素。对于股票主数据库，我预见到以下需求：

- 交易所 - 数据的原始来源是什么？
- 供应商 - 从哪个特定供应商获得？
- 金融工具/股票代码 - 股票或指数的代码，以及相关公司或基金的

公司信息。

- 价格 - 特定日期特定证券的实际价格。
- 公司行为 - 股票拆分或股息调整（可能会产生一个或多个表格）。
- 法定假日 - 为避免将交易日错误归类为缺少数据错误，存储国家法定假日和交叉引用会很有用。

关于数据存储规范代码存在重大问题。 我通过对冲基金处理这个问题的第一手经验来证明这一点！ 不同的供应商使用不同的方法来清洗数据，从而拼接和交叉多个来源以获得准确性。 此外，公司破产，接受并购活动（即更改名称/符号），拥有多个公开交易的股票类别。 你们中的许多人不必担心这一点，因为你们的代码范围将仅限于较大的指数成分（例如 S&P500 或 FTSE350）。

7.5 数据准确性评估

供应商的历史价格数据容易出现多种形式的错误：

- 公司行为 - 对股票拆分和股息调整的处理不正确。必须绝对确保公式已完成。
- 峰值 - 定价点大大超过某些历史波动率水平。这里必须小心，因为这些尖峰确实发生 - 请参阅 **May Flash Crash** 以获取一个可怕的例子。在发生股票拆分时不考虑股票拆分也可能导致高峰。尖峰过滤器脚本用于告诉交易者此类情况。（译者按：有时候过度清洗会产生一些问题，比如国内很多供应商会把超过涨跌停都洗掉，包括涨跌停价格）
- OHLC 聚合 - 来自雅虎/谷歌的免费 OHLC 数据特别容易出现对于“垃圾 tick 集”情况，较小的交易品种处理小交易（产生的价格）远偏离

当天的“主要”交易价格，从而导致错估最大值/最小值。这不是一个“错误”，而是一个需要警惕的问题。（译者按：有的时候交易规则也会使得价格漂移，这个在中国量化交易市场也很常见）

- 缺少数据 - 缺少数据可能是由于特定时间段内缺乏交易（常见于流动性差的小盘股的秒/分钟数据）、非交易日或交易系统中的错误。根据交易系统，可以填充缺失的数据（即用前一个值填充），内插值（线性或其他）或忽略。

其中许多错误依赖于人工判断，以决定如何进行。有些可以自动化解决这些错误，但自动化解决方案要困难得多。例如，必须选择峰值的阈值 - 使用多少偏离度为准？太高的 `stdev` 会错过一些高峰，但是太低，很多导致错误没有被筛除。所有这些问题都需要来量化交易者的判断。

还有如何修复错误。一旦知道错误就应该纠正错误，如果错误，是否应该进行跟踪？这将需要 `DB` 中的额外表。这将正确数据回填数据库，这是回测的一个特别隐蔽的问题。它涉及上游垃圾数据的自动纠正。如果您的数据供应商纠正了历史错误，但是根据先前不良数据的研究生成了经过反复测试的交易策略，则需要就策略有效性做出决策。通过充分了解策略绩效指标（特别是每笔交易的盈亏比特征的差异），可以在一定程度上减轻这种影响。应该选择或设计策略，使得单个数据不会在很大程度上扭曲策略的性能。

7.6 自动化

编写软件脚本以执行数据的下载，存储和清理的好处是脚本可以通过

操作系统提供的自动化工具。在基于 UNIX 的系统（例如 Mac OSX 或 Linux）中，可以使用 **crontab**，**crontab** 是一个持续运行的进程，允许在自定义时间或常规时间执行特定脚本。在 MS Windows 上有一个称为任务计划的程序。

例如，生产过程可能会在数据供应商发布后立即自动下载所有 S&P500 结束日价格。然后它将自动运行上述缺失数据和尖峰过滤脚本，通过电子邮件，短信或其他形式的通知向交易者发出警报。此时，任何回溯测试工具都将自动访问最新数据，而交易者无需抬起手指！根据您的交易系统是位于电脑上还是位于远程服务器上，您可以选择使用半自动或全自动的流程来完成这些任务。

7.7 数据可用性

一旦数据自动更新并留在 RDBMS 中，就必须将其存取到回测软件。此过程将在很大程度上取决于您的数据库的安装方式以及您的交易系统是本地（即台式计算机）还是远程（例如使用共用服务器）。

最重要的考虑因素之一是尽量减少过多的输入/输出（I/O）

带宽成本高的远程连接，在时间和金钱方面都非常昂贵。解决此问题的最佳方法是仅通过您需要的网络连接（通过选择性查询）或导出和压缩数据来移动数据。

许多 RDBMS 支持复制技术，它允许将数据库克隆到另一个远程系统上，通常具有一定程度的延迟。根据您的设置和数据量，这可能只有几分钟或几秒。一种简单的方法是将远程数据库复制到本地上。但是，请注意同步问题是常见且耗时的！

7.8 MySQL for Securities Masters

现在我们已经讨论了证券数据库的想法，现在是时候实际构建一个了。为此，我们将使用两种开源技术：**MySQL** 数据库和 **Python** 编程语言。在本章的最后，您将精通并拥有一个数据库，可以为您的量化交易研究进行进一步的数据分析。

7.8.1 安装 MySQL

在 Ubuntu 中安装 MySQL 非常简单。只需打开终端并输入以下内容：

```
sudo apt-get install mysql-server
```

最终，系统将提示您输入 **root** 密码。这是您的主要管理密码，所以千万不要忘记它！输入密码，安装将继续并完成。

7.8.2 配置 MySQL

现在 MySQL 安装在您的系统上，我们可以创建一个新的数据库和一个与之交互的用户。安装时将提示您输入 **root** 密码。要从命令行登录 MySQL，请使用以下行，然后输入您的密码：

```
mysql -u root -p
```

登录 MySQL 后，您可以创建一个名为 **securities_master** 的新数据库，然后选择它：

```
mysql> CREATE DATABASE securities_master;  
mysql> USE securities_master;
```

创建数据库后，需要添加新用户以与数据库进行交互。虽然您可以使用 **root** 用户，但从安全角度来看，它被视为不好的做法，因为它授予了太多权限并可能导致系统受损。在本地计算机上，这是无关紧要的，但在远程生产环境中，您肯定需要创建用户。在这种情况下

下，我们的用户将被称为 `sec_user`。请记住使用安全密码替换密码：

```
mysql> CREATE USER 'sec_user'@'localhost' IDENTIFIED BY 'password';  
mysql> GRANT ALL PRIVILEGES ON securities_master.* TO 'sec_user'@'localhost';  
mysql> FLUSH PRIVILEGES;
```

以上三行创建并授权用户使用 `securities_master`。从现在开始，与数据库发生的任何交互都将使用 `sec_user` 用户。

（译者按：作者的方法有点太复杂，直接找个集成环境安装就行，命令行安装反而可能出错。而且没有 MySQL 管理器会很麻烦）

7.8.3 EOD 股票的架构设计

我们现在已经安装了 MySQL 并配置了一个与我们的数据库进行交互的用户。在这个阶段，我们准备构建表来保存我们的金融数据。对于一个简单直接的股票主数据，我们将创建四个表：

- 交易所 - 交易所表列出了我们希望从中获取股票价格信息。在这种情况下，它几乎全部是纽约证券交易所（纽约证券交易所）和全国证券交易商自动报价协会（纳斯达克证券交易所）。
- DataVendor - 此表列出了有关历史价格数据供应商的信息。我们将使用 Yahoo Finance 来获取我们的日终（EOD）数据。通过引入此表，我们可以直接在必要时添加更多供应商，例如 Google 财经。
- Symbol - Symbol 表存储股票代码和公司信息列表。现在我们将避免诸如不同的共享类和多个符号名称类的问题。
- DailyPrice - 此表存储每日价格信息。如果添加许多证券，它可能会变得非常大。因此有必要对其进行优化以提高性能。

MySQL 是一个非常灵活的数据库软件，它允许您自定义数据在底层存储引擎中的存储方式。MySQL 中的两个主要竞争者是 MyISAM 和

InnoDB。虽然我不会详细介绍存储引擎，但我会说 MyISAM 对于快速读取更有用（例如查询大量价格信息），但它不支持事务（完全回滚中途失败的多步操作所必需的）。 InnoDB 虽然安全，但读取速度较慢。

InnoDB 在写入时允许行级锁定，而 MyISAM 在写入时锁定整个表。在将大量信息写入表中的任意点时（例如使用 UPDATE 语句），这可能会出现性能问题。这是一个很深入的话题，所以我将讨论留到以后！我们将使用 InnoDB，因为它本机使用很安全并提供行级锁定。如果我们发现表的读取速度很慢，我们可以创建索引作为第一步，然后在性能仍然存在问题时更改底层存储引擎。我们所有的表都将使用 UTF-8 字符集。

让我们从构架和 CREATE TABLE SQL 代码开始。它存储交易所的缩写和名称（即纽约证券交易所 - 纽约证券交易所）以及位置。它还支持货币和 UTC 的时区变化。我们还能看见内部目的存储创建和上次更新的日期。最后，我们将主索引键设置为自动递增整数 ID（足以处理 232 条记录）：

```
CREATE TABLE 'exchange' (  
  'id' int NOT NULL AUTO_INCREMENT,  
  'abbrev' varchar(32) NOT NULL,  
  'name' varchar(255) NOT NULL,  
  'city' varchar(255) NULL,  
  'country' varchar(255) NULL,  
  'currency' varchar(64) NULL,  
  'timezone_offset' time NULL,  
  'created_date' datetime NOT NULL,  
  'last_updated_date' datetime NOT NULL,  
  PRIMARY KEY ('id')  
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;
```

这是 data_vendor 表的模式和 CREATE TABLE SQL 代码。它存储名称，

网站和支持电子邮件。 我们可以及时为供应商添加更多有用信息，例如 API 端点 URL：

```
CREATE TABLE 'data_vendor' (  
  'id' int NOT NULL AUTO_INCREMENT,  
  'name' varchar(64) NOT NULL,  
  'website_url' varchar(255) NULL,  
  'support_email' varchar(255) NULL,  
  'created_date' datetime NOT NULL,  
  'last_updated_date' datetime NOT NULL,  
  PRIMARY KEY ('id')  
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;
```

以下是符号表的架构和 CREATE TABLE SQL 代码。 它包含到交易所的外键链接（我们暂时只支持交易所交易的品种），股票代码（例如 GOOG），品种类型（“股票”或“指数”），股票或股票市场指数，股票行业和货币。

```
CREATE TABLE 'symbol' (  
  'id' int NOT NULL AUTO_INCREMENT,  
  'exchange_id' int NULL,  
  'ticker' varchar(32) NOT NULL,  
  'instrument' varchar(64) NOT NULL,  
  'name' varchar(255) NULL,  
  'sector' varchar(255) NULL,  
  'currency' varchar(32) NULL,  
  'created_date' datetime NOT NULL,  
  'last_updated_date' datetime NOT NULL,  
  PRIMARY KEY ('id'),  
  KEY 'index_exchange_id' ('exchange_id')  
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;
```

以下是 daily_price 表的架构和 CREATE TABLE SQL 代码。该表是实际存储历史价格数据的位置。我们已经使用 daily_ 作为表名前缀，因为我们可能希望在以后的单独表格中创建分钟或秒级数据，以获得更高频率的策略。该表包含两个外键 - 一个用于数据供应商，另一个用于符号。这可以唯一标识数据，并让我们在同一个表中为多个供应商存

储相同的价格数据。我们还存储了价格日期（即 OHLC 数据有效的每日期限）以及为我们自己创建和日期更新。

其余字段存储开盘价 - 最低价、收盘价、最高价、收盘价。雅虎财经为我们提供股息和股票除权，其价格最终在 `adj_close_price` 列中。

请注意，数据类型为十进制（19,4）。在处理金融数据时，绝对必须准确。如果我们使用了 `float` 数据类型，由于浮点数据是内部存储的性质，我们最终会出现舍入错误。最后一个字段存储当天的交易量。这使用 `bigint` 数据类型，这样我们就不会因意外截断到最后天数错误。

```
CREATE TABLE 'daily_price' (  
  'id' int NOT NULL AUTO_INCREMENT,  
  'data_vendor_id' int NOT NULL,  
  'symbol_id' int NOT NULL,  
  'price_date' datetime NOT NULL,  
  'created_date' datetime NOT NULL,  
  'last_updated_date' datetime NOT NULL,  
  'open_price' decimal(19,4) NULL,  
  'high_price' decimal(19,4) NULL,  
  'low_price' decimal(19,4) NULL,  
  'close_price' decimal(19,4) NULL,  
  'adj_close_price' decimal(19,4) NULL,  
  'volume' bigint NULL,  
  PRIMARY KEY ('id'),  
  KEY 'index_data_vendor_id' ('data_vendor_id'),  
  KEY 'index_symbol_id' ('symbol_id')  
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;
```

通过在 MySQL 命令行中输入所有上述 SQL 命令，将创建四个表。

7.8.4 连接数据库

在我们使用 MySQL 和 Python 之前，我们需要安装 `mysqlclient` 库。

`mysqlclient` 实际上是另一个库的分支，称为 `Python-MySQL`。不幸的是 Python3 不支持后一个库，所以我们必须使用 `mysqlclient`。在 Mac OSX / UNIX flavor 机器上，我们需要运行以下命令：

```
sudo apt-get install libmysqlclient-dev  
pip install mysqlclient
```

我们现在准备开始通过 Python 和 pandas 与我们的 MySQL 数据库进行交互。

7.8.5 使用对象关系映射

对于那些具有数据库管理和开发背景的人，您可能会问是否使用对象关系映射（ORM）更为明智。ORM 允许编程语言中的对象直接映射到数据库中的表，这样程序代码完全不知道底层存储引擎。并非没有问题，但可以节省大量时间。然而，节省时间通常是以牺牲性能为代价的。

一个流行的 Python ORM 是 SQLAlchemy。它允许您指定数据库架构在 Python 本身内部，从而自动生成 CREATE TABLE 代码。由于我们专门选择了 MySQL 并且注重性能，因此我选择不在本章中使用 ORM。

符号检索

让我们首先获取与标准普尔 500 只大型股票（即标准普尔 500 指数）相关的所有股票代码。当然，这只是一个例子。如果您在英国进行交易并希望使用英国国内指数，您同样可以获得在伦敦证券交易所（伦敦证券交易所）交易的 FTSE100 公司列表。

维基百科方便地列出了标准普尔 500 指数的成分股。请注意，S&P500 中实际上有 502 个！我们将使用 Python 请求和 BeautifulSoup 库来抓取网站数据，然后将内容直接添加到 MySQL。首先确保安装了库：

```
pip install requests
pip install beautifulsoup4
```

以下代码将使用 requests 和 BeautifulSoup 库将标记直接添加到我们

之前创建的 MySQL 数据库中。请记住使用您在上面创建的密码替换“密码”：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# insert_symbols.py

from __future__ import print_function

import datetime
from math import ceil

import bs4
import MySQLdb as mdb
import requests

def obtain_parse_wiki_snp500():
    """
    Download and parse the Wikipedia list of S&P500
    constituents using requests and BeautifulSoup.

    Returns a list of tuples for to add to MySQL.
    """
    # Stores the current time, for the created_at record
    now = datetime.datetime.utcnow()

    # Use requests and BeautifulSoup to download the
    # list of S&P500 companies and obtain the symbol table
    response = requests.get(
        "http://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
    )
    soup = bs4.BeautifulSoup(response.text)

    # This selects the first table, using CSS Selector syntax
    # and then ignores the header row ([1:])
    symbolslist = soup.select('table')[0].select('tr')[1:]

    # Obtain the symbol information for each
    # row in the S&P500 constituent table
    symbols = []
    for i, symbol in enumerate(symbolslist):
        tds = symbol.select('td')
        symbols.append(
```

```

        tds[0].select('a')[0].text, # Ticker
        'stock',
        tds[1].select('a')[0].text, # Name
        tds[3].text, # Sector
        'USD', now, now
    )
)
return symbols

def insert_snp500_symbols(symbols):
    """
    Insert the S&P500 symbols into the MySQL database.
    """
    # Connect to the MySQL instance
    db_host = 'localhost'
    db_user = 'sec_user'
    db_pass = 'password'
    db_name = 'securities_master'
    con = mdb.connect(
        host=db_host, user=db_user, passwd=db_pass, db=db_name
    )

    # Create the insert strings
    column_str = ""ticker, instrument, name, sector,
        currency, created_date, last_updated_date
    """
    insert_str = ("%s, " * 7)[: -2]
    final_str = "INSERT INTO symbol (%s) VALUES (%s)" % \
        (column_str, insert_str)

    # Using the MySQL connection, carry out
    # an INSERT INTO for every symbol
    with con:
        cur = con.cursor()
        cur.executemany(final_str, symbols)

if __name__ == "__main__":
    symbols = obtain_parse_wiki_snp500()
    insert_snp500_symbols(symbols)
    print("%s symbols were successfully added." % len(symbols))

```

在这个阶段，我们将在数据库中拥有 S&P500 指数的所有 502 个当前成分。 我们的下一个任务是从不同的来源实际获取历史定价数据并将其与进行匹配。

价格检索

为了获得当前 S&P500 成分的历史数据，我们必须首先在数据库中查询所有股票代码的列表。

一旦返回了列表以及 ID，就可以调用 Yahoo Finance API 并下载每个股票代码的历史价格数据。

一旦我们有了每个股票代码，我们就可以依次将数据插入到数据库中。这是执行此操作的 Python 代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# price_retrieval.py

from __future__ import print_function

import datetime
import warnings

import MySQLdb as mdb
import requests

# Obtain a database connection to the MySQL instance
db_host = 'localhost'
db_user = 'sec_user'
db_pass = 'password'
db_name = 'securities_master'
con = mdb.connect(db_host, db_user, db_pass, db_name)
```

```

def obtain_list_of_db_tickers():
    """
    Obtains a list of the ticker symbols in the database.
    """
    with con:
        cur = con.cursor()
        cur.execute("SELECT id, ticker FROM symbol")
        data = cur.fetchall()
        return [(d[0], d[1]) for d in data]

def get_daily_historic_data_yahoo(
    ticker, start_date=(2000,1,1),
    end_date=datetime.date.today().timetuple()[0:3]
):
    """
    Obtains data from Yahoo Finance returns and a list of tuples.

    ticker: Yahoo Finance ticker symbol, e.g. "GOOG" for Google, Inc.
    start_date: Start date in (YYYY, M, D) format
    end_date: End date in (YYYY, M, D) format
    """

```

```

# Construct the Yahoo URL with the correct integer query parameters
# for start and end dates. Note that some parameters are zero-based!
ticker_tup = (
    ticker, start_date[1]-1, start_date[2],
    start_date[0], end_date[1]-1, end_date[2],
    end_date[0]
)
yahoo_url = "http://ichart.finance.yahoo.com/table.csv"
yahoo_url += "?s=%s&a=%s&b=%s&c=%s&d=%s&e=%s&f=%s"
yahoo_url = yahoo_url % ticker_tup

# Try connecting to Yahoo Finance and obtaining the data
# On failure, print an error message.
try:
    yf_data = requests.get(yahoo_url).text.split("\n")[1:-1]
    prices = []

```

```

        for y in yf_data:
            p = y.strip().split(',')
            prices.append(
                (datetime.datetime.strptime(p[0], '%Y-%m-%d'),
                 p[1], p[2], p[3], p[4], p[5], p[6])
            )
    except Exception as e:
        print("Could not download Yahoo data: %s" % e)
    return prices

def insert_daily_data_into_db(
    data_vendor_id, symbol_id, daily_data
):
    """
    Takes a list of tuples of daily data and adds it to the
    MySQL database. Appends the vendor ID and symbol ID to the data.

    daily_data: List of tuples of the OHLC data (with
    adj_close and volume)
    """

    # Create the time now
    now = datetime.datetime.utcnow()

    # Amend the data to include the vendor ID and symbol ID
    daily_data = [
        (data_vendor_id, symbol_id, d[0], now, now,
         d[1], d[2], d[3], d[4], d[5], d[6])
        for d in daily_data
    ]

    # Create the insert strings
    column_str = """data_vendor_id, symbol_id, price_date, created_date,
                    last_updated_date, open_price, high_price, low_price,
                    close_price, volume, adj_close_price"""
    insert_str = ("%s, " * 11)[: -2]
    final_str = "INSERT INTO daily_price (%s) VALUES (%s)" % \
        (column_str, insert_str)

    # Using the MySQL connection, carry out an INSERT INTO for every symbol
    with con:
        cur = con.cursor()
        cur.executemany(final_str, daily_data)

```

```

if __name__ == "__main__":
    # This ignores the warnings regarding Data Truncation
    # from the Yahoo precision to Decimal(19,4) datatypes
    warnings.filterwarnings('ignore')

    # Loop over the tickers and insert the daily historical
    # data into the database
    tickers = obtain_list_of_db_tickers()
    lentickers = len(tickers)
    for i, t in enumerate(tickers):
        print(
            "Adding data for %s: %s out of %s" %
            (t[1], i+1, lentickers)
        )
        yf_data = get_daily_historic_data_yahoo(t[1])
        insert_daily_data_into_db('1', t[0], yf_data)
    print("Successfully added Yahoo Finance pricing data to DB.")

```

请注意，我们可以优化此过程。 例如，如果我们使用 Python Scrapy 库，我们将从下载时采用高并发性，因为 Scrapy 是基于事件驱动的 Twisted 框架构建的。 目前，每个下载都将按顺序执行。

7.9 从证券数据库那里检索数据

现在我们已经下载了所有当前标准普尔 500 指数成分股的历史价格，我们希望能够在 Python 中访问它。 pandas library 使这非常简单。这个脚本从我们的证券数据库获取一段时间内 Google 股票的 Open-High-Low-Close （OHLC）数据，并输出数据集：


```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# retrieving_data.py

from __future__ import print_function

import pandas as pd
import MySQLdb as mdb

if __name__ == "__main__":
    # Connect to the MySQL instance
    db_host = 'localhost'
    db_user = 'sec_user'
    db_pass = 'password'
    db_name = 'securities_master'
    con = mdb.connect(db_host, db_user, db_pass, db_name)

    # Select all of the historic Google adjusted close data
    sql = """SELECT dp.price_date, dp.adj_close_price
              FROM symbol AS sym
              INNER JOIN daily_price AS dp
              ON dp.symbol_id = sym.id
              WHERE sym.ticker = 'GOOG'
              ORDER BY dp.price_date ASC;"""

    # Create a pandas dataframe from the SQL query
    goog = pd.read_sql_query(sql, con=con, index_col='price_date')

    # Output the dataframe tail
    print(goog.tail())
```

脚本输出如下：

price_date	adj_close_price
2015-06-09	526.69
2015-06-10	536.69
2015-06-11	534.61
2015-06-12	532.33
2015-06-15	527.20

这显然只是一个简单的脚本，但它显示了本地存储的证券数据库有多强大。使用这种方法可以非常快速地回测某些策略，因为来自数据库的输入/输出（I/O）速度将明显快于互联网连接。

第 8 章

处理金融数据

在上一章中，我们概述了如何构建基于股票的证券数据库。本章将讨论在大多数交易中不经常考虑的问题，即在策略测试中使用处理金融市场数据的问题。

本章将讨论算法交易者感兴趣的不同类型数据。首先考虑数据的频率，从季度数据（例如 SEC 报告）到 tick 和毫秒级的订单数据。然后将讲述此类数据（包括免费和商业）的来源以及获取数据的代码。最后，将讨论清理和准备用于策略的数据。

8.1 市场和金融品种分类

作为算法交易者，我们通常对广泛的金融市场数据感兴趣。基础金融产品和衍生金融工具的时间序列价格，非结构化文本数据（如新闻文章）和企业收益信息等。本书将主要讨论金融时间序列数据。

8.1.1 市场

美国和国际股票，外汇，商品和固定收益数据是算法交易者感兴趣的。在股票市场中，作为资产直接购买仍然非常普遍，而在后三个市场中，高流动性衍生工具（期货，期权或更多金融工具）更常用于交易目的。数据普遍易得性使得在股票市场上交易变得相对简单，尽管有关于公司行为数据处理的问题（见下文）。因此个人算法交易领域的很大一部分将基于股票，例如直接公司股票或交易所交易基金（ETF）。外汇市场也非常受欢迎，因为经纪商将允许以点百分比（PIP）变动进行保证金交易。pip 是货币汇率中第四个小数点的一个单位。对于以

美元计价的货币，这相当于 1/100 美分。

大宗商品和固定收益市场难以直接在底层交易。 个人算法交易者对向油库提供交割不感兴趣！ 相反，标的资产的期货合约用于投机目的。 利用保证金交易允许对此类合约进行杠杆作用。

8.1.2 品种

算法交易者可以使用各种品种和金融衍生工具。 下表描述了各种常见用例。

Market	Instruments
Equities/Indices	Stock, ETFs, Futures, Options
Foreign Exchange	Margin/Spot, ETFs, Futures, Options
Commodities	Futures, Options
Fixed Income	Futures, Options

就本书而言，我们将几乎专注于股票和 ETF，以简化实现。

8.1.3 基本数据

尽管算法交易者主要对金融价格时间序列进行分析，但基本数据（不同频率）也经常添加到分析中。 所谓的量化价值（QV）策略在很大程度上依赖于基础数据的积累和分析，如宏观经济信息，企业盈利历史，通胀指数，工资报告，利率和美国证券交易委员会备案资料。 这些数据通常也是时序形式的，尽管形式在数月，数季或数年的时间差别要大得多。但 QV 策略也适用于这些时间范围。

本书不会在很大程度上讨论 QV 策略或大型尺度时间的基本面驱动策略，而是集中于主要来自价格行为或更高频的策略。

8.1.4 非结构化数据

非结构化数据由诸如新闻文章，博客文章，论文或报告之类的文档组

成。对这些数据的分析可能很复杂，因为它依赖于自然语言处理(NLP)技术。分析非结构化数据的一种这样的用途是试图理解情绪。这对于推动交易策略非常有用。例如，通过将文本分类为“看涨”，“看跌”或“中性”，可以生成一组交易信号。该过程的是舆情分析的形式。（译者按：实际上，这类策略是表现最差的策略，原因是数据太脏，而且 NLP 技术也没那么成熟）

Python 提供了一个非常全面的文本数据分析库，称为自然语言工具包（NLTK）。事实上，关于 NLTK 的 O'Reilly 书可以通过作者的网站免费下载 - 使用 Python 进行自然语言处理[3]。（译者推荐 keras）

评论数据

有许多评论数据来源可能对生成交易策略有用。彭博（Bloomberg）和英国“金融时报”（Financial Times）等热门资金来源以及 Seeking Alpha 和 ZeroHedge 等金融评论博客提供了重要的文本来源进行分析。此外，数据供应商提供的专有新闻源也是此类数据的良好来源。

为了获得更大规模的数据，必须使用“爬虫”工具，这些工具旨在自动下载网站。请注意，自动网络抓取工具有时会违反这些网站的服务条款。在开始下载此类数据之前，请务必检查。Scrapy 库是一种特别有用的爬虫工具，它使流程高效且结构化。

社交媒体数据

在过去几年中，人们对从社交媒体数据中获取情绪信息非常感兴趣，特别是通过 Twitter 微博服务。早在 2011 年，围绕 Twitter 情绪推出了一只对冲基金，称为 Derwent Capital。实际上，学术研究[4]已经

证明有可能基于这种情绪分析产生一定程度的预测能力

如果您希望对情绪进行研究，情绪分析超出了本书的范围，那么 **Matt Russell** 有两本书通过 Web 服务提供的公共 API 获取社交媒体数据。

8.2 数据频率

在设计算法交易框架时，数据频率是最重要的考虑因素之一。它将影响有关数据存储，回测策略和执行算法的每个设计。

更高频率的策略可能会导致策略更具统计学意义的分析，这仅仅是因为将使用更多的数据（以及交易）。HFT 策略通常需要大量的时间和资金投入来开发必要的软件来实现它们。

较低频率策略更易于开发和部署，因为它们需要较少的自动化。但是，与高频策略相比，它们通常会产生更少的交易，从而导致分析的统计性较差。

8.2.1 每周和每月数据

基本数据通常按周，按月，按季或按年报告。这些数据包括工资数据，对冲基金业绩报告，美国证券交易委员会备案资料，基于通货膨胀的指数（如消费者物价指数，CPI），经济增长和企业账户。

存储此类数据通常适用于非结构化数据库，例如 MongoDB，它可以处理分层嵌套数据，从而实现查询功能。另一种方法是将平面文本文本存储在 RDBMS 中，这不太合适，因为全面查询比较棘手。

8.2.2 每日数据

大多数零售算法交易者利用每日（“一天结束” / EOD）金融时间序列数据，特别是在股票和外汇交易中。这些数据是免费提供的（见

下文),但质量往往存疑,并受某些影响。日终数据通常存储在 RDBMS 中,因为自动收报机/代号映射的性质自然适用于关系模型。

EOD 数据不需要特别大的存储要求。美国交易所一年有 252 个交易日,因此十年内每个证券将有 2,520 个交易日。即使有 10,000 个股票代码,这也是 25,200,000 个数,可以在关系数据库环境中轻松处理。

8.2.3 日内 Bars

日内策略通常使用每小时,十五,五,一分钟 OHLCV 拒绝。像 QuantQuote 和 DTN IQFeed 这样的日内数据供应商通常会根据他们的价格数据提供 tick 或秒级数据。

这些频率的数据将拥有许多“缺失”,因为在该时间段内没有进行任何交易。虽然数据准确性有所降低,但 pandas 可用于向前填充这些值。此外,如果需要,还可以使用 pandas 在不太精细的时间尺度上创建数据。

在十年期间,每分钟数据将产生近 100 万条。同样,对于秒级,同一时期的数据量将达到近六千万个。因此,存储一千个这样的证券将导致六十亿条数据。要在 RDBMS 中保存的大量数据,因此需要更复杂的方法。

如此规模的存储和检索有点超出了本书的范围,因此我不再进一步讨论。

8.2.4 tick 和订单簿数据

当在交易所或其他场所的交易发生时,会生成一个 tick。Tick 包含交易所的交易信息。交易信息与每个数据一起存储,其时间戳精确

到毫秒级别。Tick 数据通常还包括最新的最佳买/卖价。这些数据量很大，细粒度高的数据存储远远超出了本书的范围。常见的存储机制包括 HDF5，kdb 和 CSV。

交易所的多个限价订单导致订单簿的概念（译者按：类似十档\全档行情）。基本上是每个市场参与者在特定交易下的所有买入和卖出限价单的清单。这里面产生买卖价差的定义（或简称为“价差”），这是“账面价值”订单的买入价和卖出价的最小差异。通常需要创建限额订单簿的历史表示或市场模拟器来执行超高频交易（UHFT）策略。存储此类数据非常复杂，因此不在本书的讨论范围之内。

8.3 数据来源

有许多金融数据来源和供应商在广度，及时性，质量和价格方面差异很大。

从广义上讲，提供每日或更长时间的金融市场数据可以自由获得，尽管整体质量可疑，并且存在幸存者偏差的可能性。为了获得日内数据，通常需要购买商业数据。这些数据的供应商在客户服务能力，整体数据质量和金融品种广度方面差异很大。

8.3.1 免费来源

可在雅虎财经，谷歌财经和 Quandl 上找到各种美国和国际股票和期货免费的日终数据，包括产品的 OHLCV 价格。

雅虎财经

雅虎财经是作为美国股票日终数据库的“go to”资源。数据的广度非常全面，列出了成千上万的交易股票。此外，股票拆分和股息采

用复权方法处理，作为 API 输出的 CSV 输出中的“Adj Close”列（我们将在下面讨论）。这些据允许算法交易者快速启动并且零成本。

我个人在清理雅虎数据方面有很多经验。我必须说数据可能有很多错误。首先，它会遇到回填的问题。当过去的历史数据在将来某个日期得到纠正时会出现此问题，从而导致质量较差的回测随着您自己的数据库重新更新而发生变化。为了解决这个问题，每当修改历史数据点时，通常会将记录记录添加到证券数据库（在适当的记录表中）。

其次，雅虎数据只汇总来自少数几个来源的价格，形成 OHLCV。这意味着开盘价，最高价，最低价和收盘价附近的价值可能具有欺骗性，因为其他交易所/流动性来源可能已执行超过价值的不同价格。（译者按：海外有些盘前数据等等和国内不太一样）

第三，我注意到，当从雅虎那里获得金融数据时，这些错误会蔓延到 API 中。例如，多次调用具有相同日期/股票代码参数的 API 有时会导致不同的数据集。这显然是一个重大问题，必须仔细检查。

总之，如果您选择使用它来填充大型证券主数据并且需要高度准确的数据，那么准备对 Yahoo Finance 数据进行一些大规模的数据清理。

Quandl

Quandl 是一项相对较新的服务商，声称是“在网络上查找和加载数据的最方便的数据商”。我相信他们正在实现这一目标的道路上！该服务商提供了大量的美国和国际股票，利率，商品/期货，外汇交易和其他经济数据的每日数据集。此外，数据库不断扩展，项目得到了非常积极的维护。

所有数据都可以通过非常先进的 HTTP API (CSV, JSON, XML 或 HTML) 访问, 其中包含各种编程语言的插件封装, 包括 R, Python, Matlab, Excel, Stata, Maple, C#, EViews, Java, C/C++, .NET, Clojure 和 Julia。如果注册帐户, 则可以将其增加到调用 500 次、小时。事实上, 如果需要, 可以通过联系团队将改为每小时 5,000 次。我对 Quandl “大规模” 没有太多经验, 所以我不能评论数据集中的错误级别, 但我的感觉是任何错误都可能不断报告和纠正。因此, 它们值得考虑作为日终证券大师的主要数据来源。

在本章的后面, 我们将讨论如何使用 Python 和 pandas 从 Quandl 获取美国商品期货数据。

8.3.2 商业来源

为了进行日内算法交易, 通常需要购买商业数据。对于 “零售级” 数据, 定价范围在从每月 30 美元到每月 500 美元左右。机构级别的费用通常处于每月四位数范围内, 我不会在此讨论它们。

EODData

我在基金背景下使用了 EODData, 虽然只有日常数据, 主要用于外汇。但他们提供了一定程度的盘中消息。他们的 “铂金” 套餐每月费用为 25 美元。

该资源对于在全球交易所查找交易代码的完整列表非常有用, 但请记住, 这将受到幸存者偏差的影响, 我认为该列表代表当前品种。

不幸的是 (至少在 2010 年), 我发现股票拆分数据有些不准确 (至少与晨星信息相比)。这导致数据中存在一些峰值问题 (见下文),

这增加了数据清理过程中的难度。

DTN IQFeed

DTN IQFeed 是个人算法交易者最受欢迎的数据源之一。他们声称拥有超过 80,000 名客户。它们提供未过滤的实时 tick 数据以及大量历史数据。

定价从每月 50 美元开始，但实际上，一旦选择了特定服务并考虑了费用，实际价格将在每月 150 美元至 200 美元之间。我将 DTN IQFeed 用于我所有的日内股票和期货策略。在历史数据方面，IQFeed 提供股票，期货和期权：

- 180 个日历日（每笔交易）
- 7 年以上的 1 分钟历史数据
- 15 年以上的日常历史数据

主要缺点是 DTN IQFeed 软件（迷你服务器，而不是图表工具）只能在 Windows 上运行。如果你在这个操作系统中进行所有的算法交易，这可能不是问题，但我个人在 Ubuntu Linux 中开发了我的所有策略。但是，虽然我没有积极测试它，但我听说可以运行 DTN 在 WINE 模拟器下的 IQFeed。

下面我们将讨论如何在 Windows 中使用 Python 从 IQFeed 获取数据。

QuantQuote

QuantQuote 提供 1998 年至今美国股票历史分钟，秒和 tick 价格数据，此外，它们提供机构级实时 tick 信息，尽管这对个人算法交易者意义不太大。QuantQuote 的主要优点之一是由于他们的 Tick 股票代码匹

配以及在特定指数中包含所有股票的时间，他们的数据不受幸存者偏差的影响。

举例来说，购买标准普尔 500 指数的历史可以追溯到 1998 年的微观市场，包括退市的股票数据，写作时购买成本是 895 美元。定价随着数据细粒度增加而扩大。

QuantQuote 目前是 QuantConnect 基于网络的回测服务市场数据的主要提供商。QuantQuote 竭尽全力确保最大限度地减少错误，因此如果您正在寻找美国股票的高分辨数据，那么您应该考虑使用他们的服务。

8.4 获取数据

在本节中，我们将讨论如何使用 Quandl, pandas 和 DTN IQFeed 在一系列市场和一定时间范围内下载金融市场数据。

8.4.1 雅虎财经和 pandas

Pandas 库使得从 Yahoo Finance 下载 EOD 数据变得非常简单。Pandas 附带了一个 DataReader 组件，该组件与 Yahoo Finance（以及其他来源）相关联。指定具有开始和结束日期的符号足以将 EOD 系列下载到 pandas DataFrame 中，从而允许执行快速向量化操作：

```

from __future__ import print_function

import datetime
import pandas.io.data as web

if __name__ == "__main__":
    spy = web.DataReader(
        "SPY", "yahoo",
        datetime.datetime(2007,1,1),
        datetime.datetime(2015,6,15)
    )
    print(spy.tail())

```

输出如下：

	Open	High	Low	Close	Volume \
Date					
2015-06-09	208.449997	209.100006	207.690002	208.449997	98148200
2015-06-10	209.369995	211.410004	209.300003	210.960007	129936200
2015-06-11	211.479996	212.089996	211.199997	211.649994	72672100
2015-06-12	210.639999	211.479996	209.679993	209.929993	127811900
2015-06-15	208.639999	209.449997	207.789993	209.100006	121425800
	Adj Close				
Date					
2015-06-09	208.449997				
2015-06-10	210.960007				
2015-06-11	211.649994				
2015-06-12	209.929993				
2015-06-15	209.100006				

请注意，在 pandas 0.17.0 中，pandas.io.data 将被单独的 pandas-datareader 包替换。但是，暂时（即 pandas 版本 0.16.x），导入数据读取器的语法是将 pandas.io.data 作为 web 导入。

在下一节中，我们将使用 Quandl 创建一个更全面，永久的下载解决方案。

8.4.2 Quandl 和 Pandas

直到最近，以频繁更新的方式在交易所获得一致的期货数据是相当困难和昂贵的。然而，Quandl 服务的发布大大改变了这种情况，在某

些情况下金融数据可以追溯到 20 世纪 50 年代。 在本节中，我们将使用 Quandl 在多个日期下载一组期末期货合约。

注册 Quandl

首先要注册的是 Quandl。 这将增加对其 API 调用的每日使用量。 注册每天限制 500 此，而不是默认 50 个。 访问网站 www.quandl.com：

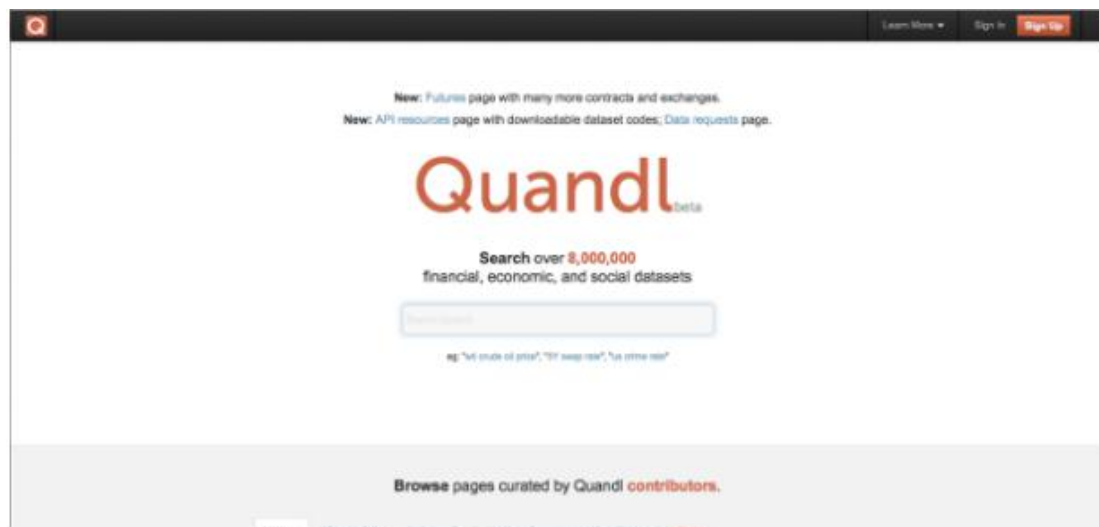


图 8.1: Quandl 主页

点击右上角的注册按钮：

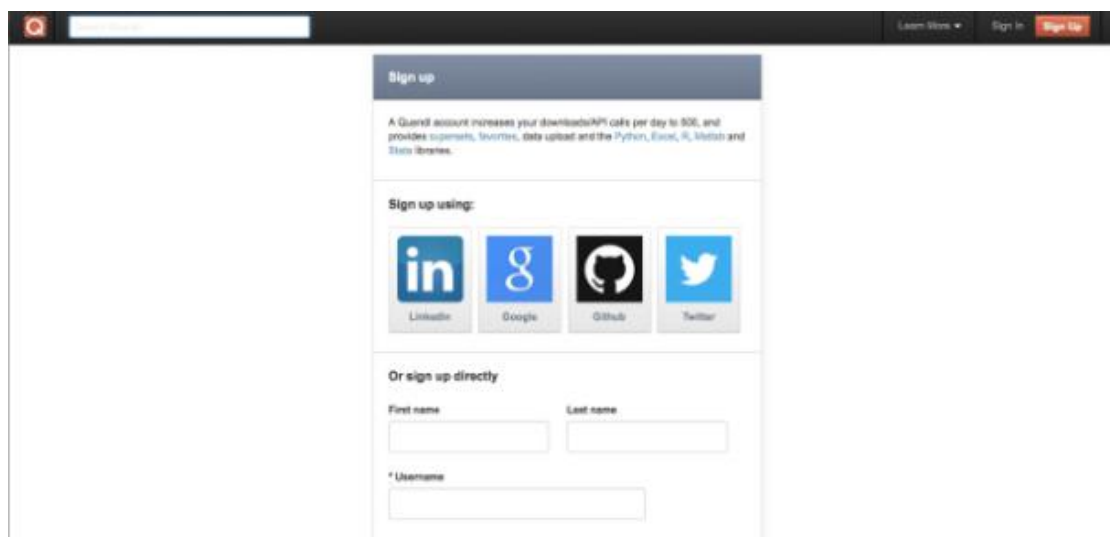


图 8.2: Quandl 注册页面

登录后，您将返回主页：

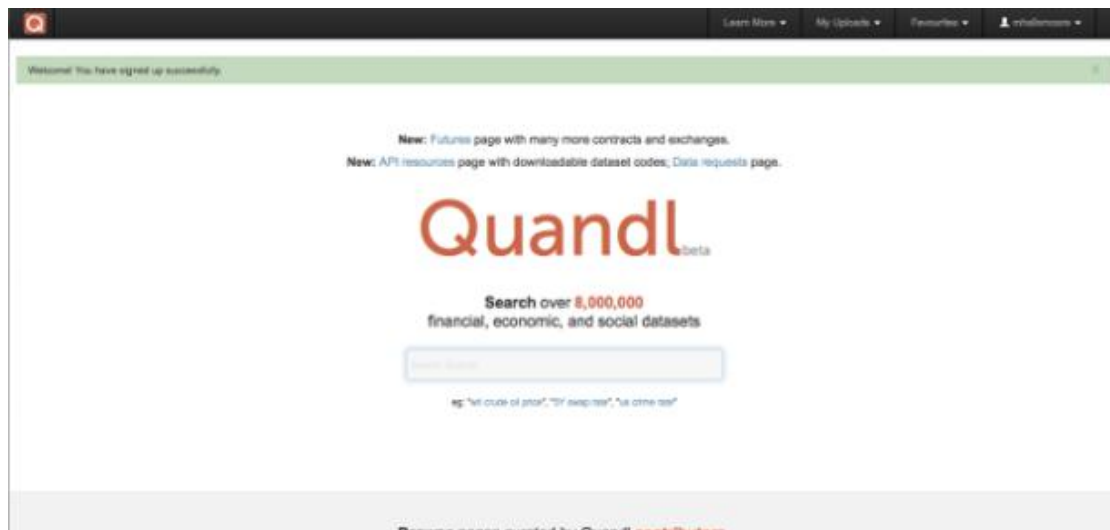


图 8.3: Quandl 授权主页

Quandl 期货数据

现在点击“新建：期货页面...”链接进入期货主页：

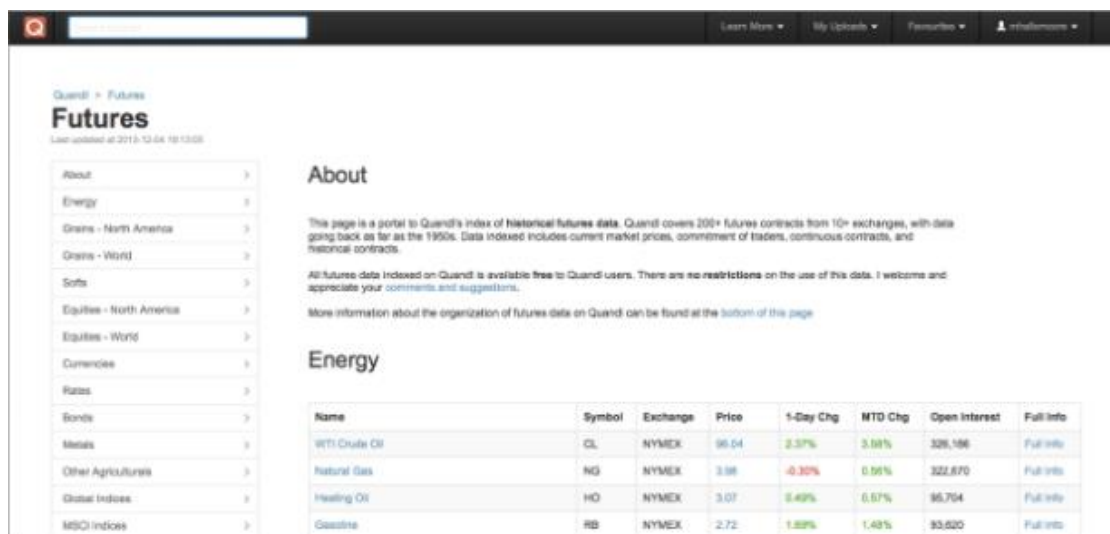


图 8.4: Quandl 期货合约主页

在本教程中，我们将考虑高流动性的 Mini S&P500 期货合约，其期货符号为 ES。要下载其他合约，可以使用其他符号替换 ES 的引用来执行本教程的其余部分。

点击 E-Mini S&P500 链接（或您选择的期货代码），您将进入以下屏幕：在屏幕上向下滚动显示可追溯到 1997 年的历史合约列表：

单击其中一个单独的合同。例如，我选择了 ESZ2014，它指的是 2014 年 12 月“交付”的合约。这将显示数据图表：

通过单击“下载”按钮，可以以多种格式获取数据：HTML，CSV，JSON 或 XML。此外，我们可以使用 Python 将数据直接下载到 pandas DataFrame 中。虽然后者对快速“翻译和数据探索很有用，但在本节中我们正在考虑开发一个本地长期数据存储表。单击下载按钮，选择“CSV”，然后复制并粘贴 API 调用：

API 调用将具有以下形式：

```
http://www.quandl.com/api/v1/datasets/OFDP/FUTURE_ESZ2014.csv?
&auth_token=MY_AUTH_TOKEN&trim_start=2013-09-18
&trim_end=2013-12-04&sort_order=desc
```

授权指令已经过编辑，并替换为 MY_AUTH_TOKEN。有必要复制“auth_token =”和“&trim_start”之间的字母数字字符串

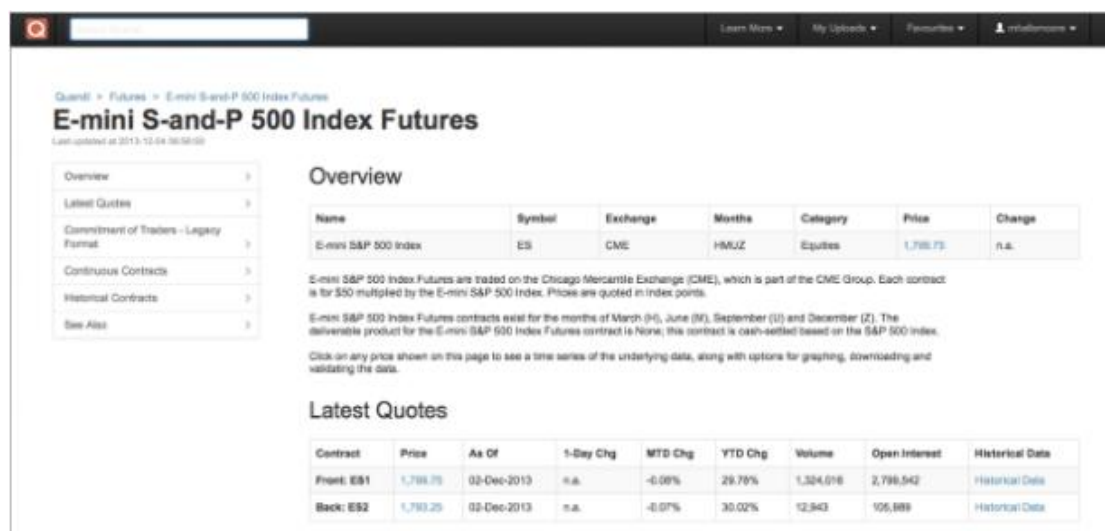


图 8.5：E-Mini S&P500 合约页面

	H	M	U	Z
1997	ESM1997	ESM1997	ESU1997	ESZ1997
1998	ESM1998	ESM1998	ESU1998	ESZ1998
1999	ESM1999	ESM1999	ESU1999	ESZ1999
2000	ESM2000	ESM2000	ESU2000	ESZ2000
2001	ESM2001	ESM2001	ESU2001	ESZ2001
2002	ESM2002	ESM2002	ESU2002	ESZ2002
2003	ESM2003	ESM2003	ESU2003	ESZ2003
2004	ESM2004	ESM2004	ESU2004	ESZ2004
2005	ESM2005	ESM2005	ESU2005	ESZ2005
2006	ESM2006	ESM2006	ESU2006	ESZ2006
2007	ESM2007	ESM2007	ESU2007	ESZ2007
2008	ESM2008	ESM2008	ESU2008	ESZ2008
2009	ESM2009	ESM2009	ESU2009	ESZ2009
2010	ESM2010	ESM2010	ESU2010	ESZ2010
2011	ESM2011	ESM2011	ESU2011	ESZ2011

图 8.6: E-Mini S&P500 历史合约



图 8.7: ESZ2014 图表（2014 年 12 月交割）

稍后在 Python 脚本中使用。 不要与任何人共享，因为它是 Quandl 下载的唯一授权指令，用于确定当天的下载速率。

此 API 调用将构成自动脚本的基础，我们将在下面编写该脚本以下载整个历史期货合约集。

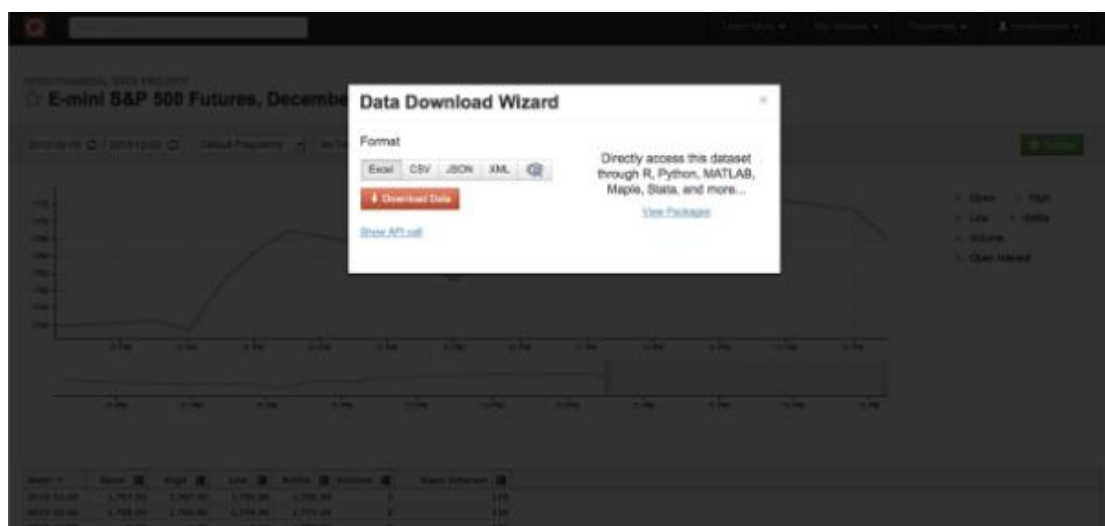


图 8.8: ESZ2014 CSV 文件的下载模式

将 Quandl Futures 下载到 Python 中

因为我们将长期使用期货数据作为证券数据库策略的一部分，所以我们希望将期货数据存储到本地。因此，我们需要创建一个目录来保存我们的 E-Mini 合约 CSV 文件。在 Mac / Linux（终端/控制台内）中，可通过以下命令实现：

```
cd /PATH/TO/YOUR/quandl_data.py
mkdir -p quandl/futures/ES
```

注意：将 / PATH / TO / YOUR 替换为 quandl_data.py 文件所在的目录。这将创建一个名为 quandl 的子目录，其中包含两个其他子目录，特别是针对期货和 ES 合约。这将有助于我们以持续的方式下载。

为了使用 Python 执行下载，我们需要导入一些库。特别是我们需要安装 pandas 以及 matplotlib 以进行绘图和数据操作：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# quandl_data.py

from __future__ import print_function

import matplotlib.pyplot as plt
import pandas as pd
import requests
```

代码中的第一个函数将生成我们希望下载的期货符号列表。 我已经为开始和结束年份添加了参数，将它们设置为 2010 年和 2014 年。当然，您可以选择使用其他时间范围：

```
def construct_futures_symbols(
    symbol, start_year=2010, end_year=2014
):
    """
    Constructs a list of futures contract codes
    for a particular symbol and timeframe.
    """
    futures = []

    # March, June, September and
    # December delivery codes
    months = 'HMUZ'
    for y in range(start_year, end_year+1):
        for m in months:
            futures.append("%s%s%s" % (symbol, m, y))
    return futures
```

现在我们需要遍历每个交易品种，从 Quandl 获取该特定合约的 CSV 文件，然后将其写入磁盘，以便我们以后访问它：

```
def download_contract_from_quandl(contract, dl_dir):
    """
    Download an individual futures contract from Quandl and then
    store it to disk in the 'dl_dir' directory. An auth_token is
    required, which is obtained from the Quandl upon sign-up.
    """
    # Construct the API call from the contract and auth_token
    api_call = "http://www.quandl.com/api/v1/datasets/"
    api_call += "OFDP/FUTURE_%s.csv" % contract
    # If you wish to add an auth token for more downloads, simply
    # comment the following line and replace MY_AUTH_TOKEN with
    # your auth token in the line below
    params = "?sort_order=asc"
    #params = "?auth_token=MY_AUTH_TOKEN&sort_order=asc"
    full_url = "%s%s" % (api_call, params)

    # Download the data from Quandl
    data = requests.get(full_url).text

    # Store the data to disk
    fc = open('%s/%s.csv' % (dl_dir, contract), 'w')
    fc.write(data)
    fc.close()
```

现在我们将上述两个函数锁定在一起的数据以下载所有需要的合约：

```
def download_historical_contracts(
    symbol, dl_dir, start_year=2010, end_year=2014
):
    """
    Downloads all futures contracts for a specified symbol
    between a start_year and an end_year.
    """
    contracts = construct_futures_symbols(
        symbol, start_year, end_year
    )
    for c in contracts:
        print("Downloading contract: %s" % c)
        download_contract_from_quandl(c, dl_dir)
```

最后，我们可以使用 main 函数将期货价格添加到 pandas 数据框中。

然后我们可以使用 matplotlib 绘制价格：

```
if __name__ == "__main__":
    symbol = 'ES'

    # Make sure you've created this
    # relative directory beforehand
```

```

dl_dir = 'quandl/futures/ES'

# Create the start and end years
start_year = 2010
end_year = 2014

# Download the contracts into the directory
download_historical_contracts(
    symbol, dl_dir, start_year, end_year
)

# Open up a single contract via read_csv
# and plot the settle price
es = pd.io.parsers.read_csv(
    "%s/ESH2010.csv" % dl_dir, index_col="Date"
)
es["Settle"].plot()
plt.show()

```

输出如图 8.4.2 所示：

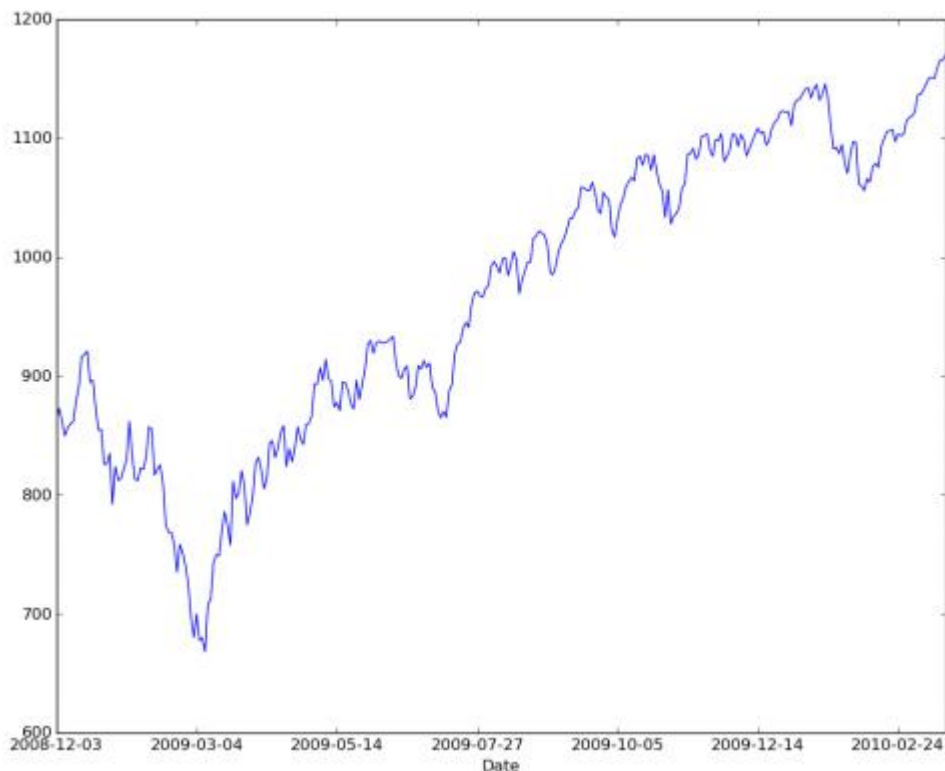


图 8.9: ESH2010 结算价格

可以修改上述代码，以便在必要时收集 Quandl 的任何期货合约组合。

请记住，除非提出更高的 API 请求，否则代码将限制为每天发出 50 个 API 请求。

8.4.3 DTN IQFeed

对于订阅 DTN IQFeed 的用户，该服务提供了用于获取日内数据的客户端。为此，需要下载 IQLink 服务器并在 Windows 上运行它。不幸的是，除非使用 WINE 模拟器，否则在 Mac 或 Linux 上执行此服务器很棘手。但是，一旦客户端运行，它就可以通过套接字段，此时可以查询数据。（译者按：类似 wind）

在本节中，我们将使用 Python 套接接口从 2007 年 1 月 1 日获取美国 ETF 的精细数据。由于美国市场每年约有 252 个交易日，每个交易日的交易时间为 6.5 小时，这相当于至少 650,000 条数据，每条数据点有 7 个数据点：时间戳，开盘价，最低价，最高价，收盘价，交易量和未平仓量。

我已选择 SPY 和 IWM ETF 下载到 CSV。在执行此脚本之前，在 Windows 中启动 IQLink 程序：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# iqfeed.py

import sys
import socket

def read_historical_data_socket(sock, recv_buffer=4096):
    """
    Read the information from the socket, in a buffered
    fashion, receiving only 4096 bytes at a time.
```

```

Parameters:
sock - The socket object
recv_buffer - Amount in bytes to receive per read
"""

buffer = ""
data = ""
while True:
    data = sock.recv(recv_buffer)
    buffer += data

    # Check if the end message string arrives
    if "!ENDMSG!" in buffer:
        break

# Remove the end message string
buffer = buffer[:-12]
return buffer

```

```

if __name__ == "__main__":
    # Define server host, port and symbols to download
    host = "127.0.0.1" # Localhost
    port = 9100 # Historical data socket port
    syms = ["SPY", "IWM"]

    # Download each symbol to disk
    for sym in syms:
        print "Downloading symbol: %s..." % sym

        # Construct the message needed by IQFeed to retrieve data
        message = "HIT,%s,60,20070101 075000,,,093000,160000,1\n" % sym

        # Open a streaming socket to the IQFeed server locally
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((host, port))

```

```

# Send the historical data request
# message and buffer the data
sock.sendall(message)

```

```

# Remove all the endlines and line-ending
# comma delimiter from each record
data = "".join(data.split("\r"))
data = data.replace(",\n","\n")[:-1]

# Write the data stream to disk
f = open("%s.csv" % sym, "w")
f.write(data)
f.close()

```

通过 DTN IQFeed 帐户中的其他订阅选项，可以下载单个期货合约（经过调整的连续合约），期权和指数。DTN IQFeed 还提供实时行情流，

但这种形式的数据不属于本书的范围。

8.5 清理金融数据

在从供应商交付金融数据之后，有必要进行数据清理。不幸的是，这可能是一个艰苦的过程，但却是非常必要的过程。有许多问题需要解决：数据不正确，考虑数据汇总和回填。在策略研究之前，数据清理包括后退/前进调整，连续合同拼接和公司行动的处理。

8.5.1 数据质量

数据供应商名声通常取决于其数据质量。简单来说，数据不良或丢失会导致错误的交易信号，从而导致潜在的损失。尽管如此，许多供应商仍然受到数据质量差或不一致的困扰。因此，总是需要进行清洁过程。

数据质量差的主要原因是数据冲突/不正确，多个数据源的不透明聚整合和纠错（“回填”）。

数据冲突和不正确

错误数据可能发生在数据里的任何位置。交易所软件发生的错误可能会在匹配交易时导致错误的价格。这会传导到供应商和交易者。

（译者按：我想到了国内某所）信誉良好的供应商将试图标记“错误”，并经常将这些点的“修正”留给交易者。

8.5.2 连续期货合约

在本节中，我们将讨论从回测的角度提出数据期货合约的特征。特别是“连续合同”的概念。我们将概述期货数据处理的主要困难，并提供一个可以部分缓解问题的 `pandas` 实现。

期货合约简介

期货是双方在未来指定日期购买或出售一定数量相关资产的一种合约形式。 此日期称为交割日或到期日。 当达到此日期时，买方必须按照合同订立日期约定的价格向卖方交付实物相关（或现金等价物）。

关于期货基础知识略！

本节的其余部分将集中讨论构建本地数据库方法，因为这最适合回溯。这是开展策略框架研究的有效方式。

我们将把 WTI 原油“近”和“远”期货合约（符号 CL）拼接在一起，以产生连续的价格序列。在撰写本文时（2014 年 1 月），近期合约为 CLF2014（1 月），远期合约为 CLG2014（2 月）。

为了进行期货数据的下载，我使用了 **Quandl** 插件。 确保在系统上设置正确的 **Python** 虚拟环境，并通过在终端中键入以下内容来安装 **Quandl** 包：

```
pip install Quandl
```

现在已经安装了 Quandl 软件包，我们需要使用 NumPy 和 pandas 来执行。创建一个新文件并输入以下 import 语句：


```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# cont_futures.py

from __future__ import print_function

import datetime

import numpy as np
import pandas as pd
import Quandl
```

主要工作在 `futures_rollover_weights` 函数中执行。 它需要一个开始日期（近合同的第一个日期），合同结算日期字典（`expiry_dates`），合同的代号和合同的天数（默认为五个）。 以下解释了代码：

```
def futures_rollover_weights(start_date, expiry_dates,
                             contracts, rollover_days=5):
    """This constructs a pandas DataFrame that contains weights
    (between 0.0 and 1.0) of contract positions to hold in order to
    carry out a rollover of rollover_days prior to the expiration of
    the earliest contract. The matrix can then be 'multiplied' with
    another DataFrame containing the settle prices of each
    contract in order to produce a continuous time series
    futures contract."""

    # Construct a sequence of dates beginning
    # from the earliest contract start date to the end
    # date of the final contract
    dates = pd.date_range(start_date, expiry_dates[-1], freq='B')

    # Create the 'roll weights' DataFrame that will store the multipliers for
    # each contract (between 0.0 and 1.0)
    roll_weights = pd.DataFrame(np.zeros((len(dates), len(contracts))),
                                index=dates, columns=contracts)

    prev_date = roll_weights.index[0]

    # Loop through each contract and create the specific weightings for
    # each contract depending upon the settlement date and rollover_days
    for i, (item, ex_date) in enumerate(expiry_dates.iteritems()):
        if i < len(expiry_dates) - 1:
```

```

roll_weights.ix[prev_date:ex_date - pd.offsets.BDay(), item] = 1
roll_rng = pd.date_range(end=ex_date - pd.offsets.BDay(),
                          periods=rollover_days + 1, freq='B')

# Create a sequence of roll weights (i.e. [0.0,0.2,...,0.8,1.0]
# and use these to adjust the weightings of each future
decay_weights = np.linspace(0, 1, rollover_days + 1)
roll_weights.ix[roll_rng, item] = 1 - decay_weights
roll_weights.ix[roll_rng,
                expiry_dates.index[i+1]] = decay_weights
else:
    roll_weights.ix[prev_date:, item] = 1
prev_date = ex_date
return roll_weights

```

现在已经生成了加权矩阵，可以将其应用于各个时间序列。 `main` 函数 `near` 和 `far`，为两者创建单个 `DataFrame`，构造 rollover 加权矩阵，然后最终产生一系列连续的价格，并适当加权：

```

if __name__ == "__main__":
    # Download the current Front and Back (near and far) futures contracts
    # for WTI Crude, traded on NYMEX, from Quandl.com. You will need to
    # adjust the contracts to reflect your current near/far contracts
    # depending upon the point at which you read this!
    wti_near = Quandl.get("OFDP/FUTURE_CLF2014")
    wti_far = Quandl.get("OFDP/FUTURE_CLG2014")
    wti = pd.DataFrame({'CLF2014': wti_near['Settle'],
                       'CLG2014': wti_far['Settle']}, index=wti_far.index)

    # Create the dictionary of expiry dates for each contract
    expiry_dates = pd.Series(
        {'CLF2014': datetime.datetime(2013, 12, 19),
         'CLG2014': datetime.datetime(2014, 2, 21)}).order()

    # Obtain the rollover weighting matrix/DataFrame
    weights = futures_rollover_weights(wti_near.index[0],
                                       expiry_dates, wti.columns)

    # Construct the continuous future of the WTI CL contracts
    wti_cts = (wti * weights).sum(1).dropna()

    # Output the merged series of contract settle prices
    print(wti_cts.tail(60))

```

输出如下：

```
2013-10-14    102.230
2013-10-15    101.240
2013-10-16    102.330
2013-10-17    100.620
2013-10-18    100.990
2013-10-21     99.760
2013-10-22     98.470
2013-10-23     97.000
2013-10-24     97.240
2013-10-25     97.950
```

```
..
```

```
..
2013-12-24     99.220
2013-12-26     99.550
2013-12-27    100.320
2013-12-30     99.290
2013-12-31     98.420
2014-01-02     95.440
2014-01-03     93.960
2014-01-06     93.430
2014-01-07     93.670
2014-01-08     92.330
```

```
Length: 60, dtype: float64
```

可以看出，该数据现在在两个合约中都是连续的。 这可以扩展到处
理多合约多次换月（交割），具体取决于您的回测需求。

Part IV

建模

第 9 章

统计学习

本书中“建模”部分的目的是提供一个强大的量化框架，用于识别金融市场数据中的关系，利用统计学习的方法可以让这些关系可以产生有利可图的交易策略。本章描述了可用于为金融交易创建量化模型的统计学习内容和相关技术。

9.1 什么是统计学习？（这段部分类似金融学里面关于 ATP 多因子的内容）

在讨论统计学习的理论方面之前，应考虑这种技术是否适用的量化情形。一个量化基金，希望对标准普尔 500 股票市场指数做出长期预测。该基金设法收集了与构成该指数的公司相关的大量基本数据。例如，基本数据包括市盈率或账面价值。基金如何利用这些数据来预测指数以创建交易工具？统计学习为这个问题提供了一种方法。

在更加量化的意义上，我们试图基于假设两者之间的关系的一组预测因子或特征来模拟结果或响应的行为。在上面的例子中，股票市场指数值是响应，与成分公司相关的基本数据是预测因子。（译者按：这类似于主成分分析 PCA，读者可以查阅相关内容可更好的理解）

这可以通过考虑具有 p 个不同特征 $x_1, x_2, \dots, x_p$ 的映射 Y 来函数化。如果我们利用表达式表示，那么我们可以定义 $X = (x_1, x_2, \dots, x_p)$ ，特征因子总数为 p 。然后我们的关系模型由下式给出：

$$Y = f(X) + \epsilon \quad (9.1)$$

其中 f 是预测变量的未知函数， ϵ 代表错误或噪音。重要的是 ϵ 不

依赖于预测因子，假设其均值为零（译者按：这都是作者的科学假设）。这是为了表示 f 内未考虑的信息。因此，我们可以回到股市指数的例子，说 Y 代表标准普尔 500 指数的价值，而 x_i 成分代表个别基本面因素的价值。统计学习的目标是根据观察到的数据来计算 f 的函数，并评估这些计算的准确程度。

9.1.1 预测和推断

统计学习中有两个常见的类型 - 预测和推断。

预测涉及基于新观察到的通过 X 来预测 Y ，如果已经确定模型关系，则使用 f 的估计来预测响应是简单的，以产生响应的公式：

$$\hat{Y} = \hat{f}(X) \quad (9.2)$$

如果计算接近现实并且其预测准确，则 f 的函数就无需考虑太多。对 f 的不同计算将产生 Y 的估计的各种准确度问题。现实与 f 函数相差的 f 相关性的误差称为可减少误差（译者按：这往往是因为特征没有被提取干净）。请注意，总有一定程度的不可减少的错误，因为我们上面说的函数包括 ϵ 。此误差可能影响响应 Y 的未测量因子。所采取的方法是尝试最小化 ϵ 误差，同时寻找基于 ϵ 的上限。

推断涉及需要理解 X 和 Y 之间关系的情况，因此必须确定其确切形式。人们可能希望识别重要的预测因子或确定个体预测因子与响应之间的关系。人们想知道这种关系是线性的还是非线性的。前者意味着该模型要具可解释性，但代价可能是更糟糕的可预测性。后者提供的模型通常更具预测性，但要忍受可解释性较差。因此，经常存在可预测性和可解释性之间的权衡。

在本书中，我们不太关心推理模型，因为 f 函数的可解释性并不像其进行准确预测那样重要。因此，本书中“建模”部分的一个重要组成部分将基于预测建模。下一节将讨论如何构建模型的建立 f 函数。

9.1.2 参数和非参数模型

在统计学习情境中，通常可以构造一组预测变量元组和 $\{(X_1, Y_1), (X_2, Y_2), \dots, (X_N, Y_N)\}$ 形式与之对应的响应，其中 X_j 指的是第 j 个预测变量因子而不是特定预测变量因子的第 j 个分量（由 x_j 表示）。这种形式的数据集称为训练数据，因为它将用于训练关于如何生成 f 函数的统计学习方法。为了实际得出 f ，我们需要找到一个 f 函数，能让 X 变量得出 Y 的合理近似。有两大类统计模型允许我们实现这一点。它们被称为参数和非参数模型。

参数模型

参数方法的定义特征是需要定义 f 的形式的规范或假设。这是一个建模基础。第一种选择是考虑线性还是非线性模型。让我们考虑线性模型的简单情况。这样的模型减少了从维度 p 的某个未知函数的计算到计算长度为 $p+1$ 的系数向量 $\beta = (\beta_0, \beta_1, \dots, \beta_p)$ 的问题。（译者按：其实是个最优欧式距离问题）

为什么 $p+1$ 而不是 p ？由于线性模型是仿射的，即在创建“最优拟合线”时它们可能不会通过原点，因此需要系数来确定“截距”。在一维线性模型（回归）设置中，这通常表示为 α 。对于我们的多维线性模型，在有 p 个预测变量的情况下，我们需要一个附加值 β_0 来表示我们的截距，因此 β 的 β 估计值中有 $p+1$ 个分量。

现在我们已经指定了 f 的（线性）函数形式，我们需要训练它。在这种情况下，“训练”意味着找到 β 的估计值，以便得出：

$$Y \approx \hat{\beta}^T X = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p \quad (9.3)$$

在线性设置中，我们可以使用诸如普通最小二乘（OLS）之类的算法，但也可以使用其他方法。估计 β 比拟合（可能是非线性的） f 简单得多。然而，通过选择参数线性方法，我们算出的 f 函数不太可能拟合 f 的真实形式，因为该模型不够灵活。

一种潜在的补救措施是为 f 添加更多参数。不幸的是，如果模型过于“灵活”，可能会导致一种非常危险的情况，即过度拟合，我们将在后续章节中详细讨论。从本质上讲，会导致该模型紧密地跟踪噪声，而不是信号。

非参数模型

另一种方法是考虑 f 的非参数形式。其好处是它可以更加符合 f 函数的存在形式，因此更灵活。不幸的是，非参数模型需要具有大量的观测数据，通常远远超过参数设置。此外，如果不仔细处理，非参数方法也易于过度拟合，如上所述。

非参数模型似乎是量化交易模型的最优选择，quant 有大量的（历史）数据可用于应用模型，但是这些方法并不总是最优的。虽然增加的灵活性对于对股票市场数据的非线性进行建模很有吸引力，但由于金融时间序列中的信号/噪声比较差，因此很容易过度拟合数据。因此，具有一定程度灵活性模型的“中间部分”是优选的。我们将在本书后面的“优化”一章中讨论这些问题。

9.1.3 监督和无监督学习（译者按：关于机器学习理论方面本人推荐《统计学习方法》by 李航，如果应用层建议初学者可以学习 `scikit-learn` 库以及深度学习库 `keras`（如果用使用 `python` 语言）作为基础）

在监督和无监督方法之间通常是对统计机器学习进行区分。在本书中，我们几乎只考虑监督技术，但无监督技术肯定也适用于金融市场。

有监督模型要求对于每个预测变量 x_j 存在相关联的结果 y_j 。当 f 的函数模型被训练，过程的“监督”用于该特定数据。例如，当拟合线性回归模型时，**OLS** 算法用于训练它，最终对回归系数 β 计算。

在无监督模型中，对于任何特定感知器使 x_j 没有对应的响应 y_j 。因此，没有什么可以“监督”模型的训练。对于算法产生结果而言，这显然是一个更具挑战性的环境，因为没有任何形式的“奖惩函数”来测试准确性。尽管有这些不同，但无人监督学习的技术非常强大。它们在聚类领域特别有用。

当参数化聚类模型提供有指定要识别的聚类数量的参数时，通常可以辨别数据中可能不容易确定的未被发觉到的关系。这些模型通常属于业务分析和消费者营销优化领域，但它们确实在金融中 useful，特别是在评估波动性的聚类方面。

本书主要集中在监督学习方法上，因为有大量的历史数据可用于训练这些模型。

9.2 技术

机器学习是一门庞大的跨学科领域，有许多不同的研究领域。本章的其余部分将阐述与量化和算法交易最相关的技术。

9.2.1 回归

回归指的是一种普遍的监督机器学习技术，可以实现预测和推理功能。量化交易的很大一部分使用回归技术，因此熟悉流程至关重要。回归试图模拟因变量和一组自变量（预测变量）之间的关系。特别地，回归的目标是在假设剩余的自变量保持固定的情况下确定其中一个自变量发生变化时产生的变化。

最广为人知的回归技术是线性回归，它假设预测因子和响应之间存在线性关系。这样的模型导致对每个预测器的线性响应的参数估计（通常由 β 表示）。通过称为普通最小二乘法（OLS）的过程计算这些参数。线性回归可用于预测和推断。

在前一种情况下，可以添加感知器的新值（没有相应的响应）以便感知新的响应值。例如，考虑一个线性回归模型，用于根据过去五天的价格数据预测第二天标准普尔 500 指数的价值。可以使用 OLS 跨历史数据拟合模型。然后，当新的市场数据产生标准普尔 500 参数时，可以将其输入模型（作为感知器），以生成明天每日价格的预测价格。这可以构建一个简单交易策略。

在后一种情况中，可以测算结果与每个预测因子之间关系的强度，以便确定对结果有影响的预测因子集。当目标是了解价格变化的原因时，例如在市场研究或现实实验中，这更有用。对于那些进行算法交易的人来说，推测通常没那么有用，因为预测的结果从根本上比潜在的关系更重要。话虽如此，由于数据中过度拟合噪声的普遍存在，人们不应仅仅依赖于“黑箱”方法。

其他技术包括 Logistic 回归，其设计用于预测分类预测（例如

“UP”，“DOWN”，“FLAT”）而不是连续预测（例如股票市场价格）。这在技术上使它成为一种分类工具（见下文），但它通常归入回归的类别。比如作为最大似然估计（MLE）的一般统计过程用于估计逻辑回归的参数值。

9.2.2 分类

分类包括有监督机器学习技术，其旨在基于与观察相关联的特征将观察（类似于感知器）分类为一组特定类别。这些类别可能是无序的，例如，“红色”，“黄色”，“蓝色”或有序，例如“低中高”。在后一种情况下，这种分类组称为序数。分类算法 - 分类器 - 广泛用于量化金融，特别是在市场方向预测领域。在本书中，我们将广泛研究分类器。

可以在算法交易中使用分类器来预测特定时间序列在后续（未知）时间段内是否具有正或负回报。这类似于回归，它没有预测时间序列的实际值，而是预测它的方向。但我们能够使用连续预测，例如先前的市场价格作为观察。我们将考虑线性和非线性分类器，包括 Logistic 回归，

线性/二分法分析，支持向量机（SVM）和人工神经网络（ANN）。请注意，以前的一些方法实际上也可以用于回归。

9.2.3 时间序列模型

算法交易的一个关键组成部分是金融时间序列的处理和预测。我们的目标通常是根据先前值或外部因素预测时间序列的未来值。因此，时间序列建模可以看作是回归和分类的混合集。时间序列模

型与非时间模型不同，因为模型有意地使用了序列的时间顺序。因此，预测变量通常基于过去或当前值，而价格通常是要预测的未来值。

关于不同时间序列模型的资料很多。时间序列模型有两大类：第一组是线性自回归积分移动平均（**ARIMA**）模型系列，用于模拟时间序列绝对值的变化。时间序列的另一组是自回归条件异方差性（**ARCH**）模型，其用于模拟时间序列随时间的方差（即波动率）。**ARCH**模型使用时间序列的先前值（波动率）来预测未来值（波动率）。这与随机波动率模型形成对比，随机波动率模型利用多于一个随机时间序列（即多个随机微分方程）来模拟波动率。（译者按：隐含波动假设不同，这是两种理念）

所有原始历史价格时间序列都是离散的，因为它们包含有限值。在量化金融领域，研究连续时间序列模型是很常见的。特别是，著名的几何布朗运动，赫斯顿随机波动率模型和 **Ornstein-Uhlenbeck** 模型都表示具有不同形式的随机行为的连续时间序列。我们将在后续章节中利用这些时间序列模型来尝试描述金融时间序列的行为，以便利用其属性来创建可行的交易策略。

第 10 章

时间序列分析

在本章中，我们将进行统计测算，以帮助我们识别具有趋势或均值回归特征的价格序列。如果我们能够从统计上识别出这样的序列，那么我们就可以通过形成动量或均值回归交易策略来利用这种行为。在后面的章节中，我们将使用这些统计测试来帮助我们确定候选时间序列，然后围绕它们创建算法策略。

10.1 均值回归测试

其中一个关键的量化交易概念是均值回归。此过程是指恢复到历史平均值的时间序列。我们进入市场时，当价格序列远离均值时，可以利用这样的时间序列来产生交易策略，期望该资产价格将回归到平均值，从而使我们平仓市场获利。均值回归策略构成了统计套利量化对冲基金的一个重要组成部分。在后面的章节中，我们将创建利用均值回归特征的日内和日间策略。

要确定时间序列是否为均值回复的基本思想是使用统计检验来测算它是否与随机游走的行为不同。随机游走是一个时间序列，其中下一次运动完全独立于任何过去的运动 - 实质上，随机时间序列没有“记忆”它的过去。然而，均值回复时间序列是不同的。下一时间段中时间序列值的变化与当前值成比例。具体而言，它与平均历史价格和当前价格之间的差异成比例。

在数学上，这种（连续的）时间序列被称为 Ornstein-Uhlenbeck 过程。如果我们能够在统计上证明价格序列的行为类似于 Ornstein

Uhlenbeck 过程，那么我们就可以开始形成围绕它的交易策略。因此，本章的目标是概述识别均值回归所必需的统计测试，然后使用 Python 库（特别是 statsmodels）来实现这些测试。特别是，我们将研究平稳性的概念以及如何测试它。

如上所述，连续均值回复时间序列可以用 OrnsteinUhlenbeck 随机微分方程表示：

$$dx_t = \theta(\mu - x_t)dt + \sigma dW_t \quad (10.1)$$

其中 θ 是回归平均值的速率， μ 是过程的平均值， σ 是过程的方差， W_t 是维纳过程或布朗运动。

该公式基本上表明，在下一个连续时间段内价格序列的变化与平均价格和当前价格之间的差异成比例，并且增加了高斯噪声。

我们可以使用这个等式 Augmented Dickey-Fuller Test 定义，我们现在将对此进行描述。

10.1.1 迪基-福勒检验（ADF）

ADF 检验利用了这样一个事实：如果价格序列具有均值回归，则下一个价格水平将与当前价格水平成比例。在数学上，ADF 基于在自回归时间序列样本中测试单位根的存在想法。

我们可以考虑时间序列的模型，称为阶数为 p 的线性滞后模型。该模型表示时间序列值的变化与常数，时间本身和时间序列的先前 p 值以及误差项成正比：

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \cdots + \delta_{p-1} \Delta y_{t-p+1} + \epsilon_t \quad (10.2)$$

译者按：CSDN 网站上面有关于 ADF 的文章，比本书的要好不少，所以理论内容我就不翻译了，主要讲实现部分

要计算 Augmented Dickey-Fuller 测试，我们可以使用 pandas statsmodels 库。前者为我们提供了一种从 Yahoo Finance 获取 Open-High-LowClose-Volume (OHLCV) 数据的直接方法，而后者则将 ADF 测试包含在一个易于调用的函数中。这使我们无需手动计算测试统计数据，从而节省了时间。

我们将从 2000 年 1 月 1 日到 2015 年 1 月 1 日对亚马逊股票作为样本价格系列进行 ADF 测试。

以下是执行测试的 Python 代码：

```
from __future__ import print_function

# Import the Time Series library
import statsmodels.tsa.stattools as ts

# Import Datetime and the Pandas DataReader

from datetime import datetime
import pandas.io.data as web

# Download the Amazon OHLCV data from 1/1/2000 to 1/1/2015
amzn = web.DataReader("AMZN", "yahoo", datetime(2000,1,1), datetime(2015,1,1))

# Output the results of the Augmented Dickey-Fuller test for Amazon
# with a lag order value of 1
ts.adfuller(amzn['Adj Close'], 1)
```

以下是亚马逊在此期间的 Augmented Dickey-Fuller 检验的输出。第一个值是计算的 test-statistic，而第二个值是 p 值。第四个是样本中的数据点数。第五个分别包含 1%，5% 和 10% 值的测试统计值的临界值。

```
(0.049177575166452235,  
0.96241494632563063,  
1,  
3771,  
{ '1%': -3.4320852842548395,  
  '10%': -2.5671781529820348,  
  '5%': -2.8623067530084247},  
19576.116041473877)
```

由于检验统计量的计算值大于 1%，5% 或 10% 的任何临界值，我们不能否定 $\gamma = 0$ 的零假设，因此我们不太可能找到平均回复时间序列。这与我们的猜测一致，因为大多数股票的行为类似于几何布朗运动（GBM），即随机游走。以上是我们如何利用 ADF 测试的结果。然而，有其他方法可以检测均值回归，特别是通过平稳性的概念，我们现在将讨论。

10.2 平稳性测试

如果时间序列（或随机过程）的联合概率分布在时间或空间的转换下是不变的，则定义为强平稳。特别是对交易者而言至关重要，过程的均值和方差不随时间或空间而变化，并且每个都不遵循趋势。

固定价格序列的一个关键特征是该价格序列从其初始值以低于 GBM 的速率扩散。通过检验这种扩散的速率，我们可以识别时间序列的性质，从而检测它是否是均值回归。

我们现在将概述一个计算，即 Hurst 指数，它可以帮助我们表征时间序列的平稳性。

10.2.1 赫斯特指数（译者按：我觉得从分形的角度理解 hurst 指数会更好）

赫斯特指数的目标是为我们提供一个指标，帮助我们识别（在统计估

计的范围内) 序列是否意味着回归, 随机行走或趋势。

Hurst Exponent 计算背后的想法是我们可以使用对数价格序列的方差来检验扩散的速率。对于任意时滞 τ , τ 的方差由下式给出:

$$\text{Var}(\tau) = \langle |\log(t + \tau) - \log(t)|^2 \rangle \quad (10.4)$$

我们的想法是将扩散速率与 **GBM** 的扩散速率进行比较。在 **GBM** 的情况下, 在长时间 (即当 τ 大时), τ 的方差与 τ 成正比:

$$\langle |\log(t + \tau) - \log(t)|^2 \rangle \sim \tau \quad (10.5)$$

如果我们发现不同于这种关系的现象, 那么我们已经确定了趋势或均值回复系列。关键是如果任何连续价格变动具有非零相关性 (称为自相关), 则上述关系不相关。它给出了 **Hurst Exponent** 值 H :

$$\langle |\log(t + \tau) - \log(t)|^2 \rangle \sim \tau^{2H} \quad (10.6)$$

因此可以看出, 如果 $H = 0.5$, 是 **GBM**。但是, 如果 H 不等于 0.5 , 那么我们就有趋势或均值回复行为。尤其是:

- $H < 0.5$ - 时间序列是平均还原
- $H = 0.5$ - 时间序列是几何布朗运动
- $H > 0.5$ - 时间序列趋势

除了时间序列的表征之外, **Hurst Exponent** 还描述了系列在分类方式中的行为程度。例如, 接近 0 的 H 值是高度均值的回复序列, 而对于接近 1 的 H , 该序列是强烈趋势。

要计算亚马逊价格系列的 **Hurst Exponent**, 如上面在 **ADF** 的解释中所使用的, 我们可以使用以下 **Python** 代码:

```

from __future__ import print_function

from numpy import cumsum, log, polyfit, sqrt, std, subtract
from numpy.random import randn

def hurst(ts):
    """Returns the Hurst Exponent of the time series vector ts"""
    # Create the range of lag values
    lags = range(2, 100)

    # Calculate the array of the variances of the lagged differences
    tau = [sqrt(std(subtract(ts[lag:], ts[:-lag]))) for lag in lags]

    # Use a linear fit to estimate the Hurst Exponent
    poly = polyfit(log(lags), log(tau), 1)

    # Return the Hurst exponent from the polyfit output
    return poly[0]*2.0

# Create a Gometric Brownian Motion, Mean-Reverting and Trending Series
gbm = log(cumsum(randn(100000))+1000)
mr = log(randn(100000)+1000)
tr = log(cumsum(randn(100000)+1)+1000)

# Output the Hurst Exponent for each of the above series
# and the price of Amazon (the Adjusted Close price) for
# the ADF test given above in the article
print("Hurst(GBM):    %s" % hurst(gbm))
print("Hurst(MR):     %s" % hurst(mr))
print("Hurst(TR):     %s" % hurst(tr))

# Assuming you have run the above code to obtain 'amzn'!
print("Hurst(AMZN):  %s" % hurst(amzn['Adj Close']))

```

Hurst Exponent Python 代码的输出如下：

```

Hurst(GBM):    0.502051910931
Hurst(MR):     0.000166110248967
Hurst(TR):     0.957701001252
Hurst(AMZN):   0.454337476553

```

从这个输出中我们可以看到 GBM 拥有一个 Hurst 指数 H 几乎正好是 0.5。平均回归 MR 的 H 几乎等于零，而趋势系列的 H 接近于 1。有趣的是，亚马逊的 H 也接近 0.5，至少在我们正在使用的样本期间，表明它几乎与 GBM 类似！

10.3 协整

实际上很难找到具有均值回归行为的可交易资产。股票广泛表现得

像 GBM，因此使得均值回归交易策略相对无用。然而，没有什么能阻止我们创建一个稳定价格序列组合。因此，我们可以将均值回归交易策略应用于投资组合。

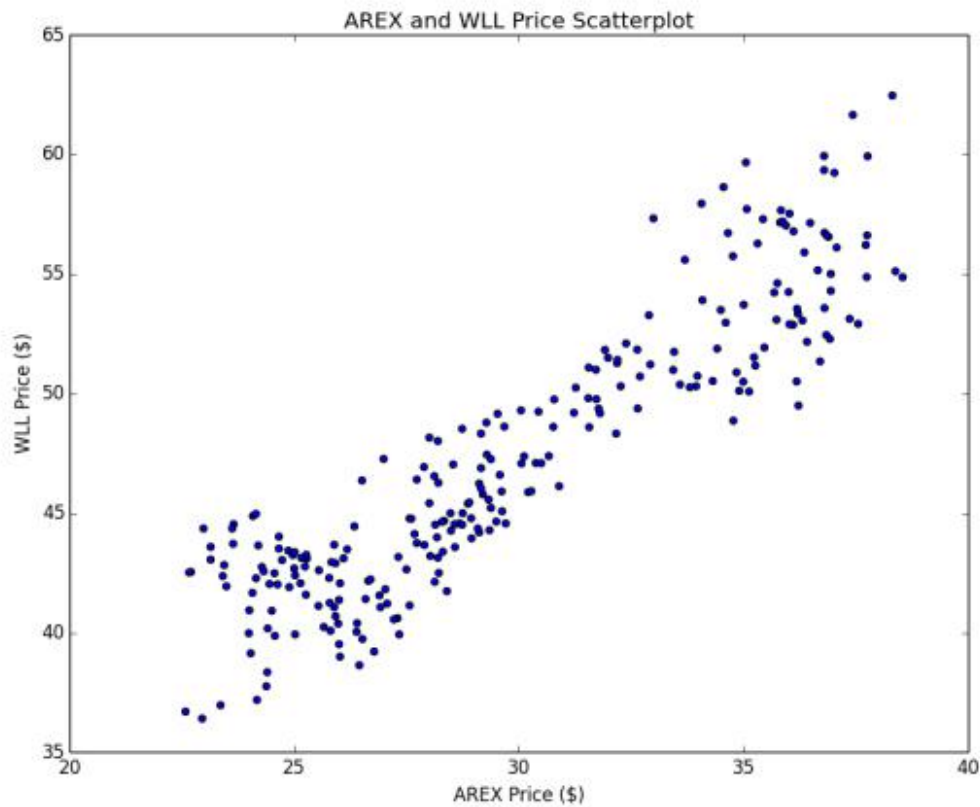
最简单的均值回归交易策略形式是经典的“配对交易”，通常涉及货币中性的多空做空股票。该理论认为，同一行业的两家公司可能会面临影响其业务的类似市场因素。偶尔他们的相对股票价格会因某些事件而发生分歧，但差值会恢复到长期运行的平均值。

让我们考虑两个能源资源行业公司股票由股票代码 WLL 和 AREX 和。两者都有类似的市场条件，因此可能具有稳定的配对关系。我们现在将创建一些图，使用 pandas 和 Matplotlib 库来演示 AREX 和 WLL 的协整特性。第一个图（图 10.1）显示了 2012 年 1 月 1 日至 2013 年 1 月 1 日期间的各自价格历史。

Figure 10.1: Time series plots of AREX and WLL



如果我们创建一个配对交易价格的散点图，我们会发现这段时间的关系是大致线性的（见图 10.2）。



$$y(t) = \beta x(t) + \epsilon(t) \quad (10.7)$$

其中 $y(t)$ 是 AREX 股票的价格，而 $x(t)$ 是 WLL 股票的价格，两者都在第 t 天。如果我们绘制残差 $\epsilon(t) = y(t) - \beta x(t)$ （我们将确定的特定 β 值）我们创建一个新的时间序列，乍一看，看起来相对固定。

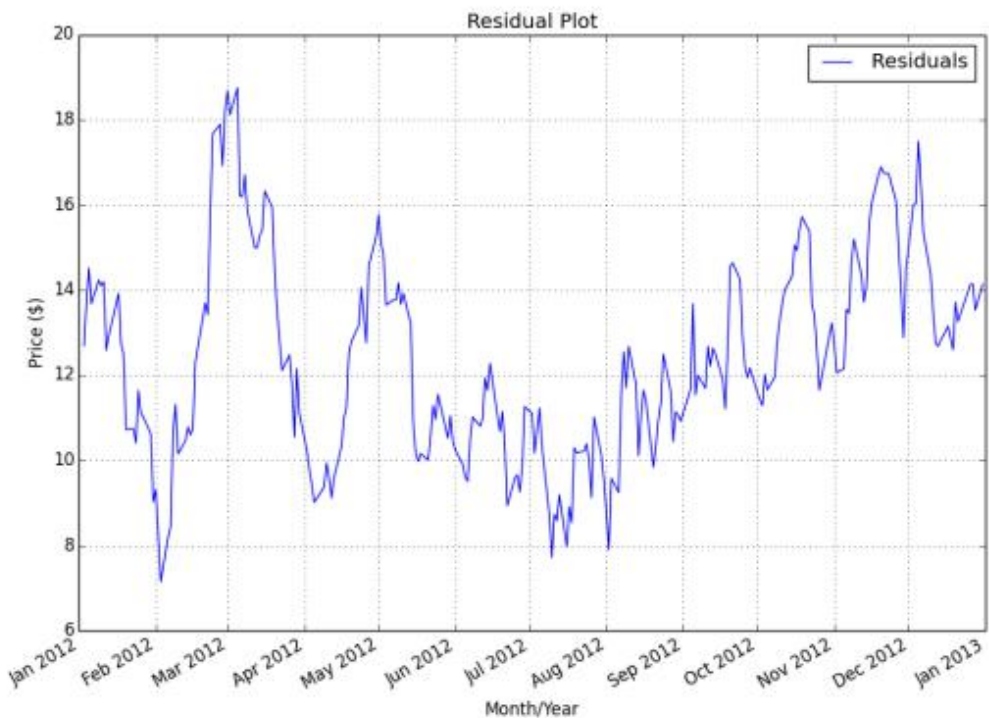


图 10.3

我们将在下面描述每个图的代码。

10.3.1 Cointegrated Augmented Dickey-Fuller 检验

为了统计地确认该系列是否是均值回复，我们可以使用上面描述的测试之一，即 **Augmented Dickey-Fuller Test** 或 **Hurst Exponent**。然而，这些测试都不能真正帮助我们确定形成线性组合所需的 β ，它们只会告诉我们，对于特定的 β ，线性组合是否是稳定的。

这是 **Cointegrated Augmented Dickey-Fuller (CADF)** 测试的用武之地。它通过对两个时间序列执行线性回归确定最佳多空比率，然后在线性组合下测试平稳性。

Python 实现

我们现在将使用 **Python** 库来测试 2012 年 1 月 1 日至 2013 年 1 月 1

日期间 AREX 和 WLL 之间的协整关系。我们将使用 Yahoo Finance 作为数据源，使用 Statsmodels 进行 ADF 测试，如上所述。第一项任务是创建一个新文件 `cadf.py`，并导入必要的库。该代码使用 NumPy，Matplotlib，Pandas 和 Statsmodels。为了正确标记轴并通过 pandas 从 Yahoo Finance 下载数据，我们导入了 `matplotlib.dates` 模块和 `pandas.io.data` 模块。我们还利用 pandas 的普通最小二乘（OLS）功能：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# cadf.py

import datetime
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
import pandas.io.data as web
import pprint
import statsmodels.tsa.stattools as ts

from pandas.stats.api import ols
```

第一个函数 `plot_price_series` 将 pandas DataFrame 作为输入，使用“ts1”和“ts2”列。调用股票数据。该功能只是在同一图表上绘制两个价格序列。这使我们可以直观地检查是否可能存在协整。我们使用 Matplotlib 日期模块从 datetime 对象中获取月份。然后我们创建一个图形和一组轴来应用标记/绘图。最后，我们绘制图：

```
# cadf.py

def plot_price_series(df, ts1, ts2):
    months = mdates.MonthLocator() # every month
    fig, ax = plt.subplots()
    ax.plot(df.index, df[ts1], label=ts1)
```

```

ax.plot(df.index, df[ts2], label=ts2)
ax.xaxis.set_major_locator(months)
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.set_xlim(datetime.datetime(2012, 1, 1), datetime.datetime(2013, 1, 1))
ax.grid(True)
fig.autofmt_xdate()

plt.xlabel('Month/Year')
plt.ylabel('Price ($)')
plt.title('%s and %s Daily Prices' % (ts1, ts2))
plt.legend()
plt.show()

```

第二个函数 `plot_scatter_series` 绘制了两个价格的散点图。这使我们直观地检查两个系列之间是否存在线性关系，从而确定它是否适合 OLS 程序和随后的 ADF 测试：

```

# cadf.py

def plot_scatter_series(df, ts1, ts2):
    plt.xlabel('%s Price ($)' % ts1)
    plt.ylabel('%s Price ($)' % ts2)
    plt.title('%s and %s Price Scatterplot' % (ts1, ts2))
    plt.scatter(df[ts1], df[ts2])
    plt.show()

```

第三个函数 `plot_residuals` 用于绘制两个价格序列的拟合线性模型的残差值。此函数要求 pandas DataFrame 有一个“res”列，表示剩余价格：

```

# cadf.py

def plot_residuals(df):
    months = mdates.MonthLocator() # every month
    fig, ax = plt.subplots()
    ax.plot(df.index, df["res"], label="Residuals")
    ax.xaxis.set_major_locator(months)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    ax.set_xlim(datetime.datetime(2012, 1, 1), datetime.datetime(2013, 1, 1))
    ax.grid(True)
    fig.autofmt_xdate()

    plt.xlabel('Month/Year')
    plt.ylabel('Price ($)')
    plt.title('Residual Plot')
    plt.legend()

    plt.plot(df["res"])
    plt.show()

```


最后，该过写在在__main__函数中。 第一项任务是从雅虎财经下载 AREX 和 WLL 的 OHLCV 数据。 然后我们创建一个单独的 DataFrame，df，使用与 AREX 框架相同的索引来存储两个调整后的收盘价格。 然后我们绘制价格序列和散点图。

绘图完成后，通过调用 WLL 和 AREX 系列上的 pandas ols 函数计算残差。 这允许我们计算 β 套期保值比率。 然后通过形成 WLL 和 AREX 的线性组合，使用多空比率来创建“res”列。

最后绘制残差并对计算的残差进行 ADF 测试。

然后我们打印 ADF 测试的结果：

```
# cadf.py

if __name__ == "__main__":
    start = datetime.datetime(2012, 1, 1)
    end = datetime.datetime(2013, 1, 1)

    arex = web.DataReader("AREX", "yahoo", start, end)
    wll = web.DataReader("WLL", "yahoo", start, end)

    df = pd.DataFrame(index=arex.index)
    df["AREX"] = arex["Adj Close"]
    df["WLL"] = wll["Adj Close"]

    # Plot the two time series
    plot_price_series(df, "AREX", "WLL")

    # Display a scatter plot of the two time series
    plot_scatter_series(df, "AREX", "WLL")

    # Calculate optimal hedge ratio "beta"
    res = ols(y=df['WLL'], x=df["AREX"])
    beta_hr = res.beta.x

    # Calculate the residuals of the linear combination
    df["res"] = df["WLL"] - beta_hr*df["AREX"]

    # Plot the residuals
    plot_residuals(df)

    # Calculate and output the CADF test on the residuals
    cadf = ts.adfuller(df["res"])
    pprint.pprint(cadf)
```

代码的输出（以及 Matplotlib 图）如下：

```
(-2.9607012342275936,  
0.038730981052330332,  
0,  
249,  
{ '1%': -3.4568881317725864,  
  '10%': -2.5729936189738876,  
  '5%': -2.8732185133016057},  
601.96849256295991)
```

可以看出，计算出的-2.96 的检验统计量小于-2.87 的 5%临界值，这意味着我们可以拒绝零假设，即在 5%的水平上没有协整关系。因此，我们可以在合理程度的确定的情况下得出结论，AREX 和 WLL 具有协整关系，至少在所分析的时间段内是这样。

我们将在后续章节中使用这一对来创建一个实际的交易策略，实现事件驱动的回测系统。

10.4 为何进行统计测试？

从根本上说，就算法交易而言，上面概述的统计测试与应用于交易策略时产生的利润一样有用。因此，做价格/时间序列水平比较，不仅仅评估策略层面的绩效是否合理。当我们使用地简单交易分析，风险/回报指标和回撤评估时，为什么还要计算所有上述指标？

首先，任何基于时间序列统计测量的实施交易策略都有更大的样本可供使用。这是因为在计算这些统计测试时，我们正在利用每一条信息，而不是每笔交易。多空交易的数量远远少于数据，因此任何交易指标测算的统计显着性都会小得多。

其次，我们实施的任何策略都将取决于某些参数，例如滚动指标的历史时间期或用于在均值、反转设置中买卖交易的指标。因此，策略级别测算仅适用于这些参数，而统计测试对于整个时间序列样本有效。

在实践中，我们想要计算两组统计数据。 Python 通过 statsmodels

和 pandas 库，使这非常简单。额外的工作实际上是相当小的但 作用很大！

第 11 章

预测

在本章中，我们将创建一个具有统计价值的预测金融时间序列的过程。这些预测将成为进一步自动化交易策略的基础。我们将扩展前面章节中讨论的统计学习主题，并使用一组分类算法来帮助我们预测金融时间序列的市场方向。

在本章中，我们将使用 **Scikit-Learn**，一个用于 **Python** 的统计机器学习库。**Scikit-learn** 包含许多机器学习技术的“现成”实现。这不仅节省了我们实施交易算法的大量时间，而且最大限度地降低了我们自己的代码错误的风险。它还允许对用其他软件包（如 **R** 或 **C++**）编写的机器学习库进行额外的验证。如果我们需要创建自己的自定义更快执行速度实现，这可以让我们充满信心，。

我们将首先讨论使用的机器学习技术在一些环境情况下评估预测准确性的方法。然后我们将考虑可用于预测技术的预测因子以及如何选择好的因子。然后我们将考虑各种有监督的学习算法。最后，我们将尝试预测标准普尔 500 指数的日线方向，后者将成为算法交易策略的基础。

11.1 评估预测准确度

在讨论预测算法和特定学习算法的选择之前，我们必须讨论它们的特征以及如何评估它们。我们感兴趣的方法包括二分类监督。也就是说，我们将尝试预测特定未来日的回报是正还是负（即我们的金融资

产是否会上涨或下跌）。

在生成预测算法中，使用回归技术，我们将非常关注此预测的幅度以及预测与实际值的偏差。

为了评估分类器的性能，我们可以使用以下两个度量指标，即 **Hit-Rate**（准确度）和 **Confusion Matrix**（混淆矩阵也称误差矩阵，是表示精度评价的一种标准格式）。

11.1.1 准确度

我们可以向监督分类器提出的最简单的问题是“我们预测正确的方向有多少次， 占有所有预测的百分比？”。 训练准确度的定义由以下公式给出：

$$\frac{1}{n} \sum_{j=1}^n I(y_j = \hat{y}_j) \quad (11.1)$$

其中 y_j 是指特定分类器的第 j 个时间段（例如，一天）的预测（向上或向下）。 $I(y_j = \hat{y}_j)$ 是指标函数，如果 $y_j = \hat{y}_j$ 则等于 1，如果 y_j 不等于 $\hat{y}_j$ 则等于 0。

因此准确度提供关于预测算法正确预测向上或向下方向的次数的百分比值。

Scikit-Learn 提供了一种计算我们准确度的方法，作为分类/训练过程的一部分。

11.1.2 混淆矩阵

混淆矩阵（或列表）是计算准确度后的下一个逻辑步骤。 它的动机是询问“我们正确预测了上涨多少次，我们正确预测下跌了多少

次？它们是否大不相同？”。

例如，可能会发现特定算法在预测“下跌天数”时始终更准确。这就代表发现了一种提高盈利能力的策略。

混淆矩阵通过确定有监督算法的误报率（统计上称为 I 型误差）和假负率（统计上称为 II 型误差）来表示。对于二分法分类（上涨或下跌）的情况，我们将有一个 2x2 矩阵：

$$\begin{pmatrix} U_T & U_F \\ D_F & D_T \end{pmatrix}$$

在 U_T 表示正确分类的上涨周期（预测也算是上涨）， U_F 表示错误分类的周期（即预测分类为下跌，但实际是上涨）， D_F 表示错误分类的下跌周期（即预测分类为上涨，实际为下跌）， D_T 表示正确分类的下跌。除了极强的之外，**Scikit-Learn** 还为我们提供了一种计算混淆矩阵的方法，作为分类/训练过程的一部分。

11.2 因子选择

资产价格预测最重要的一个方面是选择用什么作为预测因子。有大量的潜在因子可供选择，对于不熟悉金融预测的个人来说，这似乎是无法拒绝的。然而，即使是简单的机器学习技术，当与精心选择的因子一起使用时会产生相对好的结果。但请注意通常情况并非如此。“圣杯”通常会导致预测准确性不佳。

通过试图确定资产价格的基本驱动因素来进行因子选择。那么根据定义，在标准普尔 500 指数的情况下，显然按加权方式，500 个成分将成为价格的基本驱动因子！显然，如果我们知道其成分的瞬时价格，

我们就会知道 S&P500 系列的确切价格，但是在预测价格本身时使用每个成分的先前回报历史是否有任何预测能力？

或者，我们是否可以考虑与与美国进行大量贸易国家的汇率作为价格驱动因子？我们甚至可以考虑更多基本的经济和企业因子，如利率，通货膨胀，季度盈利。预测算法的准确性在很大程度上取决于算法工程师在进行模型拟合之前确定正确因子的能力。

11.2.1 滞后价格因素和交易量

在预测时间序列时经常考虑的第一类因子是时间序列本身的先前历史值。因此，通过创建时间序列收盘价的 p 滞后可以容易地获得一组因子。考虑每日时间序列。对于每个特定当前价格，因子将是时间段 $k-1, k-2, \dots, k-p$ 的历史日价格。

除了价格系列本身，我们还可以将交易量作为指标，因为它是在使用 OHLCV 数据时提供的（例如从雅虎财经，谷歌财务或 Quandl 获得）。因此，我们可以为时间序列的每一天创建一个 $p+1$ 维特征向量，其中包含 p 时间滞后和交易量序列。这自然导致一组对 (x_k, y_k) 表示在第 k 天的 $p+1$ 维特征向量 x_k 和 y_k 的实际当前收盘价。这就是我们开始监督分类练习所需的全部内容。

下面我们将考虑标准普尔 500 指数的这种滞后时间序列，并应用多种机器学习技术，看看我们是否可以预测其方向。

11.2.2 外部因子

虽然滞后的时间序列和大量信息是时间序列分析的良好起点，但我们不会受此类数据的限制。因为还有大量的宏观经济时间序列和资产价

格可供考虑。例如，我们可以根据天气来实现商品价格的长期预测，或通过国际利率变动确定外汇价格方向的变动。

如果价格之间的这种关系可以确定并显示出具有统计意义，那么我们就能够创建一个强大的交易模型。我们不会在这里过多地讨论这件事，因为我们的目标是引入建模和机器学习技术的想法。 ， 并从 Quandl 等存储库或直接从政府统计网站获取时间序列数据很容易形成关于经济数据价格关系。

11.3 分类模型

机器学习领域非常广泛，通过学术文献每月推出新模型，有许多模型可供选择，特别是在监督分类领域。 在本章中提供详尽的监督分类器列表是不切实际的，我们将考虑该领域中一些比较流行的技术。

11.3.1 Logistic 回归

我们将考虑的第一种技术是逻辑回归（LR）。 在我们的案例中将使用逻辑回归来方法实现预测二分因变量（即“上涨”或“下跌”时期）与多个独立连续变量（例如金融资产的滞后百分比收益）之间的关系。逻辑回归模型提供特定后续时间段将被分类为“上涨”或“下跌”的概率。 因此，该模型引入了一个参数，即用于对后续时间段是“上涨”还是“下跌”进行分类的概率阈值。 下面，我们将这个阈值设为 50%（即 0.5），但它肯定会被修改以适应预测模型。

逻辑回归基于逻辑公式来模拟基于连续得到“上涨”日（ $Y = U$ ）的概率。

在这种情况下，考虑我们有兴趣预测前两个滞后回报的后续时间段的

情况，我们将用（L1，L2）表示。考虑到我们已观察到前一时间段 L1 和 L2 的回报，下面的公式给出了上涨日的概率：

$$p(Y = U|L_1, L_2) = \frac{e^{\beta_0 + \beta_1 L_1 + \beta_2 L_2}}{1 + e^{\beta_0 + \beta_1 L_1 + \beta_2 L_2}}$$

使用逻辑函数代替线性函数，因为它为 L1 和 L2 的所有值是[0,1]之间的概率。在线性回归中，这些连续变量的可以是负概率，因此我们需要另一个函数。

为了拟合模型（即估计 β_i 系数），使用最大似然法。对我们来说幸运的是逻辑回归模型的拟合和预测的实现已经由 Scikit-Learn 库处理。当我们试图预测标准普尔 500 指数的方向时，该算法将在下面阐述。

11.3.2 判别分析

判别分析是逻辑回归的另一种统计方法。虽然逻辑回归在其假设中的限制性要小于判别分析，但如果满足更严格的假设，它有更准确的预测。

我们现在将考虑线性方法和非线性判别分析方法。

线性判别分析

在逻辑回归中，我们使用逻辑回归函数模拟了看到“上涨”时段的概率，给定前两个滞后回报（ $P(Y = U | L_1, L_2)$ ）作为给定预测变量 L_i 的响应 Y 的条件分布。

在线性判别分析（LDA）中， L_i 变量的分布进行建模，给定 Y ，并且通过贝叶斯定理得到 $P(Y = U | L_1, L_2)$ 。

本质上，LDA 的结果是假设预测变量是从多元高斯分布中提取的。在计算该分布的参数之后，可以将参数插入贝叶斯定理中，以便预测观

察属于哪个类。

LDA 的一个重要数学假设是所有类（例如“上涨”和“下跌”）使用相同的协方差矩阵。

我不会详述用于计算预测所需的分布或后验概率的公式，因为 `scikit-learn` 再一次为我们处理这个问题。

Quadratic Discriminant Analysis

Quadratic Discriminant Analysis（QDA）与 LDA 密切相关。不同是现在每个类可以拥有自己的协方差矩阵。

当决策边界是非线性时，QDA 通常表现更好。当训练观察较少时（即需要减少方差时），LDA 通常表现更好。另一方面，QDA 在训练集很大时表现良好（即方差较少受到影响）。。

与 LR 和 LDA 一样，`Scikit-Learn` 负责 QDA 实现，因此我们只需要为其提供用于参数估计和预测的训练/测试数据。

11.3.3 支持向量机

为了使用支持向量机（SVM），我们需要考虑分类器的概念，该分类器通过线性切割边界分离不同的类。如果存在这种直接的分离，那么我们可以仅基于确定新特征是位于该线性分类平面之上还是之下来创建有监督分类器。实际上，在量化交易情况下很少存在这种切割，因此我们需要考虑软边际分类器或支持向量分类器（SVC）。

SVC 是尝试在特征空间中定位线性分离边界来实现，该边界通过在两个类之间创建最佳距离边界来正确地对大多数（但不是全部）训练观察进行分类。如果分类主要是线性的，有时这样的边界是非常

有效的。然而，其他时候这种分割是不可能的，并且有必要利用其他技术。

使用 SVC 是让非线性决策边界。这是支持向量机（SVM）的领域。SVM 的主要优点是它们允许非线性放大特征空间以得出识别非线性函数，同时仍然保持很好的计算效率（不是学习效率），使用称为“核心技巧”的过程。

SVM 通过许多不同的“核”实现非线性决策边界。特别是，我们可以使用二次多项式，高阶多项式甚至径向基来描述非线性边界，而不是像 SVC 中那样使用完全线性的分离边界。这使我们具有很大程度的灵活性。

我们下面将使用的 SVM 尝试通过非线性边界来划分特征空间（即滞后价格因子），这让我们对后续时间资产价格是向上移动还是向下移动做出合理的预测。

11.3.4 决策树和随机森林

决策树是一种有监督学习，它利用树结构通过树的每个节点处的“决策”将特征空间划分为递归集。

例如，可以询问昨天的价格是高于还是低于某个阈值，该阈值立即将特征空间划分为两个集。然后，对于两个集中的每一个，可以继续采用价格是高于还是低于阈值，从而创建四个集合。这个过程一直持续到分区不再有预测能力为止。

与 SVM 或判别分析器的更“黑箱”不透明方法相比，决策树提供了自然可解释的分类机制，因此是一种流行的监督学习。

随着计算能力的提高，出现了解决分类问题的新方法，即集成学习。基本思路很简单。从相同的基础数据创建大量分类器，使用不同的参数训练它们。然后将预测结果平均结合起来，希望获得的预测准确度高于任何单个成分所带来的预测。

最广泛的集成方法之一是随机森林的方法，它采用多个决策树迭代（通常数万或更多）组合预测。这样的合并预测通常表现得非常好。Scikit-Learn 在其模块中附带 RandomForestClassifier（RFC）类。

RFC 最主要的两个主要参数是 `n_estimators` 以及 `n_jobs`，前者输入的是决策树迭代数，后者输入的并行运算的数量。我们将在下面的实现部分讨论的内容。

11.3.5 主成分分析

以上所有技术属于监督学习。机器学习的另一种方法是无监督训练过程，而是允许算法自己确定“特征”。这种方法被称为无监督学习。

无监督学习的常见用例包括将问题的维数减少到仅被认为重要的那些，在大量文本文档中发现可在时间序列分析中提供预测能力的特征。

我们在本节中感兴趣的是降维的概念，旨在确定一组提供最可预测性的因子中最重要的组成部分。特别地，我们将使用称为主成分分析（PCA）的无监督学习来在我们的监督分类器中使之减小特征空间的大小。PCA 的基本思想是将一组可能相关的变量（例如时间序列自相关）转换为一组线性不相关的变量，称为主成分。这些主成分按照

它们描述的方差以正交方式排序。因此，如果我们有一个非常高维的特征空间（10 个以上的特征），那么我们可以通过 PCA 将特征空间减少到可能提供几乎所有数据变为 2 或 3 个主要特征值，从使得简化数据集的监督分类器模型更加稳健。

11.3.6 使用哪种机器学习算法？

在量化领域存在大量训练数据的情况下，应考虑使用诸如支持向量机（SVM）之类的模型。但是，SVM 缺乏可解释性。决策树和随机森林集合则不是这样。后者通常用于保持可解释性，这是“黑箱”机器学习算法（如 SVM）不提供的。

最终，当数据有很多可选时（例如，tick 数据），最终使用哪种方法将非常重要。在此阶段，出现其他影响我们采用哪种机器学习因素，例如算法的计算效率和可扩展性。宽泛的经验法则是训练数据的越多将效果越好，但随着数据量的大幅增加，计算效率越低。

有监督机器学习的统计和数学理论非常复杂，但每个机器学习的基本思想都很容易理解。另外 - 请注意，以下每个机器学习将有一组不同的假设，以确定它们何时最佳，因此如果您发现分类器性能不佳，可能是因为使用的数据集违反了以下假设。

朴素贝叶斯分类器

虽然我们在上面的示例中没有考虑过朴素贝叶斯分类器，但我想在其中包含对其完整性的讨论。Naive Bayes（特别是 Multinomial Naive Bayes - MNB）在有限的数据集时很好用。这是因为它是一种高偏差分类器。MNB 分类器的主要假设是条件独立性。从本质上讲，

它无法识别各个特征之间的关系，除非它们被添加额外识别特征。

例如，考虑文档分类情况，在量化金融领域尝试执行舆情分析时。**MNB** 可以知道诸如“猫”和“狗”之类的单词可以分别指代与猫和狗有关的意思，但是“猫和狗”（英国俚语大量下雨）这个词不会被认为是气象！ 对此的补救措施是将“猫和狗”视为额外的特征，具体而言，然后将其与气象类别联系起来。

Logistic 回归

Logistic 回归有一些优于朴素贝叶斯模型的优点，因为对特征之间的相关性的关注较少，并且根据模型的性质对结果存在概率解释。这最适合需要使用阈值的环境。 例如，我们在“上涨”或“下跌”结果上设置 80%（例如）的阈值，以便正确选择它，而不是选择最高概率类别。 在后一种情况下，“向上”的预测可以是 51%，“向下”的预测可以是 49%。 在这种情况下，将类别设置为“上涨”并不是一个非常强烈的预测。

决策树和随机森林

决策树（**DT**）将空间划分为布尔值的层次结构，从而导致基于相应决策的分类或分组。 这使得它们具有高度可解释性（假设树中有“合理”数量的决策/节点！）。 **DT** 具有许多优点，包括处理特征之间的交互以及非参数的能力。

它们对于将线性数据分离成类（这是支持向量机所需的条件）的情况也很有用但这并不简单（或不可能）。 使用单个决策树的缺点是它们易于过度拟合（高方差）。 使用随机森林解决此问题。 在机

器学习竞争中使用随机森林实际上是一些“最佳”分类器，因此应自始至终考虑随机森林。

支持向量机

支持向量机（SVM）虽然拥有复杂的拟合程序，但实际上相对简单易懂。线性 SVM 实质上尝试使用线性分离边界将空间划分为多个不同的类。对于某些类型的数据，这可以非常好地工作并能得出良好的预测结果。然而，许多数据不是线性可分的，因此线性 SVM 在这里表现不佳。

解决方案是修改 SVM 使用的核，使其具有划分非线性决策边界的能力。因此它是非常灵活的模型，但是，需要选择正确的 SVM 边界以获得最佳结果。SVM 在具有高维度的文本分类方面特别好。它们的计算复杂性，调整难度以及拟合模型难以解释的事实使它们不利于使用。

11.4 预测股指运动

标准普尔 500 指数是美国股票市场中按市值计算的 500 家最大上市公司的加权指数。它通常被用作股票基准。存在许多衍生产品以允许对指数进行投机或对冲。特别是，标准普尔 500 指数迷你指数期货合约是一种流动非常好的指数合约。

在本节中，我们将使用一组分类器来预测第 k 天收盘价的方向，仅基于第 $k-1$ 天已知的价格信息。向上的方向移动意味着 k 的收盘价高于价格在 $k-1$ ，而向下移动意味着收盘价在 k 低于 $k-1$ 。

如果我们能够以显着超过 50% 准确度的方式确定运动方向，且具有

低误差和良好的统计显著性，那么我们可以根据我们的预测形成基本的系统交易策略。

11.4.1 Python 实现

为了实现这些预测，我们将使用前面章节中安装的 NumPy, Pandas 和 Scikit-Learn。

第一步是导入相关的模块和库

用于此预测的 LogisticRegression, LDA, QDA, LinearSVC（线性支持向量机），SVC（非线性支持向量机）和 RandomForest 分类器：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# forecast.py

from __future__ import print_function

import datetime
import numpy as np
import pandas as pd
import sklearn

from pandas.io.data import DataReader
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.lda import LDA
from sklearn.metrics import confusion_matrix
from sklearn.qda import QDA
from sklearn.svm import LinearSVC, SVC
```

导入库后，我们需要创建一个 pandas DataFrame，其中包含先前天数的滞后收益率（默认为五天）。 create_lagged_series 为股票代码（由 Yahoo Finance 提供）并在指定的时间段内创建滞后的 DataFrame。代码很方便，所以可以直截的看看发生了什么：


```

def create_lagged_series(symbol, start_date, end_date, lags=5):
    """
    This creates a Pandas DataFrame that stores the
    percentage returns of the adjusted closing value of
    a stock obtained from Yahoo Finance, along with a
    number of lagged returns from the prior trading days
    (lags defaults to 5 days). Trading volume, as well as
    the Direction from the previous day, are also included.
    """

    # Obtain stock information from Yahoo Finance
    ts = DataReader(
        symbol, "yahoo",
        start_date-datetime.timedelta(days=365),
        end_date
    )

    # Create the new lagged DataFrame
    tslag = pd.DataFrame(index=ts.index)
    tslag["Today"] = ts["Adj Close"]
    tslag["Volume"] = ts["Volume"]

    # Create the shifted lag series of prior trading period close values
    for i in range(0, lags):
        tslag["Lag%s" % str(i+1)] = ts["Adj Close"].shift(i+1)

    # Create the returns DataFrame
    tsret = pd.DataFrame(index=tslag.index)
    tsret["Volume"] = tslag["Volume"]
    tsret["Today"] = tslag["Today"].pct_change()*100.0

    # If any of the values of percentage returns equal zero, set them to
    # a small number (stops issues with QDA model in Scikit-Learn)
    for i,x in enumerate(tsret["Today"]):
        if (abs(x) < 0.0001):
            tsret["Today"][i] = 0.0001

    # Create the lagged percentage returns columns
    for i in range(0, lags):
        tsret["Lag%s" % str(i+1)] = \
            tslag["Lag%s" % str(i+1)].pct_change()*100.0

    # Create the "Direction" column (+1 or -1) indicating an up/down day
    tsret["Direction"] = np.sign(tsret["Today"])
    tsret = tsret[tsret.index >= start_date]

    return tsret

```

我们将分类程序与__main__函数联系起来。 我们将尝试使用 2001 年至 2004 年的数据来预测 2005 年的美国股市走势。

首先，我们创建一个滞后的 S&P500 数组。该数组还包括交易量。但是，我们将限制预测器集仅使用前两个滞后值。因此，我们隐含地向分类者说明使用更多的滞后值会导致较低的预测价值。另外，在统计概念下更具体地研究了这种效应，尽管这超出了本书的范围。

在创建预测器数组 x 和响应向量 y 之后，我们可以将数组分为训练和测试集。前一个集用于实际训练函数，而后者用于实际测试函数性能。我们将在 2005 年 1 月 1 日拆分训练和测试集，留下完整的交易年数据（大约 250 天）。

一旦我们创建了训练/测试数据分割，我们需要创建一个分类模型数组，每个分类模型都是一个附有缩写名称的元组。虽然我们没有做 Logistic 回归，线性/二次判别分析器或线性支持向量分类器模型设置任何参数，但我们使用了一组径向支持向量机（RSVM）和随机森林（RF）的默认参数。

最后，迭代模型在训练数据上训练（拟合）每个数组，然后对测试集进行测试。最后，我们输出模型的准确度和混淆矩阵：

```

if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1", "Lag2"]]
    y = snpret["Direction"]

    # The test data is split into two parts: Before and after 1st Jan 2005.
    start_test = datetime.datetime(2005,1,1)

    # Create training and test sets
    X_train = X[X.index < start_test]
    X_test = X[X.index >= start_test]
    y_train = y[y.index < start_test]
    y_test = y[y.index >= start_test]

```

```

# Create the (parametrised) models
print("Hit Rates/Confusion Matrices:\n")
models = [("LR", LogisticRegression()),
          ("LDA", LDA()),
          ("QDA", QDA()),
          ("LSVC", LinearSVC()),
          ("RSVM", SVC(
              C=1000000.0, cache_size=200, class_weight=None,
              coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
              max_iter=-1, probability=False, random_state=None,
              shrinking=True, tol=0.001, verbose=False)
          ),
          ("RF", RandomForestClassifier(
              n_estimators=1000, criterion='gini',
              max_depth=None, min_samples_split=2,
              min_samples_leaf=1, max_features='auto',
              bootstrap=True, oob_score=False, n_jobs=1,
              random_state=None, verbose=0)
          )]

```

```

# Iterate through the models
for m in models:

    # Train each of the models on the training set
    m[1].fit(X_train, y_train)

```

```
# Make an array of predictions on the test set
pred = m[1].predict(X_test)

# Output the hit-rate and the confusion matrix for each model
print("%s:\n%0.3f" % (m[0], m[1].score(X_test, y_test)))
print("%s\n" % confusion_matrix(pred, y_test))
```

11.4.2 结果

所有分类模型的输出如下。您可能会在 RF（随机森林）输出中看到不同的值，因为它在构造中有自带的随机性：

```
Hit Rates/Confusion Matrices:

LR:
0.560
[[ 35  35]
 [ 76 106]]

LDA:
0.560
[[ 35  35]
 [ 76 106]]

QDA:
0.599
[[ 30  20]
 [ 81 121]]

LSVC:
0.560
[[ 35  35]
 [ 76 106]]

RSVM:
0.563
[[ 9  8]
 [102 133]]

RF:
0.504
[[48 62]
 [63 79]]
```

请注意，所有准确度介于 50% 和 60% 之间。因此，我们可以看到滞后变量并不是预测未来方向的重要指标。但是，如果我们看一下二次判别分析器，我们可以看到它在测试集上的整体预测性能不到 60%。

该模型（以及其他一般模型）的混淆矩阵也表明“下跌”日的正

确率远远高于“上涨”日。因此，如果我们要根据这些信息制定交易策略，我们可以考虑将交易限制在标准普尔 500 指数的空头头寸，作为提高盈利能力的潜在手段。

在后面的章节中，我们将这些模型作为交易策略的基础，将它们直接纳入事件驱动的回测框架并直接使用（如交易所交易基金（ETF））。

Part V 绩效和风险管理

第 12 章

绩效评估

绩效测算是算法交易的绝对关键组成部分。如果不对绩效进行评估，再加上可靠的历史记录，很难确定我们的策略回报是由于运气还是由于策略产生的超额收益。

为了在算法交易中取得成功，有必要了解可能影响交易盈利能力的因素，并最终了解策略。我们应该不断尝试在算法交易迭代实现各个方面的改进。特别是我们应该始终尽量减少交易成本（费用，佣金和滑点），改进我们的软件和硬件，提高数据源的清洁度，并不断寻找新的策略来增加投资组合。所有这些领域的绩效衡量提供了衡量策略的标准。

最终，算法交易就是要产生利润。因此，我们必须在多个细粒度级别上衡量我们的系统如何以及为何产生这些利润。这促进了交易，策略和投资组合层面的绩效评估。特别是我们正在寻找：

- 策略编制的系统规则是否实际产生了一致的回报，以及该策略是否在回测中具有积极的表现。
- 策略是否在实时实施中保持这种积极表现，或者是否需要砍掉。
- 比较多个策略/投资组合，以便我们可以降低与分配有限数量的交易资金最大利用机会成本。

我们感兴趣的绩效定量分析的具体项目如下：

- 回报 - 交易策略中最明显的方面涉及自成立以来的收益率，无论是在回测还是实时交易环境中，这里的两个主要业绩指标是总回报率和

复合年增长率（CAGR）。

- 下跌 - 下跌是一个负面表现的时期，从先前的高水位线开始下跌，其本身被定义为策略或投资组合权益曲线上的前一个最高峰。我们将在下面更具体地定义它，但你现在可以把它想象成你的收益图表上的（有点痛苦！）下坡。

- 风险 - 风险包括许多领域，我们将在下一章中花费大量时间来讨论它们，通常它指的是资本损失的风险，例如亏损和回报的波动率。波动率通常被计算为收益的年化标准差。

- 风险/回报率 - 机构投资者主要对风险回报比感兴趣。由于较高的波动率通常会导致更高的回报而有更大的隐含风险，因此他们总是关注每单位回报所承担的风险。因此，已经发明了一系列绩效指标来量化战略绩效的这一方面，即夏普比率，信息比率和 CALMAR 比率等。其中的夏普通常是机构投资者在讨论策略绩效时要讨论的第一个指标。

- 交易分析 - 以前的绩效衡量指标均适用于战略和投资组合。查看交易明细的表现也是有益的，并且存在许多衡量其表现的措施。特别是，我们将计算赢/输交易的数量（译者按：胜率），每笔交易的平均利润和赢/输比率（译者按：平均盈亏比）等。

交易是算法策略中最细节的方面，因此我们将首先讨论交易分析。

12.1 交易分析

分析任何策略的第一步是考虑实际交易的表现。这些指标在策略之间可能会有很大差异，一个典型的例子是与均值回复策略相比，趋势跟

踪策略的指标的差异。

趋势跟踪策略通常包括许多亏损交易，每个交易都有可能很小的损失。当趋势已经成立并且这些交易可以显示较大数量的亏损交易，较少数量的有利可图的交易发生（译者安：但平均盈亏比大）。配对交易均值回归策略正好相反。它们通常由许多小的盈利交易组成。但是，如果并不意味着差价会以预期的方式回归，那么该策略的多/空交易可能导致大量损失。甚至会把大量的小额收益亏掉。

必须了解策略的交易概况性质和您自己的心理承受情况，因为两者需要保持一致。否则你会发现你可能无法在艰难的亏损期间坚持到底。我们现在开始研究我们感兴趣的策略统计数据作为执行策略的交易前提。

12.1.1 统计指标

在考虑我们的交易时，我们对以下一组统计指标感兴趣。这里“期间”是指包含 OHLCV 数据的交易所涵盖的时间段。对于长期策略，通常使用每日 K 线。对于更高频率的策略，我们可能对每小时或细小的 K 线图感兴趣。

- 总利润/损失（PnL） - 总 PnL 直接表明特定交易是否有利可图。
- 平均周期 PnL - 平均值。期间 PnL 表明平均值是否会产生利润或损失。
- 最大期间利润 - 迄今为止该交易所产生的最大期限利润。
- 最大期间损失 - 迄今为止该交易所造成的最大期限损失。请注意，这没有说明未来最大期间损失！未来的损失可能远大于此。

- 平均期间利润 - 所有盈利期间时间的平均值。
- 平均期间损失 - 所有无利可图期间的时间平均值。
- 获胜期 - 所有获胜期的计数。
- 亏损期 - 所有亏损期的计数。
- 胜率 - 所有获胜期间与亏损期间的百分比。趋势跟踪和均值回复类型策略会有显著差异。

值得庆幸的是，从我们的产品组合输出中生成此信息非常简单，完全消除了手动记录保存的需要。但是，这不会让我们分析数据之时发现问题！

所以交易必须至少每月评估一次或两次。这样做是一个有用的早期预警检测，有助于确定策略性能何时开始下跌。它通常比单纯考虑累积 PnL 要好得多。

12.2 策略和投资组合分析

交易水平分析在长期策略中非常有用，特别是对于采用复杂交易的策略，例如涉及金融衍生工具的策略。对于更高频率的策略，我们对任何单个交易都不太感兴趣，而是考虑整体策略的绩效指标。显然，对于策略战略，我们同样对整体策略绩效感兴趣。我们主要关注以下三个关键领域：

- 收益分析 - 策略的回报包含了盈利概念。在机构环境中，它们通常被作为资金变动，因此可以真实地了解投入的资金数额。回报可能很难计算，尤其是现金流入/流出。
- 风险/报酬分析 - 通常，外部投资者在策略中首先考虑的是其样本

夏普比率（我们在下面描述）。这是一个行业标准指标，旨在表示每单位风险的回报率。

•回撤分析 - 在市场环境中，回撤可能是三个方面中最重要的。策略，投资组合或基金的表现和投资范围构成风险管理的关键组成部分。我们将在下面定义下沉。尽管我已经强调了机构的绩效标准。作为个人交易者，这些仍然是非常重要的指标，并且适当的风险管理（见下一章）将构成策略规避风险的基础。

12.2.1 收益分析

在机构和个人投资者中讨论策略绩效时，最常见引用的数字通常是总回报，年回报和月回报。对冲基金公布月度回报业绩报告是极为常见的。此外，每个人都想知道策略的“回报”是什么。总回报计算相对简单，在没有外部投资者或现金流入/流出的零售环境中。可以按百分比计算，计算如下：

$$r_t = (P_f - P_i) / P_i \times 100 \quad (12.1)$$

（译者按：原文是这样，100 应为 100%）

其中 r_t 是总回报， P_f 是最终的投资组合美元价值， P_i 是最初的投资组合价值。我们主要对净总回报感兴趣，即扣除所有交易/业务成本后的投资组合/基金的价值。

请注意，此公式仅适用于长期未杠杆化的投资组合。如果我们希望增加卖空或杠杆，我们需要修改我们计算回报的方式，因为我们在技术上交易的是比这里使用的杠杆的投资组合。这被称为保证金组合。

比如，考虑一个交易策略已经超过 1000 美元的一种资产，然后卖空 1,000 美元的另一种资产的情况。这是一个美元中性的投资组合，总的名义交易是 2,000 美元。如果这个策略产生 200 美元收益，那么这个名义上的总收益就是 10%。当您考虑借贷成本以及借贷的利率来为保证金提供资金时，它会变得更加复杂。考虑这些成本导致净总回报变化，这通常被称为“总回报”。

资金曲线

资金曲线通常是对冲基金业绩报告中最受关注的内容之一 - 假设基金表现良好！这是基金组合价值随时间变化的图。实质上，它用于显示自基金开始以来账户的增长情况。同样，在个人账户中，它用于显示账户资产随时间的增长。有关典型的权益曲线图，请参见图 12.2.1：

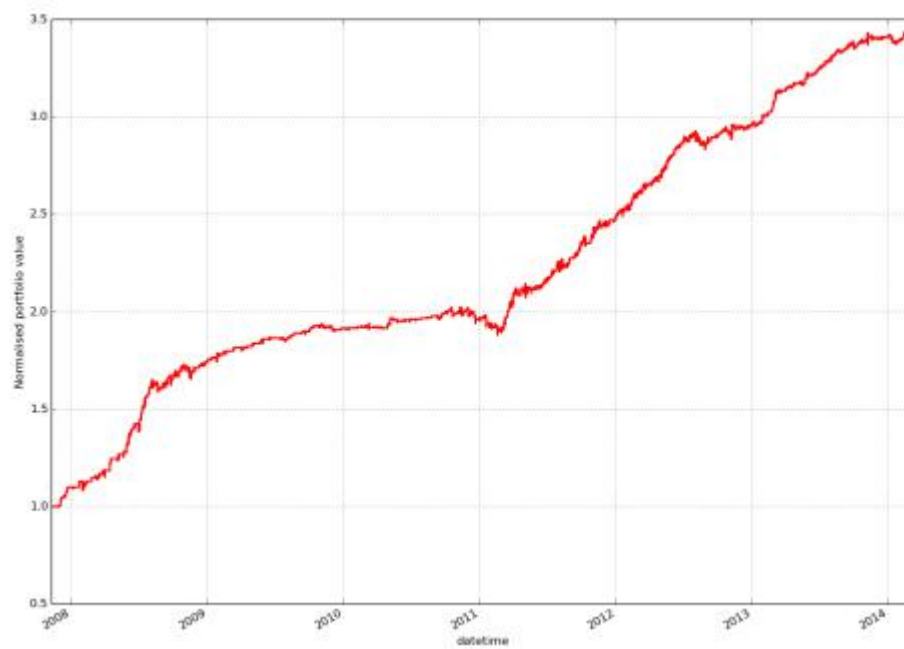


图 12.1：典型的日内策略权益曲线

这样有什么好处？从本质上讲，它给出了策略过去波动性的“特征”，以及策略是否遭受长时间高位震荡甚至回撤的视觉指示。它基本上提供了关于如何在策略交易期结束时计算总回报数的情况。

在股票曲线中，我们正在寻求确定不寻常的历史事件如何影响策略净值。例如，一个常见的问题是，2008 年左右的策略是否存在过度波动。另一个问题可能涉及其回报和策略的一致性。

必须非常谨慎地解读收益曲线，因为在市场上它们通常被显示为“向上倾斜的线”。通过解读这样的曲线可以获得有趣的见解，可以在强烈波动或长时间下跌的时期分析，否则在考虑整个时间段时回撤或波动截断曲线可能看起来不那么严重。因此，需要在相关的背景下考虑和分析权益曲线，特别是风险/回报分析和回撤分析。

12.2.2 风险/回报分析

正如我们在上面提到的那样，风险回报分析的概念在机构投资中极为重要。这并不意味着作为个人投资者，我们可以忽略这一概念。您应该特别关注策略的风险/回报指标，因为它们会对您的资金，杠杆率和整体复合增长率产生重大影响。

这些概念将在下一节“风险与资金管理”中进行扩展。目前我们将讨论指标，特别是夏普比率，它在量化金融中作为比较衡量标准无处不在。由于它在程序化量化交易中受到如此高的重视，我们将进行仔细剖析。

夏普比率

考虑我们提出两种具有相同回报的策略的情况。我们如何知道

哪一个包含更多风险？此外，“风险更大”甚至意味着什么呢？在金融方面，我们经常关注回报的波动率和回撤期。因此，如果这些策略中的一个具有明显更高的收益波动率，我们可能会发现它的吸引力较小，尽管它的历史回报可能相似，如果不相同的话。策略比较和风险评估的这些问题促使了夏普比率的使用。

William Forsyth Sharpe 是获得诺贝尔奖的经济学家，他帮助创建了资本资产定价模型（CAPM）并于 1966 年开发了夏普比率（后来于 1994 年更新）。夏普比率 S 由以下关系定义：

$$S = \frac{E(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)}} \quad (12.2)$$

其中 R_a 是资产或策略的期间回报， R_b 是合适的基准回报，如无风险利率。

该比率将资产或策略的超额收益的平均值与超额收益的标准差进行比较。因此，假设平均回报相同，回报的较低波动将导致夏普比率更高。

执行交易策略的人经常引用的“夏普比率”是年化的夏普，其计算取决于衡量回报的交易时间。假设一年有 N 个交易期，则年化夏普的计算方法如下：

$$S_A = \sqrt{N} \frac{E(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)}}$$

请注意，夏普比率本身必须基于该特定时间段的夏普来计算。对于基于正常交易周期的策略， $N = 252$ （一年有 252 个交易日，而不是

365)， R_a ， R_b 必须是每日回报。类似地，小时 $N = 252 \times 6.5 = 1638$ ，而不是 $N = 252 \times 24 = 6048$ ，因为在一个交易日只有 6.5 小时（至少对于美国股市来说！）。

上述夏普比率的公式暗示了基准的使用。基准被用作“尺度”或“阻碍”，特定策略必须克服它才能值得考虑。例如，一个使用美国大盘股票的简单多头策略应该希望平均击败标准普尔 500 指数，或者将其在同等收益下，波动性降低，不仅仅是通过在和投资指数获得同等收益。

基准的选择有时可能不清楚。例如，是否应将交易所交易基金（ETF）用作个别股票或标准普尔 500 指数本身的业绩基准？那为什么不用罗素 3000？同样，对冲基金策略应该将自己与市场指数或其他对冲基金指数进行对比吗？

还存在“无风险利率”的问题。是否应该使用国内政府债券？还是一篮子国际债券？短期或长期票据？很明显，有很多方法可以选择基准。夏普比率通常采用无风险利率，而且对于美国股票策略而言，这通常基于 10 年期政府国库券。

在一个特定的例子中，对于市场中性策略，是否使用无风险利率或零作为基准存在特定的复杂性。不应该利用市场指数本身，因为该策略在设计上与市场无关。对市场中性投资组合的正确选择不是减少无风险利率。由于您从持有保证金获得利息 R_f ，实际的回报计算为： $(R_a + R_f) - R_f = R_a$ 。因此，美元中性策略的无风险利率实际上没有减去。

尽管量化金融中夏普比率普遍存在，但它确实受到一些限制。夏普比率是向后看的。它只考虑历史收益分布和波动性，而不是未来发生的净值。根据夏普比率进行判断时，隐含的假设是过去与未来相似。显然并非总是如此，特别是在市场变化下。

夏普比率计算假设正在使用的回报是正态分布的（即高斯）。不幸的是，市场经常遭遇高于正态分布的峰度。从本质上讲，回报的分布具有“肥尾”，因此极端事件更可能发生，而高斯分布将导致我们发生错误。因此，夏普比率在表现尾部风险方面很差。

在极易发生此类风险的策略中可以清楚地看到这一点。例如，出售看涨期权又名“蒸汽压路机下便士”。随着时间的推移，通过出售看涨期权产生稳定的权利金，导致回报的低波动性，超过基准的强劲超额回报。在这种情况下，策略将具有基于历史数据的高夏普比率。但是，它没有考虑到此类期权可能会突然暴跌，导致权益曲线显着下降甚至消失。因此，与算法交易策略性能的任何度量一样，夏普比率不能单独使用。

到目前为止，一个尚未解决的明显问题是“策略的夏普比率是多少？”。这实际上是一个非常难以回答的问题，因为每个投资者都有不同的风险。一般的经验法则是，不应经常考虑年化夏普比率 $S < 1$ 的量化策略。但是，也有例外情况，特别是在趋势跟踪期货领域。

量化基金倾向于忽略任何具有夏普比率 $S < 2$ 的策略，我熟悉的著名的量化对冲基金甚至在研究中不会考虑夏普比率 $S < 3$ 的策略。作为个人算法交易者，如果您可以实现实盘交易（即实时交易！）夏

普比率 $S > 2$ ，那么您的表现非常好。

夏普比率通常会随着交易频率而增加。一些高频策略将具有较高的个位数（有时是低两位数）夏普比率，因为它们几乎每天都可以盈利，当然也可以每月盈利。这些策略很少遭受灾难性风险（在巨大损失的意义上），从而最大限度地降低其回报的波动性，从而导致夏普比率如此之高。请注意，尽管像这样的高频策略可能会突然停止运作，这是风险的另一个方面，并未在夏普比率中得到充分反映。

现在让我们考虑一些实际的夏普示例。我们将简单地开始，考虑长期购买并持有个人股票，然后考虑市场中性策略。这两个例子都是用 Pandas 进行的。

第一项任务是实际获取数据并将其放入 Pandas DataFrame 对象中。在前面关于使用 Python 和 MySQL 实现证券数据库的章节中，我们创建了一个实现此目的的系统。或者，我们可以利用这个更简单的代码直接获取 Google 财经数据，并将其直接放入 DataFrame 中。在这个脚本的底部，我创建了一个函数来计算基于时间段回报的年化夏普比率：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# sharpe.py

from __future__ import print_function
```

```

import datetime
import numpy as np
import pandas as pd
import pandas.io.data as web

def annualised_sharpe(returns, N=252):
    """
    Calculate the annualised Sharpe ratio of a returns stream
    based on a number of trading periods, N. N defaults to 252,
    which then assumes a stream of daily returns.

    The function assumes that the returns are the excess of
    those compared to a benchmark.
    """
    return np.sqrt(N) * returns.mean() / returns.std()

```

现在我们有能力从 Google 财经获取数据并直接计算年化夏普比率，我们可以测试两个股票的买入和持有策略。我们将在 2000 年 1 月 1 日至 2013 年 1 月 1 日期间使用 Google（GOOG）。

我们可以创建一个额外的辅助函数，使我们能够在相同（编码）时段内快速查看跨多个股票的买入并持有的 Sharpe:

```

def equity_sharpe(ticker):
    """
    Calculates the annualised Sharpe ratio based on the daily
    returns of an equity ticker symbol listed in Google Finance.

    The dates have been hardcoded here for brevity.
    """
    start = datetime.datetime(2000,1,1)
    end = datetime.datetime(2013,1,1)

    # Obtain the equities daily historic data for the desired time period
    # and add to a pandas DataFrame
    pdf = web.DataReader(ticker, 'google', start, end)

    # Use the percentage change method to easily calculate daily returns
    pdf['daily_ret'] = pdf['Close'].pct_change()

    # Assume an average annual risk-free rate over the period of 5%
    pdf['excess_daily_ret'] = pdf['daily_ret'] - 0.05/252

    # Return the annualised Sharpe ratio based on the excess daily returns
    return annualised_sharpe(pdf['excess_daily_ret'])

```

对谷歌而言，买入和持有的夏普比率为 0.703:

```
>>> equity_sharpe('GOOG')
0.70265563285799615
```

现在我们可以尝试相同的计算，以实现市场中性的策略。该策略的目标是将特定股权的表现与整体市场完全隔离开来。实现这一目标的最简单方法是跟踪此类市场的交易所交易基金（ETF）的等额（以美元计）。美国大盘股票市场最好的选择是标准普尔 500 指数，该指数由 SPDR ETF 追踪，其股票代码为 SPY。

为计算此类策略的年化夏普比率，我们将获得 SPY 的历史价格，并以与先前股票类似的方式计算回报百分比，但我们不会使用无风险基准。我们将计算每日净收益，它需要减去长期和短期收益之间的差异然后除以 2，因为我们现在有两倍的交易资本。这是执行此操作的 Python / pandas 代码：

```
def market_neutral_sharpe(ticker, benchmark):
    """
    Calculates the annualised Sharpe ratio of a market
    neutral long/short strategy involving the long of 'ticker'
    with a corresponding short of the 'benchmark'.
    """
    start = datetime.datetime(2000, 1, 1)
    end = datetime.datetime(2013, 1, 1)

    # Get historic data for both a symbol/ticker and a benchmark ticker
    # The dates have been hardcoded, but you can modify them as you see fit!
    tick = web.DataReader(ticker, 'google', start, end)
    bench = web.DataReader(benchmark, 'google', start, end)

    # Calculate the percentage returns on each of the time series
    tick['daily_ret'] = tick['Close'].pct_change()
    bench['daily_ret'] = bench['Close'].pct_change()

    # Create a new DataFrame to store the strategy information
    # The net returns are (long - short)/2, since there is twice
    # the trading capital for this strategy
    strat = pd.DataFrame(index=tick.index)
    strat['net_ret'] = (tick['daily_ret'] - bench['daily_ret'])/2.0

    # Return the annualised Sharpe ratio for this strategy
    return annualised_sharpe(strat['net_ret'])
```

对谷歌而言，多头/空头市场中性策略的夏普比率为 0.832：

```
>>> market_neutral_sharpe('GOOG', 'SPY')
0.83197496084314604
```

我们现在将简要介绍其他风险/回报率指标。

索提诺比率

索提诺比率的动机是夏普比率在其分母中捕获上涨和下跌的波动率。然而，当我们出现大幅上行波动时，投资者（和对冲基金经理）通常不会太烦恼！从风险管理的角度来看，实际上感兴趣的是下跌波动和回撤期。

因此，索提诺比率定义为平均超额收益除以平均下跌波动：

$$\text{Sortino} = \frac{\mathbb{E}(R_a - R_b)}{\sqrt{\text{Var}(R_a - R_b)_d}} \quad (12.3)$$

索提诺比率有时在机构评测中被引用，但肯定不如夏普比率那么普遍。

CALMAR 比率

人们还可以争辩说，投资者/交易者只关心最大跌幅，而不是平均跌幅。这激发了 CALMAR（CALifornia 管理账户报告）比率，也称为 Calmar 比率，意味着超额回报到最大亏损：

$$\text{CALMAR} = \frac{\mathbb{E}(R_a - R_b)}{\text{max. drawdown}} \quad (12.4)$$

CALMAR 并没有像夏普比率那样应用广泛。

12.2.3 回撤分析

在我看来，回撤的概念是算法交易系统性能的最重要方面。简而言

之，如果您的账户资产被清零，那么其他任何绩效指标都不重要！回撤分析涉及衡量先前高水位的账户净值下降。高水位线定义为权益曲线上达到的最后一个账户净值峰值。

在这种环境下，回撤的概念尤为重要，因为大多数对冲基金只有在账户资产不断创造新的高水位时才会获得报酬。也就是说，基金经理获得超额业绩报酬，基金仍处于“水下”，即账户资产处于回撤期。大多数投资者担心基金会下跌 10%，并且一旦跌幅超过 30%，他们可能会赎回他们的投资。在个人交易中，情况非常不同。个人可能会忍受更深的回撤，希望获得更高的回报。

最大亏损和持续时间

两个关键的回撤指标是最大亏损和回撤持续时间。第一个描述了从前一个峰值到账户资产当前或之前的低谷的最大下跌百分比。在试图销售基金时，它经常在被引用。个人交易者也应该非常注意这个数字。第二个描述了回撤的实际持续时间。此数字通常以天为单位使用，但更高频率的策略可能会使用更精细的时间段。

在回测中，这些衡量标准提供了一些策略在未来可能会如何发挥作用的一些想法。经过计算的回测后，整体账户净值曲线看起来很有吸引力。然而，向上的收益曲线可以很容易地掩盖以前的回撤期实际可能经历的亏损程度。

当策略开始下跌至账户资产资产的 10% 以下，甚至低于 20% 时，需要很大的意志力来继续策略，尽管策略可能在历史上，至少在后测中，已经过了类似的时期。这是算法交易和回测交易的一致问题。它自然

地促使需要设置先前的最大回撤和特定规则，例如在回撤违反这些规定的情况下将执行的账户范围的“止损”。

回撤曲线

虽然了解最大亏损和回撤持续时间非常重要，但在交易持续时间内查看策略回撤的时间序列图显然更有启发性。图 12.2.3 非常清楚地表明这种策略的持续回撤表现

从 2010 年第三季度开始到 2011 年第二季度的回撤期，达到最大下降 14.8%。虽然该策略本身在长期内继续显着盈利，但这个很难持久。此外，这是迄今为止发生的最大历史回撤。该策略未来可能会进一步回撤。因此，有必要在其生成背景的情况下，即通过历史数据而非未来数据，考虑回撤曲线，与其他历史观察期进行比较。

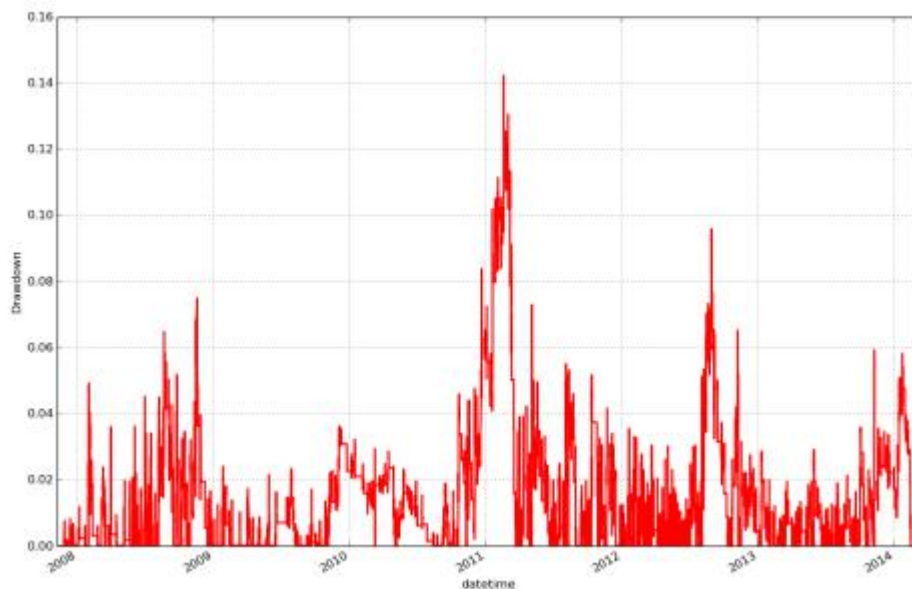


图 12.2：典型策略回撤曲线

在下一章中，我们将考虑量化风险管理的概念，并阐述可以帮助我们最大限度地减少亏损和最大化回报的技术，同时保持合理的风险水平。

第 13 章

风险与资金管理

本章涉及适用于量化交易策略的风险管理。通常有两种形式，首先是识别和减少可能影响算法交易策略的性能或操作的内部和外部因素，其次，如何最佳地管理策略组合以最大化收益率并最小化账户回撤。

在第一部分中，我们将考虑可能影响算法交易（个人或机构）的长期绩效的不同风险来源（内在和外在）。

在第二部分中，我们将研究能够保护我们的投资组合免于爆仓的资金管理技术，并尝试最大化收益的长期增长率。

在最后一节中，我们考虑可以在个人交易中轻松应用的机构级风险管理技术，以帮助保护交易资金。

13.1 风险来源

众多的风险来源可能对算法交易策略的正确运作产生影响。在这种情况下，“风险”通常被定义为账户损失的可能性。但是，我将在更广义的背景下对其进行定义，以表示任何提供一定程度的不确定性并可能影响我们的策略或投资组合的表现的因素。

我们将考虑的广泛风险领域包括策略风险，投资组合风险，市场风险，交易对手风险和操作风险。

13.1.1 策略风险

策略风险或模型风险包括基于统计模型设计和实施交易策略所产生的风险类别。它包括我们在回测章节中讨论过的所有先前问题，例如

曲线拟合，幸存者偏差和前向偏差。它还包括与策略模型的统计分析直接相关的其他风险。

任何统计模型都基于假设。这些假设有时不是适当的或可能完全被忽略。这意味着基于这些假设的统计模型可能是不合适的，因此导致较差的预测能力。一般示例出现在线性回归的中。线性回归假设响应数据是同方差的（即响应在其误差中具有恒定的方差（译者按:2003 年诺贝尔经济学奖就是因为推翻这个假设而颁发的，在我们学习的传统模型，80 年代以前我们都是认为方差恒定））。如果不是这种情况，那么线性回归在参数估计中提供的精度较低。

许多量化策略利用历史价格数据的描述性统计。特别是，他们通常会使用历史回测，例如策略回报的均值，方差，偏斜和峰度。这些模型（包括下面概述的 Kelly Criterion）通常依赖于这些时刻在时间上假设是恒定。在市场体系变化下，这些时刻可能会发生巨大变化，从而导致模型退化。通常使用具有“滚动参数”的模型来缓解该问题。

13.1.2 投资组合风险

投资组合包含一个或多个策略。因此，它间接受到如上所述的策略风险的影响。此外，投资组合层面也存在特定风险。这些通常仅在机构或高净值客户考虑，其中投资组合跟踪是在稳定的交易策略上进行的。

当投资组合回归到一系列因子时，例如行业部门，资产类别或金融实物资产，可以确定特定因子是否“加载”投资组合中。例如，科技股

对股票投资组合的影响可能非常大，因此极易受到影响整个科技行业的任何问题的影响。因此，在投资组合层面，通常需要覆盖特定策略以解决超载因子风险。在机构中，这是一个更为重要的问题，在这种机构中，需要分配更多的资金，资金的保护优先于资金的长期增长率。

对于机构来说另一个主要是制度的问题（除非交易更多非流动资产）是对每日交易量的限制。对于个人交易者，在大盘股或商品期货市场执行策略，不用考虑到交易对市场本身的影响。然而，在流动性较低的金融工具中，由于潜在的市场影响以及因此无效的交易还是很多（通常不考虑市场影响），人们必须小心不要交易日交易量的过大。为避免这种情况，有必要计算平均每日交易占总交易量的比例（例如，使用回测期间的平均值）并保持该数字在限度内。

运行策略组合会带来策略相关性问题。相关性可以通过统计技术估算，例如 **Pearson Product Moment Correlation Coefficient**。然而，相关性本身并不是静态的，也会受到变化的影响，特别是在市场流动性限制下，通常被称为 **market-wide**。一般而言，策略应设计为避免因不同的资产类别或时间范围而相互关联。如果考虑投资组合方法，滚动相关性可以在长时间段内估算，这也应该是回溯测试的标准部分。

13.1.3 交易对手风险

交易对手风险通常被视为一种信用风险。对方不承担对其承担责任的金融资产的义务。有一部分定量融资与对应风险对冲工具的定价有关，但这对我们作为个人算法交易者并不是太相关。我们更关注交易所或经纪等供应商的违约风险。

虽然这似乎很学术性，但我个人可以向您保证，这些问题非常现实！我亲身经历了经纪人破产，意味着并非所有的交易资本都被退回。因此，我现在将这些风险纳入投资组合。 建议的减轻这个问题的方法是利用多个经纪人，尽管在保证金交易时，这可能会使交易变得有些棘手。

交易对手风险通常在机构更受关注，因此我不会赘述这里太多了！

13.1.4 操作风险

操作风险包括基金或交易运营设备在内的风险来源，包括业务/企业风险，IT 风险和外部监管或法律变更。 这些主题通常不会被深入讨论，我认为这些主题有点短视，因为它们有可能永久地完全停止交易操作（译者按：比如某些程序化接入被暂停）。

设备风险通常与信息技术系统和其他相关的交易设施相关。这还包括员工风险（例如欺诈，突然离职）。随着设施规模的扩大，“单点故障”（SPOF）的可能性也随之增加。这是交易设施中的一个关键组成部分，在故障情况下，可能导致整个运营的灾难从而停止交易。从 IT 意义上讲，这通常是架构的结果。在非 IT 意义上，这可能是设计糟糕的组织结构的结果。

这些问题仍然与个人交易者完全相关。通常，IT /交易设施最终可能会“不完整”并“被黑客攻击”。此外，糟糕的记录保存和其他行政问题可能导致巨大的潜在税收负担。这些类型的问题，即考虑来自市场和策略以外的来源的风险，通常会使成功的长期算法交易者与因灾

难性操作崩溃而减少个人之间产生差异。值得庆幸的是，“云”架构提供了系统冗余的能力和流程自动化可以带来良好的管理习惯。

影响对冲基金领域的另一个问题是报告和合规性。2008 年后的立法给资产管理公司带来了沉重的负担，这可能对其现金流和运营支出产生很大影响。对于想要合并这样一家公司来说，为了扩大战略或与外部资金合作，也要谨慎地考虑立法和监管环境，因为它有点像“moving target”。

13.2 资金管理

本节涉及交易中最基本的概念之一 - 自由裁量和算法 - 即资金管理。一个天真的投资者/交易者可能会认为唯一重要的投资目标是尽可能多地赚钱。然而，长期交易的现实更为复杂。由于市场参与者具有不同的风险偏好和约束，投资者可能拥有许多目标。

许多个人交易者认为，唯一的目标是持续增加账户净值，很少或根本不考虑实现这种增长的策略的“风险”。散户投资者衡量账户下降，并根据他们的风险偏好，可能能够应对账户资产的大幅下降（比如说 50%）。他们能够处理如此规模的回撤的原因是他们认识到通过杠杆作用，这种行为对于投资组合的长期增长率可能是最佳的。

机构投资者可能会以不同的方式考虑风险。机构投资者通常要求最大回撤（例如 20%），并对行业分配和平均每日交易量限制进行了重要考虑。它们将是对战略资本配置的“优化问题”的额外限制。这些因素甚至可能比最大化投资组合的长期增长率更重要。

因此，我们处于这样一种情况：我们可以在通过杠杆最大化长期

增长率和通过试图限制回测持续时间和范围来最小化我们的“风险”之间取得平衡。 帮助我们实现这一目标的主要工具称为 **Kelly Criterion**。

13.2.1 Kelly Criterion

在本节中，**Kelly Criterion** 将成为我们控制一组构成多策略投资组合的算法交易策略的杠杆和分配的工具。

我们将杠杆定义为投资组合规模与该投资组合中实际账户净资产的比率。为了明确这一点，我们可以使用购买抵押房屋的类比。您的首付款构成您的账户净值，而首付款加抵押贷款价值相当于投资组合的规模。因此，在 200,000 美元的房屋（抵押贷款为 150,000 美元）上支付 50,000 美元构成了 $(150000 + 50000) / 50000 = 4$ 的杠杆。因此，在这种情况下，您将获得 4 倍的房屋杠杆。保证金账户组合的行为类似。有一个“现金”部分，然后通过保证金借入更多股票，以提供杠杆。

在我们具体说明 **Kelly Criterion** 之前，我想概述其推导的假设，这些假设具有不同程度的准确性：

- 假设每种算法交易策略都具有正态分布的回报（即高斯分布）。此外，每种策略都有自己固定的均值和收益的标准差。该公式假设这些平均值和标准值不会改变，即它们在过去和未来相同。大多数策略显然不是这种情况，因此请注意这一假设。

- 此处考虑的回报是超额回报，这意味着它们是支付所有融资成本后的净额，例如保证金和交易成本支付的利息。如果策略是在机

构产品中进行的，这也意味着回报是支付管理和绩效费用后的净额。

- 所有交易利润均进行再投资，不进行收益提取。这显然不适用于上述管理费用被取消且投资者经常提取收益的情形。

- 所有策略在统计上都是独立的（策略之间没有相关性）。

现在我们就来实际的 **Kelly Criterion**！让我们假设我们有一套 **N** 算法交易策略，我们希望确定如何应用每个策略的最佳杠杆，以最大化增长率（但最小化下跌）以及如何在每个策略之间分配资金。如果我们将每个策略之间的分配 **i** 表示为长度为 **N** 的向量 **f**，则为 **s.t. f = (f1, ..., fN)**，则每个策略 **fi** 的最佳分配的 **Kelly** 准则由下式给出：

$$f_i = \mu_i / \sigma_i^2 \quad (13.1)$$

其中 μ_i 是平均超额收益， σ_i 是策略 **i** 的超额收益的标准差。该公式基本上描述了应该应用于每种策略的最佳杠杆。

虽然 **Kelly Criterion** 的 **fi** 为我们提供了最佳杠杆和策略分配，但我们仍需要实际计算投资组合的预期长期复合增长率，我们用 **g** 表示。其公式如下：

$$g = r + S^2 / 2 \quad (13.2)$$

其中 **r** 是无风险利率，即您可以向经纪人借款的利率，**S** 是该策略的年化夏普比率。后者的计算方法是年化平均超额收益除以超额收益的年化标准差。有关夏普比率的详细信息，请参见前一章“绩效评估”。

一个实际的例子

让我们考虑单一策略案例中的一个例子 ($i = 1$)。假设我们长期存在股票 XYZ，其平均年回报率为 $m = 10.7\%$ ，年度标准差为 $\sigma = 12.4\%$ 。另外假设我们能够以无风险利率借入 $r = 3.0\%$ 。这意味着平均超额收益为 $\mu = m - r = 10.7 - 3.0 = 7.7\%$ 。这给出了夏普比率 $S = 0.077 / 0.124 = 0.62$ 。

有了这个，我们可以通过 $f = \mu / \sigma^2 = 0.077 / 0.124^2 = 5.01$ 来计算最佳凯利杠杆。因此，凯利杠杆表示，对于 10 万美元的投资组合，我们应该再借 401,000 美元，使其总投资组合价值为 501,000 美元。（在实践中，我们的经纪人不太可能让我们以如此大的杠杆进行交易，因此需要调整 Kelly Criterion。）

然后我们可以使用夏普比率 S 和利率 r 来计算 g ，即预期的长期复合增长率。 $g = r + S^2 / 2 = 0.03 + 0.62^2 / 2 = 0.22$ ，即 22%。因此，我们应该期望每年从这一策略中获得 22% 的回报。

Kelly Criterion 实践

重要的是要意识到凯利标准需要不断重新平衡资金分配才能保持有效。显然，这在实际交易进行实时调整是不可能的，因此必须进行近似。这里的标准“经验法则”是每天更新一次凯利分配。此外，应使用平均值和标准偏差以及回顾窗口定期重新计算 Kelly Criterion 本身。同样，对于每天大约交易一次的策略，应设置为每日计算回溯 3-6 个月的数值。

以下是在 Kelly Criterion 下重新平衡投资组合的示例，这有些导致一些违反直觉的行为。我们假设我们有上述策略。我们使用 Kelly

Criterion 借入现金，将我们的投资组合规模调整为 501,000 美元。假设我们在第二天获得 5% 的回报，这将我们的账户规模提升至 526,050 美元。凯利标准告诉我们，我们应该借更多资金来保持相同的杠杆率 5.01。特别是我们的账户净值为 126,050 美元，投资组合为 526,050，这意味着当前的杠杆率为 4.17。为了将其增加到 5.01，我们需要额外借入 105,460 美元，以便将我们的账户规模增加到 631,510.5 美元（这是 5.01×126050 ）。

现在考虑第二天我们的投资组合损失了 10%。这意味着总投资组合规模现在为 568,359.45 美元（ 631510.5×0.9 ）。我们的总账户净值现在为 62,898.95 美元（ $126050 - 631510.45 \times 0.1$ ）。这意味着我们目前的杠杆因子是 $568359.45 / 62898.95 = 9.03$ 。因此，我们需要通过出售 253,235.71 美元的股票来减少我们的账户，以便将我们的总投资组合价值减少到 315,123.73 美元，这样我们再次获得 5.01 的杠杆率（ $315123.73 / 62898.95 = 5.01$ ）。

因此，我们已经买入了盈利并卖出亏损。这种卖亏的过程在情感上可能非常困难，但凯利的假设已经得到满足，那么在数学上这是“正确”的事情。这是为了最大化长期复合增长率而采取的方法。

您可能已经注意到，在每天之间重新分配资金非常严峻。这是该例子的人为性质和所采用的广泛杠杆的结果。在高频算法交易中，一天 10% 的损失并不是特别常见，但它确实有助于显示杠杆在严峻条件下的广泛性。

由于均值和标准差的估计总是存在不确定性，实际上许多交易者

倾向于使用更保守的杠杆制度，例如凯利准则除以 2，被亲切地称为“半凯利”。 **Kelly Criterion** 应该被视为使用杠杆的上限，而不是直接使用 **Kelly Criterion**。 如果不注意这个建议，那么由于策略的非高斯性质，使用直接凯利值会导致破产（即账户净值消失为零）。

你应该使用 **Kelly Criterion** 吗？

每个算法交易者都是不同的，风险偏好也是如此。 当选择采用杠杆策略（其中 **Kelly Criterion** 就是一个例子）时，您应该考虑风险约束。 在个人交易中，您可以设置自己的最大亏损限制，从而提高您的杠杆率。 在机构中，您需要从非常多的角度考虑风险，杠杆因素将是更大框架的一个组成部分，通常在许多其他限制条件下。

13.3 风险管理

13.3.1 风险价值

估算算法交易策略或策略组合的损失风险对于长期资金增长至关重要。现在机构已经开发了许多风险管理技术。 特别是一种被称为风险价值或在险价值（**VaR**）的技术将成为本节的主题。

我们将 **VaR** 的概念应用于单一策略或一组策略，以帮助我们量化交易组合中的风险。 **VaR** 的定义如下：**VaR** 在给定的置信度下给定时间段内投资组合损失大小的估计。

在这种情况下，“投资组合”可以指单一策略，一组策略，交易者账簿，对冲基金或整个投资银行。“给定的置信度”将是例如 95% 或 99% 的值。 将选择“给定时间段”以反映如果要清算投资组合将导致最大市场影响的那个时间段。

例如，在一天的时间段内，在 95% 置信水平下等于 500,000 美元的 VaR 将简单地表明在第二天有 95% 的概率损失不超过 500,000 美元。在公式表示为：

$$P(L \leq -5.0 \times 10^5) = 0.05 \quad (13.3)$$

或者，更一般地说，对于损失 L 超过具有置信水平 c 的值 VaR ，我们具有：

$$P(L \leq -VaR) = 1 - c \quad (13.4)$$

VaR 的“标准”计算做出以下假设：

- 标准市场条件 - VaR 不应考虑极端事件或“尾部风险”，而是应该在正常的“日常”操作下损失的预期。
- 波动性和相关性 - 风险价值要求考虑的资产的波动性以及它们各自的相关性。这两个数量很难估算，并且会不断变化。
- 收益的正常性 - VaR 以其标准形式假设资产或投资组合的收益是正态分布的，但对大多数资产来说这是不切实际的。

13.4 优点和缺点

VaR 在金融行业普遍存在，因此您应该熟悉它的优点和缺点。

VaR 的一些优点如下：

- VaR 可以非常直接地计算个人资金，算法策略，量化投资组合，对冲基金甚至投资银行的风险。
- 可以针对具有不同时间范围的多个交易策略计算与 VaR 相关联的时间段。

- 不同的风险价值可以与不同形式的风险相关联，例如按资产类别或工具类型细分。例如，这可以很容易地发现大多数投资组合风险可能聚集的位置。

- 策略可以设定约束，整个投资组合也可以基于其个人 VaR 进行约束。

- VaR 很容易被（可能的）不专业外部投资者和基金经理解释基金经理。但是，VaR 并非没有缺点：

- VaR 没有讨论超出 VaR 值的预期损失的大小，即它会告诉我们，我们可能会看到损失超过一个值，但不能发现不会超过它的值。

- 它没有考虑极端事件，只考虑了典型的市场条件。

- 由于它使用历史数据，因此不会考虑可能改变资产波动性和相关性的未来市场变化。 不应单独使用 VaR。 它应该始终与一套风险管理技术一起使用，例如最佳投资组合分配和谨慎使用杠杆等。

计算方法

到目前为止，我们还没有讨论 VaR 的实际计算，无论是在一般情况下还是在具体的交易示例中。 我们感兴趣的技术有三种。 第一种是方差 - 协方差方法（使用常态假设），第二种是蒙特卡罗方法（基于潜在的非正态分布），第三种称为自举法，它利用历史回报信息 计算当前的资产。

在本节中，我们将集中讨论方差 - 协方差方法。

方差 - 协方差法

考虑一个 P 美元的投资组合，其置信水平为 c 。 我们正在考虑

每日回报，资产（或策略）历史标准差 σ 和平均值 μ 。然后，在单一资产（或策略）的方差 - 协方差方法下的每日 VaR 计算如下：

$$P - (P(\alpha(1 - c) + 1)) \quad (13.5)$$

其中 α 是平均值 μ 和标准偏差 σ 正态分布的累积分布函数的倒数。

我们可以使用 SciPy 和 pandas 库来计算这些值。如果我们设置 $P = 106$ 和 $c = 0.99$ ，我们可以使用 SciPy ppf 方法生成逆累积分布函数的值到正态分布， μ 和 σ 从一些实际金融数据中获得，在这种情况下是历史日收益 CitiGroup（我们可以在这里轻松替换算法策略的回报）：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# var.py

from __future__ import print_function

import datetime

import numpy as np
import pandas.io.data as web
from scipy.stats import norm

def var_cov_var(P, c, mu, sigma):
    """
    Variance-Covariance calculation of daily Value-at-Risk
    using confidence level c, with mean of returns mu
    and standard deviation of returns sigma, on a portfolio
    of value P.
    """
    alpha = norm.ppf(1-c, mu, sigma)
    return P - P*(alpha + 1)
```

```

if __name__ == "__main__":
    start = datetime.datetime(2010, 1, 1)
    end = datetime.datetime(2014, 1, 1)

    citi = web.DataReader("C", 'yahoo', start, end)
    citi["rets"] = citi["Adj Close"].pct_change()

    P = 1e6    # 1,000,000 USD
    c = 0.99   # 99% confidence interval
    mu = np.mean(citi["rets"])
    sigma = np.std(citi["rets"])

    var = var_cov_var(P, c, mu, sigma)
    print("Value-at-Risk: $%.2f" % var)

```

VaR 的计算值由下式给出：

Value-at-Risk: \$56503.12

VaR 在财务管理的所有领域都是一种非常有用和普遍的技术，但它并非没有缺陷。著名的对冲基金经理大卫艾因霍恩（David Einhorn）将 VaR 描述为“安全气囊所有时间都工作的很好，除了出车祸的时候”实际上，您应该始终使用 VaR 作为风险管理的增强，而不是作为单个指标！

Part VI

自动交易

第 14 章

事件驱动的交易引擎实施

本章提供了一个用 **Python** 编写的完全独立的事件驱动的回测系统的实现。特别是本章的内容是为了扩展其他算法交易文本和论文中通常省略的细节。以下代码将允许您模拟股票，外汇和期货市场进行动量和均值回归域的高频（分钟到秒）策略算法交易。

然而，广泛的细节带来了复杂性。这里提供了回测系统

需要许多组件，每个组件本身都是综合性的。因此，第一步是概述事件驱动的软件是什么，然后描述回测系统的组件以及整个系统如何搭建。

14.1 事件驱动的软件

在我们深入研究这种回测系统的开发之前，我们需要了解事件驱动系统的概念。视频游戏为事件驱动的软件提供了一个自然的例子，并提供了一个简单的探索示例。视频游戏具有多个组件，这些组件在高帧率的实时设置中彼此交互。这是事件循环或游戏循环的“无限”循环中运行整个计算集合处理。

在游戏循环的每个 **tick**，调用函数以处理接收最新事件，该事件将由游戏内的一些相应的先前动作生成。根据事件的性质（可能包括按键或鼠标单击），将执行一些后续操作，这将终止循环或生成一些其他事件。然后该过程将持续。

这是一些伪代码示例：

```

while True: # Run the loop forever
    new_event = get_new_event() # Get the latest event

    # Based on the event type, perform an action
    if new_event.type == "LEFT_MOUSE_CLICK":
        open_menu()
    elif new_event.type == "ESCAPE_KEY_PRESS":
        quit_game()
    elif new_event.type == "UP_KEY_PRESS":
        move_player_north()
    # ... and many more events

    redraw_screen() # Update the screen to provide animation
    tick(50) # Wait 50 milliseconds

```

代码不断检查新事件，然后根据这些事件执行操作。特别是它允许实时响应处理，因为代码不断循环并检查事件。很明显，这正是我们为了进行高频交易模拟所必须的。

14.1.1 为什么是事件驱动的回测？

与向量化方法相比，事件驱动系统具有许多优势：

- 代码重用 - 按设计，事件驱动的回测系统可用于历史回测和实时交易，只需最少的组件切换。向量化回测系统不是这样，其中所有数据用以进行统计分析。
- 前向偏见 - 基于事件驱动的回测系统没有前向偏差，因为市场数据被当作事件来处理。但把市场数据“喂给”事件驱动的回测系统也是可能的，像订单管理和投资组合系统那样。
- 现实主义 - 基于事件驱动的回测系统允许很大的自由度，比如订单执行和交易成本定制。处理基本的市价单和限价单，开市单，收市单等都是很简单的，因为可以自定义交易所处理。

尽管事件驱动系统具有许多优点，但它们比简单的向量化系统具有两个主要缺点。首先，它们的实施和测试要复杂得多。有更多的“活

动部分”导致更多的 bug 为了缓解软件测试压力，应该引入测试驱动系统。

其次，与向量化系统相比，它们的执行速度较慢。在进行数学计算时，无法利用最佳的向量化运算。

14.2 组件对象

为了将事件驱动的方法应用于回测系统，必须定义将处理特定任务的组件（或对象）：

- **Event 事件** - 事件是事件驱动系统的基本类单元。它包含一个类型（例如“MARKET”，“SIGNAL”，“ORDER”或“FILL”），用于确定在事件循环中如何处理它。
- **Event Queue 事件队列** - 事件队列存储所有事件对象，这些事件对象由软件产生，事件队列是内存型的 Python 队列对象。
- **DataHandler - DataHandler** 是一个抽象基类（ABC），它提供了一个用于处理历史或实时市场数据的接口。这提供了显著的灵活性，因为策略和组合模块因此可以在两种方法之间重用。DataHandler 在系统的每次心跳时生成一个新的 MarketEvent（见下文）。
- **Strategy 策略** - 策略也是一个抽象基类，它用于获取市场数据并生成相应的 SignalEvent，最终由 Portfolio 对象使用。SignalEvent 包含一个股票代码，一个方向（LONG 或 SHORT）和一个时间戳。
- **投资组合** - 这个类负责策略当前和接下来的仓位相关的订单管理。它也会处理跨组合的风险管理，包括分散投资和仓位管理。在更复杂的实现中，可以将其委托给 RiskManagement 类。Portfolio 从 Queue

获取 `SignalEvents` 并生成添加到 `Queue` 的 `OrderEvents`。

- `ExecutionHandler` - `ExecutionHandler` 模拟与经纪商的连接。处理程序的工作是从队列中获取 `OrderEvent` 并通过模拟方法或直接的方法与经纪商实际连接来执行它们。一旦执行了订单，处理程序就会创建 `FillEvent`，它事件描述交易执行，包括费用，佣金和滑点（如果建模）。

- `Backtest` - 所有这些组件都包装在一个事件循环中，它将正确地处理各种事件，转发各种事件到适当的组件中去处理。

尽管组件数量很多，但这是交易引擎的基本模型。有很大的扩展空间，特别是在如何使用投资组合方面。此外，不同的交易成本模型也可能被抽象到他们自己的类层次结构中。

14.2.1 事件

要讨论的第一个组件是 `Event` 类层次结构。在此基础结构中，有四种类型的事件允许通过事件队列在上述组件之间进行通信。它们是 `MarketEvent`，`SignalEvent`，`OrderEvent` 和 `FillEvent`。

`Event`

这些层次结构中的父类称为 `Event`。它是一个基类，不提供任何功能或特定接口。由于在许多实现中，`Event` 对象可能会产生更大的复杂性，因此可以创建一个面向未来的类层级。

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# event.py

from __future__ import print_function

class Event(object):
    """
    Event is base class providing an interface for all subsequent
    (inherited) events, that will trigger further events in the
    trading infrastructure.
    """
    pass
```

MarketEvent

当回测系统的外部 `while` 循环开始新的“心跳”时触发 `MarketEvents`。当 `DataHandler` 对象收到当前正在跟踪的任何符号的市场数据的新更新时，就会发生这种情况。它用于触发策略对象生成新的交易信号。事件对象只包含一个标识，表明它是一个市场事件，没有其他结构。

```
# event.py

class MarketEvent(Event):
    """
    Handles the event of receiving a new market update with
    corresponding bars.
    """

    def __init__(self):
        """
        Initialises the MarketEvent.
        """
        self.type = 'MARKET'
```

SignalEvent

`Strategy` 对象利用市场数据创建新的 `SignalEvent`。`SignalEvent` 包含策略 ID，品种代码，生成时间的时间戳，方向（多或空）和“强度”指

标(这对于均值回归策略很有用)。Portfolio 对象使用这个 SignalEvent 作为辅助决定如何进行交易。

```
# event.py

class SignalEvent(Event):
    """
    Handles the event of sending a Signal from a Strategy object.
    This is received by a Portfolio object and acted upon.
    """

    def __init__(self, strategy_id, symbol, datetime, signal_type, strength):
        """
        Initialises the SignalEvent.

        Parameters:
        strategy_id - The unique identifier for the strategy that
            generated the signal.
        symbol - The ticker symbol, e.g. 'GOOG'.
        datetime - The timestamp at which the signal was generated.
        signal_type - 'LONG' or 'SHORT'.
        strength - An adjustment factor "suggestion" used to scale
            quantity at the portfolio level. Useful for pairs strategies.
        """

        self.type = 'SIGNAL'
        self.strategy_id = strategy_id
        self.symbol = symbol
        self.datetime = datetime
        self.signal_type = signal_type
        self.strenath = strenath
```

OrderEvent

当 Portfolio 对象收到 SignalEvents 时，它会在风险和头寸规模方面控制进行评估。最终会生成 OrderEvent 事件，该事件会被 ExecutionHandler 对象处理。

OrderEvent 稍微比 SignalEvent 复杂，除了 SignalEvent 事件的一些字段以外，它还包含了数量字段。Portfolio 对象规定了这个数量字段。另外 OrderEvent 还包括一个 print_order() 方法，用于必要时输出订单信息。

```

# event.py

class OrderEvent(Event):
    """
    Handles the event of sending an Order to an execution system.
    The order contains a symbol (e.g. GOOG), a type (market or limit),
    quantity and a direction.
    """

    def __init__(self, symbol, order_type, quantity, direction):
        """
        Initialises the order type, setting whether it is
        a Market order ('MKT') or Limit order ('LMT'), has
        a quantity (integral) and its direction ('BUY' or
        'SELL').

        Parameters:
        symbol - The instrument to trade.
        order_type - 'MKT' or 'LMT' for Market or Limit.
        quantity - Non-negative integer for quantity.
        direction - 'BUY' or 'SELL' for long or short.
        """

        self.type = 'ORDER'
        self.symbol = symbol
        self.order_type = order_type
        self.quantity = quantity
        self.direction = direction

    def print_order(self):
        """
        Outputs the values within the Order.
        """
        print(
            "Order: Symbol=%s, Type=%s, Quantity=%s, Direction=%s" %
            (self.symbol, self.order_type, self.quantity, self.direction)
        )

```

FillEvent

当 ExecutionHandler 收到 OrderEvent 时，它必须处理订单。一旦订单被交易，它就会生成一个 FillEvent，它描述了买卖的成本，例如手续费或滑点。

FillEvent 是最复杂的事件。它包含了订单被执行时的时间戳，交易品种和交易所，交易数量，真正的交易价格以及交易佣金。

该佣金使根据证券委员会确定。对于美国 API 订单，此佣金为每笔

订单最低 1.30 美元，固定汇率为 0.013 美元或每股 0.08 美元，具体取决于交易规模是否大于 500 单位股票。

```
# event.py

class FillEvent(Event):
    """
    Encapsulates the notion of a Filled Order, as returned
    from a brokerage. Stores the quantity of an instrument
    actually filled and at what price. In addition, stores
    the commission of the trade from the brokerage.
    """

    def __init__(self, timeindex, symbol, exchange, quantity,
                  direction, fill_cost, commission=None):
        """
        Initialises the FillEvent object. Sets the symbol, exchange,
        quantity, direction, cost of fill and an optional
        commission.

        If commission is not provided, the Fill object will
        calculate it based on the trade size and Interactive
        Brokers fees.

        Parameters:
        timeindex - The bar-resolution when the order was filled.
        symbol - The instrument which was filled.
        exchange - The exchange where the order was filled.
        quantity - The filled quantity.
        direction - The direction of fill ('BUY' or 'SELL')
        fill_cost - The holdings value in dollars.
        commission - An optional commission sent from IB.
        """

        self.type = 'FILL'
        self.timeindex = timeindex
        self.symbol = symbol
        self.exchange = exchange
        self.quantity = quantity
        self.direction = direction
        self.fill_cost = fill_cost
```

```

# Calculate commission
if commission is None:
    self.commission = self.calculate_ib_commission()
else:
    self.commission = commission

def calculate_ib_commission(self):
    """
    Calculates the fees of trading based on an Interactive
    Brokers fee structure for API, in USD.

    This does not include exchange or ECN fees.

    Based on "US API Directed Orders":
    https://www.interactivebrokers.com/en/index.php?f=commission&p=stocks2
    """
    full_cost = 1.3
    if self.quantity <= 500:
        full_cost = max(1.3, 0.013 * self.quantity)
    else: # Greater than 500
        full_cost = max(1.3, 0.008 * self.quantity)
    return full_cost

```

14.2.2 DataHandler

事件驱动的交易系统的目标之一是最小化回测和实盘系统之间的代码重复。理想情况下，对历史回测和实时交易使用相同的信号生成方法和投资组合管理组件是最佳的。为了达到这个目标，必须跟 Strategy 对象和 Portfolio 对象协作，前者产生 SignalEvent 事件，后者基于事件产生订单 Order。针对历史数据和实盘交易需要利用统一的接口来完成。

基于 DataHandler 创建一个类层次结构，所有子类都有一个接口用于提供市场数据给其他系统中其他组件来处理，这种方式下 DataHandler 每个类都可以被替换但是并不影响策略或者投资组合计算。

子类包括 HistoricCSVDataHandler, QuantdldataHandler,

SecuritiesMasterDataHandler,

InteractiveBrokersMarketFeedDataHandler 等等。在本章中，我们将仅考虑创建历史 CSV 数据处理程序，该数据处理程序将把

Open-Low-High-Close-Volume-OpenInterest K 线中加载股票的日内 CSV 数据。然后，这可以用于在逐个方式“喂给”Strategy 和 Portfolio 处理，避免前向偏差。

首先要导入必要的 Python 库，特别是要导入 pandas 和抽象基类工具。

由于 DataHandler 生成 MarketEvents，如上所述还需要引入 event.py。

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# data.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
import os, os.path

import numpy as np
import pandas as pd

from event import MarketEvent
```

DataHandler 是一个抽象基类 (ABC)，这意味着无法直接实例化。只能实例化子类。其基本原理是 ABC 提供了一个接口，所有后续 DataHandler 子类必须遵守这个接口，从而确保与其他与之通信的类的兼容性。

我们使用__metaclass__属性让 Python 知道这是一个 ABC。此外，我们使用@abstractmethod装饰器让 Python 知道该方法将在子类中重写

（这与 C++ 中的纯虚函数相同）。

这个类有 6 个方法。前 2 个方法，`get_latest_bar` 和 `get_latest_bars` 用于查找最近的历史交易数据，主要在 `Strategy` 和 `Portfolio` 中使用，随时都需要知道当前市场价格和成交量。

接下来的 1 个方法 `get_latest_bar_datetime`，简单返回一个 Python 时间对象代表 `Bar` 的时间戳。

接下来 2 个方法，`get_latest_bar_value` 和 `get_latest_bar_values` 用于查找特定 `Bar` 的值或者一系列 `Bar` 的值。通常情况下策略只对收盘价感兴趣，我们能直接这两个方法返回一系列收盘价的浮点值，而不是从 `Bar` 对象序列中获取。这增加了策略的效率利用所谓的“回看窗口”。

最后的方法，`update_bars`，提供了一个“feed”的机制，把 `Bar` 信息置入新的数据结构，可以严格限制“前向偏差”。这是事件驱动的回测系统和基于向量的回测系统的关键差异之处。 注意，如果尝试实例化该类，则会引发异常：


```

@abstractmethod
def get_latest_bars_values(self, symbol, val_type, N=1):
    """
    Returns the last N bar values from the
    latest_symbol list, or N-k if less available.
    """
    raise NotImplementedError("Should implement
    get_latest_bars_values()")

```

```

@abstractmethod
def update_bars(self):
    """
    Pushes the latest bars to the bars_queue for each symbol
    in a tuple OHLCVI format: (datetime, open, high, low,
    close, volume, open interest).
    """
    raise NotImplementedError("Should implement update_bars()")

```

为了基于历史数据创建回测系统，我们需要考虑通过公共数据源导入数据的机制。我们在前面的章节中讨论了证券数据库的好处。因此，制作 **DataHandler** 类的一个很好的选择。

但是，为了方便起见，我想讨论一种更简单的机制，即导入（可能很大）（**CSV**）文件。这将使我们能够专注于创建 **DataHandler** 的机制，而不是关注连接到数据库和使用 **SQL** 查询来获取数据。

因此，我们将定义 **HistoricCSVDataHandler** 子类，该子类用于处理多个 **CSV** 文件，每个交易品种一个文件，并将这些文件转换为可由前面提到的方法访问的 **pandas DataFrames** 字典。

DataHandle 需要一些参数，即要推送的事件队列 **MarketEvent** 信息，**CSV** 文件的绝对路径和品种列表。下面是类的初始化：

```
# data.py

class HistoricCSVDataHandler(DataHandler):
    """
    HistoricCSVDataHandler is designed to read CSV files for
    each requested symbol from disk and provide an interface
    to obtain the "latest" bar in a manner identical to a live
    trading interface.
    """

    def __init__(self, events, csv_dir, symbol_list):
        """
        Initialises the historic data handler by requesting
        the location of the CSV files and a list of symbols.

        It will be assumed that all files are of the form
        'symbol.csv', where symbol is a string in the list.

        Parameters:
        events - The Event Queue.
        csv_dir - Absolute directory path to the CSV files.
        symbol_list - A list of symbol strings.
        """
        self.events = events
        self.csv_dir = csv_dir
        self.symbol_list = symbol_list

        self.symbol_data = {}
        self.latest_symbol_data = {}
        self.continue_backtest = True

        self._open_convert_csv_files()
```

handler 将在目录 `csv_dir` 中查找文件，并尝试使用 “.csv” 格式打开它们，其中 `SYMBOL` 是股票代码（例如 `GOOG` 或 `AAPL`）。数据文件的格式与 Yahoo Finance 一致，当然处理其他格式比如 Quandl 或者 DTN IQFeed 也很容易修改。方法 `_open_convert_csv_files` 处理文件打开。在

`CreativeCSVDataHandler` 内部使用 `pandas` 作为数据存储区的一个好处是所有被跟踪符号的索引可以一起处理。这将允许缺失的数据被补上，对于均值回归策略是必要的。注意混合所有品种所有品种时 `union` 和 `reindex` 方法的使用。


```
# data.py

def _open_convert_csv_files(self):
    """
    Opens the CSV files from the data directory, converting
    them into pandas DataFrames within a symbol dictionary.

    For this handler it will be assumed that the data is
    taken from Yahoo. Thus its format will be respected.
    """
    comb_index = None
    for s in self.symbol_list:
        # Load the CSV file with no header information, indexed on date
        self.symbol_data[s] = pd.io.parsers.read_csv(
            os.path.join(self.csv_dir, '%s.csv' % s),
            header=0, index_col=0, parse_dates=True,
            names=[
                'datetime', 'open', 'high',
                'low', 'close', 'volume', 'adj_close'
            ]
        ).sort()

        # Combine the index to pad forward values
        if comb_index is None:
            comb_index = self.symbol_data[s].index
        else:
            comb_index.union(self.symbol_data[s].index)

    # Set the latest symbol_data to None
    self.latest_symbol_data[s] = []

    # Reindex the dataframes
    for s in self.symbol_list:
        self.symbol_data[s] = self.symbol_data[s].\
            reindex(index=comb_index, method='pad').iterrows()
```

`_get_new_bar` 方法创建一个生成器以提供新的 bar。这意味着对方法的后续调用将产生一个新的 bar，直到达到品种数据的末尾：

```
# data.py

def _get_new_bar(self, symbol):
    """
    Returns the latest bar from the data feed.
    """
    for b in self.symbol_data[symbol]:

        yield b
```

要实现的 `DataHandler` 的抽象方法是 `get_latest_bar` 和 `get_latest_bars`。这些方法只提供 `latest_symbol_data` 结构的一个 Bar 数据或者最近 N 个 Bar 的数据序列：

```
# data.py

def get_latest_bar(self, symbol):
    """
    Returns the last bar from the latest_symbol list.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-1]

def get_latest_bars(self, symbol, N=1):
    """
    Returns the last N bars from the latest_symbol list,
    or N-k if less available.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-N:]
```

下一个方法 `get_latest_bar_datetime` 在最新的 bar 中查询表示“最后市场价格”的日期时间对象：

```
def get_latest_bar_datetime(self, symbol):
    """
    Returns a Python datetime object for the last bar.
    """
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return bars_list[-1][0]
```

接下来要实现的两个方法是 `get_latest_bar_value` 和 `get_latest_bar_values`。这两种方法都使用 Python `getattr` 函数，该函数查询对象以查看对象上是否存在特定属性。因此，我们可以将诸如“open”或“close”之类的字符串传递给 `getattr` 并直接从 bar 获取值，从而使该方法更加灵活。不需要再写其他方法比如 `get_latest_bar_close`。例如：

```
def get_latest_bar_value(self, symbol, val_type):
    """
    Returns one of the Open, High, Low, Close, Volume or OI
    values from the pandas Bar series object.
    """
```

```
    try:
        bars_list = self.latest_symbol_data[symbol]
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return getattr(bars_list[-1][1], val_type)

def get_latest_bars_values(self, symbol, val_type, N=1):
    """
    Returns the last N bar values from the
    latest_symbol list, or N-k if less available.
    """
    try:
        bars_list = self.get_latest_bars(symbol, N)
    except KeyError:
        print("That symbol is not available in the historical data set.")
        raise
    else:
        return np.array([getattr(b[1], val_type) for b in bars_list])
```

最后一个方法 `update_bars`，它简单的生成一个 `MarketEvent` 事件，并且被加入到队列中，当然也被作为最近的 `Bar` 数据被增加到 `latest_symbol_data` 字典中：

```
# data.py

def update_bars(self):
    """
    Pushes the latest bar to the latest_symbol_data structure
    for all symbols in the symbol list.
    """
    for s in self.symbol_list:
        try:
            bar = next(self._get_new_bar(s))
        except StopIteration:
            self.continue_backtest = False
        else:
            if bar is not None:
                self.latest_symbol_data[s].append(bar)
    self.events.put(MarketEvent())
```

我们有个 `DataHandler` 继承类，其余组件使用它来跟踪市场数据。
`Strategy`，`Portfolio` 和 `ExecutionHandler` 对象都需要当前的市场数据，因此集中它以避免这些类之间的存储重复数据。

14.2.3 策略

Strategy 对象封装了基于市场数据的所有计算工作，并生成建议信号给 **Portfolio** 对象。因此，所有“策略逻辑”都在这个类中处理。之所以在回测系统中分开 **Strategy** 和 **Portfolio** 两个对象，主要是因为这样可以处理多个 **Strategy** 生成信号给更大的 **Portfolio** 对象，这个 **Portfolio** 对象能处理它自己的风险，（如部门分配，杠杆）。在更高频率的交易中，策略和投资组合概念将紧密结合并且极其依赖于硬件。然而，这远远超出了本章的范围！

这个阶段的事件驱动回测系统开发不关心 **Indicator** 或者 **filter** 等其他技术交易介绍的，为这些创建相应的类层次也是有意义的，不过超出了本章的范围。像这种机制将会在继承的策略对象中使用。

策略层次结构相对简单，因为它由一个抽象基类组成，该基类具有用于生成 **SignalEvent** 对象的单个纯虚方法。为了创建策略层次结构，必须导入 **NumPy**, **pandas**, **Queue** 对象，抽象基类工具和 **SignalEvent**:

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# strategy.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
try:
    import Queue as queue
except ImportError:
    import queue

import numpy as np
import pandas as pd

from event import SignalEvent
```

Strategy 抽象类简单定义了一个纯虚函数 **calculate_signals** 方法，在继

承类中这个方法主要是基于市场数据生成 `SignalEvent` 对象。

```
# strategy.py

class Strategy(object):
    """
    Strategy is an abstract base class providing an interface for
    all subsequent (inherited) strategy handling objects.

    The goal of a (derived) Strategy object is to generate Signal
    objects for particular symbols based on the inputs of Bars
    (OHLCV) generated by a DataHandler object.

    This is designed to work both with historic and live data as
    the Strategy object is agnostic to where the data came from,
    since it obtains the bar tuples from a queue object.
    """

    __metaclass__ = ABCMeta

    @abstractmethod
    def calculate_signals(self):
        """
        Provides the mechanisms to calculate the list of signals.
        """
        raise NotImplementedError("Should implement calculate_signals()")
```

14.2.4 投资组合

本节描述了一个投资组合对象，该对象跟踪投资组合中的头寸，并根据信号生成固定数量的股票的订单。更复杂的投资组合对象可能包括风险管理和头寸调整工具（如 Kelly Criterion）。事实上，在接下来的章节中，我们将为我们的一些交易策略添加这些工具，以了解它们与更“天真”的投资组合方法的比较。

投资组合订单管理系统可能是事件驱动程序中最复杂的组件。它的作用是跟踪所有当前市场头寸以及头寸的市场价值（称为“持仓”）。这是仓位价值结算的一个简单估计，部分来自回测的数据处理。

除了头寸和持股管理之外，投资组合还必须了解风险因素和头寸规模控制，以优化发送给经纪上或其他形式的市场准入的订单。

不幸的是，投资组合和订单管理系统（OMS）可能变得相当复杂！因

此，我在此决定保持 **Portfolio** 对象相对简单，以便您可以理解关键想法及其实现方式。面向对象设计的本质是允许以自然的方式延伸到适应更复杂的情况。

继续在 **Event** 类层次结构中，**Portfolio** 对象必须能够处理 **SignalEvent** 对象，生成 **OrderEvent** 对象并解释 **FillEvent** 对象以更新仓位。因此，就代码行（LOC）而言，**Portfolio** 对象通常是事件驱动系统的最大组件也就不足为奇了。

我们创建一个新的文件 `portfolio.py` 并导入必要的库。这些与大多数其他类实现相同，但 **Portfolio** 不是一个抽象类。相反，它将是正常的类。这意味着它可以实例化，当我们测试策略时，**Portfolio** 是第一个可用的对象。其他 **Portfolio** 对象可以从这个基类继承，部分功能被重载以增加复杂性。

为完整起见，这是 `performance.py` 文件：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# performance.py

from __future__ import print_function

import numpy as np
import pandas as pd

def create_sharpe_ratio(returns, periods=252):
    """
    Create the Sharpe ratio for the strategy, based on a
    benchmark of zero (i.e. no risk-free rate information).

    Parameters:
    returns - A pandas Series representing period percentage returns.
    periods - Daily (252), Hourly (252*6.5), Minutely(252*6.5*60) etc.
    """
    return np.sqrt(periods) * (np.mean(returns)) / np.std(returns)

def create_drawdowns(pnl):
    """
    Calculate the largest peak-to-trough drawdown of the PnL curve
    as well as the duration of the drawdown. Requires that the
    pnl_returns is a pandas Series.

    Parameters:
    pnl - A pandas Series representing period percentage returns.

    Returns:
    drawdown, duration - Highest peak-to-trough drawdown and duration.
    """

    # Calculate the cumulative returns curve
    # and set up the High Water Mark
    hwm = [0]

    # Create the drawdown and duration series
    idx = pnl.index
    drawdown = pd.Series(index = idx)
    duration = pd.Series(index = idx)

    # Loop over the index range
    for t in range(1, len(idx)):
        hwm.append(max(hwm[t-1], pnl[t]))
        drawdown[t] = (hwm[t]-pnl[t])
        duration[t] = (0 if drawdown[t] == 0 else duration[t-1]+1)
    return drawdown, drawdown.max(), duration.max()
```

这是 Portfolio.py 文件的导入列表。 我们需要从 math 库中导入 floor 函数，以生成整数值的订单大小。 我们还需要 FillEvent 和 OrderEvent

对象，因为 Portfolio 处理两者。 注意我们增加了另外两个函数，
create_sharpe_ratio 和 create_drawdowns 都是来自于上面的
performance.py 文件。

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# portfolio.py

from __future__ import print_function

import datetime
from math import floor
try:
    import Queue as queue
except ImportError:
    import queue

import numpy as np
import pandas as pd

from event import FillEvent, OrderEvent
from performance import create_sharpe_ratio, create_drawdowns
```

Portfolio 对象的初始化需要访问条形 DataHandler，Event Queue，启动时间戳以及初始化资本数值（默认为 100,000 美元）。

投资组合旨在处理头寸规模和当前持仓，但只是以“哑”方式执行交易订单，只需将其直接发送给经纪商，而不管现金持有量。 这些都是不切实际的假设，但它们有助于了解投资组合订单管理系统(OMS)如何以事件驱动的方式运作。

portfolio 包含 all_positions 和 current_positions 两个成员。前者存储之前市场数据事件发生时记录的所有的仓位列表。简单说，仓位就是拥有投资品的数量。负仓位意味着投资品被卖空。

除了头寸数据外，投资组合还存储了持股，这些持股描述了持有头寸的当前市场价值。 在这种情况下，“当前市场价值”是指从当前市场 bar 获得的收盘价，这显然是近似值，暂时是合理的。 all_holdings

存储所有品种持仓的历史资产价值,current_holdings 存储所有品种最新的持仓资产价值:

```
# portfolio.py

class Portfolio(object):
    """
    The Portfolio class handles the positions and market
    value of all instruments at a resolution of a "bar",
    i.e. secondly, minutely, 5-min, 30-min, 60 min or EOD.

    The positions DataFrame stores a time-index of the
    quantity of positions held.

    The holdings DataFrame stores the cash and total market
    holdings value of each symbol for a particular
    time-index, as well as the percentage change in
    portfolio total across bars.
    """

    def __init__(self, bars, events, start_date, initial_capital=100000.0):
        """
        Initialises the portfolio with bars and an event queue.
        Also includes a starting datetime index and initial capital
        (USD unless otherwise stated).

        Parameters:
        bars - The DataHandler object with current market data.
        events - The Event Queue object.
        start_date - The start date (bar) of the portfolio.
        initial_capital - The starting capital in USD.
        """
        self.bars = bars
        self.events = events
        self.symbol_list = self.bars.symbol_list

        self.start_date = start_date
        self.initial_capital = initial_capital

        self.all_positions = self.construct_all_positions()
        self.current_positions = dict( (k,v) for k, v in \
            [(s, 0) for s in self.symbol_list] )

        self.all_holdings = self.construct_all_holdings()
        self.current_holdings = self.construct_current_holdings()
```

下面的方法, construct_all_positions 针对每个品种简单创建一个字典, 每个品种的仓位都初始化零值, 并且增加时间戳, 最后把字典加入列表中。实际上最后形成的结果是字典的列表:

```
# portfolio.py
```



```
def construct_all_positions(self):
    """
    Constructs the positions list using the start_date
    to determine when the time index will begin.
    """
    d = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    d['datetime'] = self.start_date
    return [d]
```

`construct_all_holdings` 方法与上述方法类似，但增加了现金，佣金和总额的 key，分别代表任何购买后账户中的剩余现金，累计佣金和包括现金和任何未平仓头寸在内的总账户净值。起始现金和总账户净值都设定为初始资本。

以这种方式，每个品种都有单独的“账户”，“手头现金”，“支付佣金”（Interactive Broker 费用）和“总”投资组合价值。显然，这并未考虑保证金要求或卖空约束条件，但足以让您了解如何创建此类

OMS:

```
# portfolio.py

def construct_all_holdings(self):
    """
    Constructs the holdings list using the start_date
    to determine when the time index will begin.
    """
    d = dict( (k,v) for k, v in [(s, 0.0) for s in self.symbol_list] )
    d['datetime'] = self.start_date
    d['cash'] = self.initial_capital
    d['commission'] = 0.0
    d['total'] = self.initial_capital
    return [d]
```

下面的方法，`construct_current_holdings` 几乎跟上面一样，除了它不需要把字典加入列表，因为它只需要创建一个记录：

```
# portfolio.py

def construct_current_holdings(self):
    """
    This constructs the dictionary which will hold the instantaneous
    value of the portfolio across all symbols.
    """
    d = dict( (k,v) for k, v in [(s, 0.0) for s in self.symbol_list] )
    d['cash'] = self.initial_capital
    d['commission'] = 0.0
    d['total'] = self.initial_capital
    return d
```

每次心跳，即每次从 `DataHandler` 对象请求新的市场数据时，投资组合必须更新所有持仓的当前市场价值。在实时交易场景中，这些信息可以直接从经纪商处下载和解析，但对于回测试实施，有必要根据数据库 `DataHandler` 手动计算这些值。

不幸的是，由于 `bid/ask` 和流动性问题，没有“当前市场价值”这样的东西。因此，有必要通过将所持有的资产数量乘以特定的近似“价格”来估计它。我在这里采用的方法是使用收到的最后一个 `bar` 的收盘价。对于日内策略，这是相对现实的。对于每日策略而言，这不太现实，因为开盘价可能与收盘价大不相同。

`update_timeindex` 方法处理新的馆藏跟踪。它首先获得了

`update_timeindex` 方法跟踪处理新的持仓。首先获得来自于

`DataHandler` 最新的价格，创建新的品种字典代表当前仓位，并且设置“新”的仓位数据等于“当前”仓位数据。

仅仅当收到 `FillEvent` 事件时，当前持仓数据才需要修改，这将在后面 `Portfolio` 代码中处理。后面会添加当前持仓资产价值到 `all_positions` 列表。

然后 `holdings` 以类似的方式更新持仓，除了通过将当前头寸数与最新 `bar` 的收盘价相乘来重新计算市场价值。

最后，新的持仓价值加到 `all_holdings`:

```
# portfolio.py

def update_timeindex(self, event):
    """
    Adds a new record to the positions matrix for the current
    market data bar. This reflects the PREVIOUS bar, i.e. all
    current market data at this stage is known (OHLCV).

    Makes use of a MarketEvent from the events queue.
    """
    latest_datetime = self.bars.get_latest_bar_datetime(
        self.symbol_list[0]
    )

    # Update positions
    # =====
    dp = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    dp['datetime'] = latest_datetime

    for s in self.symbol_list:
        dp[s] = self.current_positions[s]

    # Append the current positions
    self.all_positions.append(dp)

    # Update holdings
    # =====
    dh = dict( (k,v) for k, v in [(s, 0) for s in self.symbol_list] )
    dh['datetime'] = latest_datetime
    dh['cash'] = self.current_holdings['cash']
    dh['commission'] = self.current_holdings['commission']
    dh['total'] = self.current_holdings['cash']

    for s in self.symbol_list:
        # Approximation to the real value
        market_value = self.current_positions[s] * \
            self.bars.get_latest_bar_value(s, "adj_close")
        dh[s] = market_value
        dh['total'] += market_value

    # Append the current holdings
    self.all_holdings.append(dh)
```

`update_positions_from_fill` 方法检查 `FillEvent` 事件确定买卖方向，然后根据增加或者减少持仓数量更新 `current_positions` 字典:

```
# portfolio.py

def update_positions_from_fill(self, fill):
    """
    Takes a Fill object and updates the position matrix to
    reflect the new position.

    Parameters:
    fill - The Fill object to update the positions with.
    """
    # Check whether the fill is a buy or sell
    fill_dir = 0
    if fill.direction == 'BUY':
        fill_dir = 1
    if fill.direction == 'SELL':
        fill_dir = -1

    # Update positions list with new quantities
    self.current_positions[fill.symbol] += fill_dir*fill.quantity
```

相应的 `update_holdings_from_fill` 与上述方法类似，但更新了持有值。为了模拟交易的成本，以下方法不使用与 `FillEvent` 关联的成本。为什么是这样？简单地说，在回测环境中，填充成本实际上是未知的（市场影响和书的深度未知），因此必须进行估算。

因此，交易成本被设置为“当前市场价格”，即当前市场 `bar` 的收盘价。然后将特定品种的持有量设置为等于交易成本乘以交易量。对于流动市场中的大多数低频交易策略，这是一个合理的近似值，但在高频率下，这些问题需要在生产回测和实时交易引擎中考虑。

一旦知道交易成本，就可以更新当前的持仓，现金和总价值，累计佣金也会更新：


```
# portfolio.py

def update_holdings_from_fill(self, fill):
    """
    Takes a Fill object and updates the holdings matrix to
    reflect the holdings value.

    Parameters:
    fill - The Fill object to update the holdings with.
    """
    # Check whether the fill is a buy or sell
    fill_dir = 0
    if fill.direction == 'BUY':
        fill_dir = 1
    if fill.direction == 'SELL':
        fill_dir = -1

    # Update holdings list with new quantities
    fill_cost = self.bars.get_latest_bar_value(fill.symbol, "adj_close")
    cost = fill_dir * fill_cost * fill.quantity
    self.current_holdings[fill.symbol] += cost
    self.current_holdings['commission'] += fill.commission
    self.current_holdings['cash'] -= (cost + fill.commission)

self.current_holdings['total'] -= (cost + fill.commission)
```

update_fill 方法实现在下面，这个方法简单执行前面 2 个函数，update_positions_from_fill 和 update_holdings_from_fill 两个方法处理接收到的 FillEvent 事件。

```
# portfolio.py

def update_fill(self, event):
    """
    Updates the portfolio current positions and holdings
    from a FillEvent.
    """
    if event.type == 'FILL':
        self.update_positions_from_fill(event)
        self.update_holdings_from_fill(event)
```

虽然 Portfolio 对象必须处理 FillEvents，但它还必须在收到一个或多个 SignalEvent 时生成 OrderEvents。generate_naive_order 方法只需要一个信号来做多或做空信号，发送订单就可以获得 100 股这样的资产。显然，100 是任意值，并且显然取决于模拟中的投资组合总权益。在实际实施中，该值将由风险管理或仓位大小确定。然而，这是一

个简单的投资组合，所以它直接从信号发送所有订单，没有风险系统。该方法基于当前数量和特定符号来处理仓位的买入，卖空和平仓。然后生成相应的 OrderEvent 对象：

```
# portfolio.py

def generate_naive_order(self, signal):
    """
    Simply files an Order object as a constant quantity
    sizing of the signal object, without risk management or
    position sizing considerations.

    Parameters:
    signal - The tuple containing Signal information.
    """
    order = None

    symbol = signal.symbol
    direction = signal.signal_type
    strength = signal.strength

    mkt_quantity = 100
    cur_quantity = self.current_positions[symbol]
    order_type = 'MKT'

    if direction == 'LONG' and cur_quantity == 0:
        order = OrderEvent(symbol, order_type, mkt_quantity, 'BUY')
    if direction == 'SHORT' and cur_quantity == 0:
        order = OrderEvent(symbol, order_type, mkt_quantity, 'SELL')

    if direction == 'EXIT' and cur_quantity > 0:
        order = OrderEvent(symbol, order_type, abs(cur_quantity), 'SELL')
    if direction == 'EXIT' and cur_quantity < 0:
        order = OrderEvent(symbol, order_type, abs(cur_quantity), 'BUY')

    return order
```

update_signal 方法简单调用上面的方法，并增加 OrderEvent 事件到信号事件队列：

```
# portfolio.py

def update_signal(self, event):
    """
    Acts on a SignalEvent to generate new orders
    based on the portfolio logic.
    """
    if event.type == 'SIGNAL':
        order_event = self.generate_naive_order(event)
        self.events.put(order_event)
```

倒数第二个方法是关于净值曲线的生成。简单地创建一个 returns 序

列用于收益计算，然后基于百分比正规化收益曲线。账户初始值为 1.0，相对于全部美元总量：

```
# portfolio.py

def create_equity_curve_dataframe(self):
    """
    Creates a pandas DataFrame from the all_holdings
    list of dictionaries.
    """
    curve = pd.DataFrame(self.all_holdings)
    curve.set_index('datetime', inplace=True)
    curve['returns'] = curve['total'].pct_change()
    curve['equity_curve'] = (1.0+curve['returns']).cumprod()
    self.equity_curve = curve
```

投资组合中的最终方法是净值曲线的输出和与策略相关的各种绩效统计。最后一行将一个文件 `equity.csv` 输出到与代码相同的目录，该目录可以加载到 Matplotlib Python 脚本（或电子表格，如 MS Excel 或 LibreOffice Calc）中，以供后续分析。

请注意，DrawDown Duration 是指回测持续的时间，以数据 Bar 的绝对数目来计算，与特定的时间周期相关。

```
def output_summary_stats(self):
    """
    Creates a list of summary statistics for the portfolio.
    """
    total_return = self.equity_curve['equity_curve'][-1]
    returns = self.equity_curve['returns']
    pnl = self.equity_curve['equity_curve']

    sharpe_ratio = create_sharpe_ratio(returns, periods=252*60*6.5)
    drawdown, max_dd, dd_duration = create_drawdowns(pnl)
    self.equity_curve['drawdown'] = drawdown

    stats = [("Total Return", "%0.2f%%" % \
                ((total_return - 1.0) * 100.0)),
            ("Sharpe Ratio", "%0.2f" % sharpe_ratio),
            ("Max Drawdown", "%0.2f%%" % (max_dd * 100.0)),
            ("Drawdown Duration", "%d" % dd_duration)]

    self.equity_curve.to_csv('equity.csv')
    return stats
```

Portfolio 对象是整个事件驱动的回测系统中最复杂的方面。这里的

实施虽然错综复杂，但在处理职位时却相对基础。

14.2.5 执行处理程序

在本节中，我们将通过创建一个类层次结构来研究交易订单的执行，该层次结构将代表模拟的订单处理机制，并最终与经纪上或其他市场连接方式相关联。

这里描述的 `ExecutionHandler` 是非常简单的，因为它把所有订单的价格都定为当前市场价格，这实际上是不可行的，但可以作为改进的基础。

我们也需要导入先前定义的抽象基类，另外也需要导入 `FillEvent` 事件和 `OrderEvent` 事件：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# execution.py

from __future__ import print_function

from abc import ABCMeta, abstractmethod
import datetime
try:
    import Queue as queue
except ImportError:
    import queue

from event import FillEvent, OrderEvent
```

`ExecutionHandler` 像之前抽象基类类似由一个纯虚函数，

`execute_order`:


```
# execution.py

class ExecutionHandler(object):
    """
    The ExecutionHandler abstract class handles the interaction
    between a set of order objects generated by a Portfolio and
    the ultimate set of Fill objects that actually occur in the
    market.

    The handlers can be used to subclass simulated brokerages
    or live brokerages, with identical interfaces. This allows
    strategies to be backtested in a very similar manner to the
    live trading engine.
    """

    __metaclass__ = ABCMeta

    @abstractmethod
    def execute_order(self, event):
        """
        Takes an Order event and executes it, producing
        a Fill event that gets placed onto the Events queue.

        Parameters:
        event - Contains an Event object with order information.
        """
        raise NotImplementedError("Should implement execute_order()")
```

为了回溯策略，我们需要模拟交易的交易方式。最简单的实施方案是假设所有订单都以当前市场价格填写所有数量。这显然是非常不现实的，改善回测的很大一部分将来自设计更复杂的滑点和市场影响模型。

请注意，fill_cost 的 FillEvent 值为 None（请参阅 execute_order 中的倒数第二行），因为我们已经处理了上述 Portfolio 对象中的填充成本。在更现实的实施中，我们将利用“当前”市场数据值来获得实际的填充成本。

为了回测我只是简单地使用 ARCA 交易所。只是一个字符串占位符，实时交易执行环境中，这将是相当重要的：

```
# execution.py

class SimulatedExecutionHandler(ExecutionHandler):
    """
    The simulated execution handler simply converts all order
    objects into their equivalent fill objects automatically
    without latency, slippage or fill-ratio issues.

    This allows a straightforward "first go" test of any strategy,
    before implementation with a more sophisticated execution
    handler.
    """

    def __init__(self, events):
        """
        Initialises the handler, setting the event queues
        up internally.

        Parameters:
        events - The Queue of Event objects.
        """
        self.events = events

    def execute_order(self, event):
        """
        Simply converts Order objects into Fill objects naively,
        i.e. without any latency, slippage or fill ratio problems.

        Parameters:
        event - Contains an Event object with order information.
        """
        if event.type == 'ORDER':
            fill_event = FillEvent(
                datetime.datetime.utcnow(), event.symbol,
                'ARCA', event.quantity, event.direction, None
            )
            self.events.put(fill_event)
```

14.2.6 回测

我们现在可以创建 **Backtest** 类。 **Backtest** 类封装了事件处理逻辑，并且基本上将我们上面讨论过的所有其他类联系在一起。

Backtest 对象旨在执行嵌套的 **while** 循环事件驱动系统以处理放置在 **Event Queue** 对象上的事件。外部 **while** 循环称为“心跳循环”，并决定回测系统的时间精度。在实盘环境中，此值将为正数，例如 600 秒（每十分钟）。因此，市场数据和头寸将仅在此时间范围内更新。

对于这里描述的回测系统，“心跳”可以设置为零，而不管它是什么

策略频率，因为历史数据都已经存在。

我们可以以任何速度运行回测，因为事件驱动的系统对数据何时可用是不可知的，只要它具有相关的时间戳。因此，我只是将其包含在内，以演示实时交易引擎的运作方式。一旦 `DataHandler` 通过使用 `boolean continue_backtest` 属性让 `Backtest` 对象知道，外部循环就会结束。

内部 `while` 循环实际上处理信号并根据事件类型将它们发送到正确的组件。因此，事件队列不断被填充和减少事件。这就是系统事件驱动的意义。

首先是导入必要的库。我们导入 `pprint`（“pretty-print”），因为我们想以输出友好的方式显示统计数据：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# backtest.py

from __future__ import print_function

import datetime
import pprint
try:
    import Queue as queue
except ImportError:
    import queue
import time
```

回测对象的初始化需要 `CSV` 目录，完整的交易品种列表，初始化资本，心态间隔（毫秒计），启动时间戳，`DataHandler`, `ExecutionHandler`, `Portfolio` 以及 `Strategy` 对象等参数。`Queue` 用于存储事件。信号数量，订单数量，交易数量也是很重要的：

```
# backtest.py

class Backtest(object):
    """
    Encapsulates the settings and components for carrying out
    an event-driven backtest.
    """

    def __init__(
        self, csv_dir, symbol_list, initial_capital,
        heartbeat, start_date, data_handler,
        execution_handler, portfolio, strategy
    ):
        """
        Initialises the backtest.
```

```
Parameters:
csv_dir - The hard root to the CSV data directory.
symbol_list - The list of symbol strings.
initial_capital - The starting capital for the portfolio.
heartbeat - Backtest "heartbeat" in seconds
start_date - The start datetime of the strategy.
data_handler - (Class) Handles the market data feed.
execution_handler - (Class) Handles the orders/fills for trades.
portfolio - (Class) Keeps track of portfolio current
              and prior positions.
strategy - (Class) Generates signals based on market data.
"""

self.csv_dir = csv_dir
self.symbol_list = symbol_list
self.initial_capital = initial_capital
self.heartbeat = heartbeat
self.start_date = start_date

self.data_handler_cls = data_handler
self.execution_handler_cls = execution_handler
self.portfolio_cls = portfolio
self.strategy_cls = strategy

self.events = queue.Queue()

self.signals = 0
self.orders = 0
self.fills = 0
self.num_strats = 1

self._generate_trading_instances()
```

第一个方法 `_generate_trading_instances`, 生成所有内部交易对象
(`DataHandler`, `Strategy`, `Portfolio` 和 `ExecutionHandler`) :


```
# backtest.py

def _generate_trading_instances(self):
    """
    Generates the trading instance objects from
    their class types.
    """
    print(
        "Creating DataHandler, Strategy, Portfolio and ExecutionHandler"
    )
    self.data_handler = self.data_handler_cls(self.events, self.csv_dir,
        self.symbol_list)
    self.strategy = self.strategy_cls(self.data_handler, self.events)
    self.portfolio = self.portfolio_cls(self.data_handler, self.events,
        self.start_date,
        self.initial_capital)
    self.execution_handler = self.execution_handler_cls(self.events)
```

`_run_backtest` 方法是执行 Backtest 引擎的信号处理的地方。如上所述，有两个 `while` 循环，一个嵌套在另一个循环中。外部跟踪系统的心跳，而内部检查 `Queue` 对象中是否有事件，并根据事件类型调用合适组件的相应方法。

对于 `MarketEvent`，`Strategy` 对象被告知重新计算新事件，而 `Portfolio` 对象被告知重新索引时间。`Portfolio` 收到一个 `SignalEvent`，处理之后会生成一系列 `OrderEvent` 事件。如果 `ExecutionHandler` 收到 `OrderEvent` 事件，它将会把订单提交到交易所处理（实盘交易环境）。最后，如果收到 `FillEvent`，投资组合将更新自己以了解新的头寸：

```
# backtest.py

def _run_backtest(self):
    """
    Executes the backtest.
    """
    i = 0
    while True:
        i += 1
        print i
        # Update the market bars
        if self.data_handler.continue_backtest == True:
            self.data_handler.updateBars()
        else:
            break

    # Handle the events
    while True:
        try:
            event = self.events.get(False)
        except queue.Empty:
            break
        else:
            if event is not None:
                if event.type == 'MARKET':
                    self.strategy.calculate_signals(event)
                    self.portfolio.update_timeindex(event)
```

```
                elif event.type == 'SIGNAL':
                    self.signals += 1
                    self.portfolio.update_signal(event)

                elif event.type == 'ORDER':
                    self.orders += 1
                    self.execution_handler.execute_order(event)

                elif event.type == 'FILL':
                    self.fills += 1
                    self.portfolio.update_fill(event)

            time.sleep(self.heartbeat)
```

一旦回测模拟完成，策略的表现情况就可以显示出来了。创建收益曲线并显示摘要统计，信号数量，订单数量，交易数量等。

```
# backtest.py

def _output_performance(self):
    """
    Outputs the strategy performance from the backtest.
```

```

"""
self.portfolio.create_equity_curve_dataframe()

print("Creating summary stats...")
stats = self.portfolio.output_summary_stats()

print("Creating equity curve...")
print(self.portfolio.equity_curve.tail(10))
pprint.pprint(stats)

print("Signals: %s" % self.signals)
print("Orders: %s" % self.orders)
print("Fills: %s" % self.fills)

```

最后一个方法是 `simulate_trading` 实现。简单按顺序调用之前两个方法：

```

# backtest.py

def simulate_trading(self):
    """
    Simulates the backtest and outputs portfolio performance.
    """
    self._run_backtest()
    self._output_performance()

```

这就结束了事件驱动的 `backtester` 操作对象。

14.3 事件驱动的执行

上面我们描述了一个基本的 `ExecutionHandler` 类，它只为每个 `OrderEvent` 创建了一个相应的 `FillEvent` 实例。这正是我们对“首次通过”回测所需要的，但是当我们希望将系统实际连接到经纪商时，我们需要更复杂的处理。在本节中，我们定义了 `IBExecutionHandler`，这个类允许我们与流行的 `Interactive Brokers API` 进行通信，从而自动执行。

`IBExecutionHandler` 类的基本思想是从事件队列接收 `OrderEvent` 实例，然后使用开源 `IbPy` 库直接针对 `Interactive Brokers` 订单 API 执行它们。该类还将处理通过 API 发回的“服务器返回”消息。在此阶段，所采取的唯一操作是创建相应的 `FillEvent` 事件，然后将其添加事件队列。

这个类本身可能变得相当复杂，具有执行优化逻辑以及复杂的错误处理。但是，我选择在这里保持相对简单，以便您可以看到主要想法并将其扩展到适合您特定交易风格的方向。

与往常一样，首先是创建 Python 文件并导入必要的库。该文件名为 `ib_execution.py`，与其他事件驱动文件位于同一目录中。我们导入必要的 `date/time` 处理库，`ibPy` 对象和 `IBExecutionHandler` 处理的特定事件类：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# ib_execution.py

from __future__ import print_function

import datetime
import time

from ib.ext.Contract import Contract
from ib.ext.Order import Order
from ib.opt import ibConnection, message

from event import FillEvent, OrderEvent
from execution import ExecutionHandler
```

我们现在定义 `IBExecutionHandler` 类。 `__init__` 构造函数首先需要知道事件队列。 它还需要知道 `order_routing`，我已经默认为“SMART”。如果您有特定的交易要求，可以在此处指定。

默认货币也已设置为美元。 在该方法中，我们创建了一个 `fill_dict` 字典，稍后需要用于生成 `FillEvent` 实例。我们还创建了一个 `twse_conn` 连接对象，用于将我们的连接信息存储到 Interactive Brokers API。 我们还必须创建一个初始默认 `order_id`，它会跟踪所有后续订单以避免重复。最后，我们注册消息处理程序（我们将在下面更详细地定义）：

```
# ib_execution.py

class IBExecutionHandler(ExecutionHandler):
    """
    Handles order execution via the Interactive Brokers
    API, for use against accounts when trading live
    directly.
    """

    def __init__(
        self, events, order_routing="SMART", currency="USD"
    ):
        """
        Initialises the IBExecutionHandler instance.
        """
        self.events = events
        self.order_routing = order_routing
        self.currency = currency
        self.fill_dict = {}

        self.tws_conn = self.create_tws_connection()
        self.order_id = self.create_initial_order_id()
        self.register_handlers()
```

IB API 使用基于消息的事件系统允许我们的类以特定方式处理某些消息，其方式与事件驱动的 `backtester` 本身类似。我没有包括任何真正的错误处理（为了简洁起见），仅仅只是通过 `_error_handler` 方法向终端输出错误。

另一方面，`_reply_handler` 方法用于确定是否需要生成 `FillEvent` 事件。方法会检查是否收到 "openOrder" 消息，并检查我们的 `fill_dict` 中是否已设置此特定 `orderId` 的条目。如果没有，则创建一个。

如果它看到 "orderStatus" 消息则表明该订单已被交易，接下来应该调用 `create_fill` 创建 `FillEvent` 事件，为了调试目的，也会向终端输出日志信息：

```
# ib_execution.py

def _error_handler(self, msg):
    """
```



```

Handles the capturing of error messages
"""
# Currently no error handling.
print("Server Error: %s" % msg)

def _reply_handler(self, msg):
    """
    Handles of server replies
    """
    # Handle open order orderId processing
    if msg.typeName == "openOrder" and \
        msg.orderId == self.order_id and \
        not self.fill_dict.has_key(msg.orderId):
        self.create_fill_dict_entry(msg)
    # Handle Fills
    if msg.typeName == "orderStatus" and \
        msg.status == "Filled" and \
        self.fill_dict[msg.orderId]["filled"] == False:
        self.create_fill(msg)
    print("Server Response: %s, %s\n" % (msg.typeName, msg))

```

下面的方法 `create_tws_connection`，使用 `IbPy` 库的 `ibConnection` 对象创建了一个与 IB API 之间的连接，默认端口为 7496 和默认的 `clientId` 为 10。一旦对象创建，为了进行连接，`connect` 方法可以被调用。

```

# ib_execution.py

def create_tws_connection(self):
    """
    Connect to the Trader Workstation (TWS) running on the
    usual port of 7496, with a clientId of 10.
    The clientId is chosen by us and we will need
    separate IDs for both the execution connection and
    market data connection, if the latter is used elsewhere.
    """
    tws_conn = ibConnection()
    tws_conn.connect()
    return tws_conn

```

为了跟踪不同的订单（跟踪成交的目的）使用了下面的方法 `create_initial_order_id`。我们设置的默认值为 1，一个更好的方法是通过 IB 查询最近可用 ID 后使用。你可以通过 `Trader Workstation > Global Configuration > API 配置面板` 重置当前的 API 订单 ID：

```
# ib_execution.py

def create_initial_order_id(self):
    """
    Creates the initial order ID used for Interactive
    Brokers to keep track of submitted orders.
    """
    # There is scope for more logic here, but we
    # will use "1" as the default for now.
    return 1
```

下面的方法 `register_handlers`，通过 TWS 连接简单地注册了错误处理和回复处理方法：

```
# ib_execution.py

def register_handlers(self):
    """
    Register the error and server reply
    message handling functions.
    """
    # Assign the error handling function defined above
    # to the TWS connection
    self.tws_conn.register(self._error_handler, 'Error')

    # Assign all of the server reply messages to the
    # reply_handler function defined above
    self.tws_conn.registerAll(self._reply_handler)
```

为了实际交易，有必要创建一个 `IbPy Contract` 实例，然后将其与 `IbPy Order` 实例配对，该实例将被发送到 IB API。以下方法 `create_contract` 生成此对的第一个组件。期望交易品种代码，证券模式（例如股票或期货），交易所/主要交易所和货币。它返回 `Contract` 实例：

```
# ib_execution.py

def create_contract(self, symbol, sec_type, exch, prim_exch, curr):
    """
    Create a Contract object defining what will
    be purchased, at which exchange and in which currency.

    symbol - The ticker symbol for the contract
    sec_type - The security type for the contract ('STK' is 'stock')
    exch - The exchange to carry out the contract on
    prim_exch - The primary exchange to carry out the contract on
    curr - The currency in which to purchase the contract
    """
    contract = Contract()
    contract.m_symbol = symbol
    contract.m_secType = sec_type
    contract.m_exchange = exch
    contract.m_primaryExch = prim_exch
    contract.m_currency = curr
    return contract
```

下面方法 `create_order`，它需要交易类型参数 `order_type`（市价单或者限价单），交易资产数量和交易方向（买或者卖）。最终返回 `Order` 对象：

```
# ib_execution.py

def create_order(self, order_type, quantity, action):
    """
    Create an Order object (Market/Limit) to go long/short.

    order_type - 'MKT', 'LMT' for Market or Limit orders
    quantity - Integral number of assets to order
    action - 'BUY' or 'SELL'
    """
    order = Order()
    order.m_orderType = order_type
    order.m_totalQuantity = quantity

    order.m_action = action
    return order
```

为了避免为订单 ID 重复 `FillEvent` 实例，我们使用名为 `fill_dict` 的字典来存储与特定订单 ID 数据。交易生成后，特定订单 ID 的条目的

“filled”键将设置为 `True`。如果从 IB 收到后续的“服务器返回”消息，表明订单已被交易（并且是重复的消息），则不会导致新的交易。

以下方法 `create_fill_dict_entry` 执行此操作：


```
# ib_execution.py

def create_fill_dict_entry(self, msg):
    """
    Creates an entry in the Fill Dictionary that lists
    orderIds and provides security information. This is
    needed for the event-driven behaviour of the IB
    server message behaviour.
    """
    self.fill_dict[msg.orderId] = {
        "symbol": msg.contract.m_symbol,
        "exchange": msg.contract.m_exchange,
        "direction": msg.order.m_action,
        "filled": False
    }
```

下面的方法 `create_fill`，真正创建 `FillEvent` 事件对象然后放入事件队列：

```
# ib_execution.py

def create_fill(self, msg):
    """
    Handles the creation of the FillEvent that will be
    placed onto the events queue subsequent to an order
    being filled.
    """
    fd = self.fill_dict[msg.orderId]

    # Prepare the fill data
    symbol = fd["symbol"]
    exchange = fd["exchange"]
    filled = msg.filled
    direction = fd["direction"]
    fill_cost = msg.avgFillPrice

    # Create a fill event object
    fill = FillEvent(
        datetime.datetime.utcnow(), symbol,
        exchange, filled, direction, fill_cost
    )

    # Make sure that multiple messages don't create
    # additional fills.
    self.fill_dict[msg.orderId]["filled"] = True

    # Place the fill event onto the event queue
    self.events.put(fill_event)
```

现在已经实现了所有前面的方法，它仍然是从 `ExecutionHandler` 抽象基类重写 `execute_order` 方法。该方法实际上使用 IB API 执行订单。我们首先检查接收到此方法的事件实际上是否为 `OrderEvent`，然后使

用各自的参数准备 Contract 和 Order 对象。一旦创建了两者，那么就可以调用 IbPy 库连接对象的 placeOrder 方法，参数是订单 order_id。调用 time.sleep(1) 方法确保订单实际进入 IB 非常重要。删除此行会导致 API 的行为不一致，至少在我的系统上是这样！

最后，我们增加订单 ID 以确保我们不重复订单：

```
# ib_execution.py

def execute_order(self, event):
    """
    Creates the necessary InteractiveBrokers order object
    and submits it to IB via their API.

    The results are then queried in order to generate a
    corresponding Fill object, which is placed back on
    the event queue.

    Parameters:
    event - Contains an Event object with order information.
    """
    if event.type == 'ORDER':
        # Prepare the parameters for the asset order
        asset = event.symbol
        asset_type = "STK"
        order_type = event.order_type
        quantity = event.quantity
        direction = event.direction

        # Create the Interactive Brokers contract via the
        # passed Order event
        ib_contract = self.create_contract(
            asset, asset_type, self.order_routing,
            self.order_routing, self.currency
        )

        # Create the Interactive Brokers order via the
        # passed Order event
        ib_order = self.create_order(
            order_type, quantity, direction
        )

        # Use the connection to the send the order to IB
        self.tws_conn.placeOrder(
            self.order_id, ib_contract, ib_order
        )

        # NOTE: This following line is crucial.
        # It ensures the order goes through!
        time.sleep(1)

        # Increment the order ID for this session
```

```
self.order_id += 1
```

此类构成 Interactive Brokers 执行处理程序的基础，可用于代替模拟执行处理程序，该处理程序仅适用于回测。然而，在可以使用 IB 处理程序之前，仍然需要创建一个实盘数据处理以取代回测系统的历史数据处理。

通过这种方式，我们尽可能地从回测和实时系统中重用，以确保代码“替换”最小化，因此两者之间的行为不同，但类似。

第 15 章

交易策略实施

在本章中，我们将考虑使用上述事件驱动的回测系统全面实施交易策略。特别是，我们将使用名义投资组合金额为所有交易策略生成净值曲线，从而模拟保证金/杠杆的概念，与基于向量化/基于回报的方法相比，这是一种更为现实的方法。

首先策略可以使用雅虎财经，谷歌财经或 **Quandl** 免费提供的数据进行。这些策略适用于那些希望仅研究策略的交易信号生成方面或整个端到端系统的长期算法交易者。这种策略通常具有较小的夏普比率，但更容易实现和执行。

后一策略使用日内股票数据进行。这些数据通常不是免费提供的，通常需要商业数据供应商来提供足够的质量和数量的数据。我自己使用 **DTN IQFeed**。这种策略通常具有更大的夏普比率，但需要更复杂的实施，因为高频率需要深入的自动化。

我们将看到，我们在日间数据上创建交易策略的前两次尝试并未完全成功。一旦考虑到交易成本，就在日间数据上找出有利可图的交易策略具有挑战性。后者是许多关于算法交易的文本往往遗漏的东西。但是，我认为必须在回测中加入尽可能多的因素，以尽量减少意外情况。

此外，本书主要是关于如何有效地创建一个真实的日间或日内回测系统（以及一个实时执行平台），而不是关于特定的个人策略。在互联网上找到交易策略比创建一个逼真的强大回测系统要困难得多！

虽然提出的前两个策略并不是特别有吸引力，但后一种策略（日内数据）表现良好，让我们有信心使用更高频率的数据。

15.1 均线交叉策略

我很喜欢均线交叉技术策略，因为它是一个很简单的策略，用于回测的实现非常方便。在回测期较长的情况下，在单个品种上生成的信号很少，因此很容易手动验证系统的行为是否符合预期。

基于先前的回测代码生成模型，我们需要按照前一章所述继 `Strategy` 对象，以创建 `MovingAverageCrossStrategy` 对象，该对象将包含简单均线的逻辑和交易信号的生成。

此外，我们需要创建 `__main__` 函数，该函数将加载 `Backtest` 对象并实际封装程序的执行。以下文件 `mac.py` 包含这两个对象。

与往常一样，首先是正确导入必要的组件。我们主要导入上一章中描述的所有对象：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# mac.py

from __future__ import print_function

import datetime

import numpy as np
import pandas as pd
import statsmodels.api as sm

from strategy import Strategy
from event import SignalEvent
from backtest import Backtest
from data import HistoricCSVDataHandler
from execution import SimulatedExecutionHandler
from portfolio import Portfolio
```

现在我们转向 `MovingAverageCrossStrategy` 的创建。该策略需要 `DataHandler`，`Event Queue` 以及将在策略中使用的简单均线。我已选择 100 和 400 作为此策略的“短期”和“长期”均线指标。

在回测实际上是“在市场中”买入的信号用于告知策略。开仓信号

仅在“OUT”时生成，并且只有在“LONG”或“SHORT”信号产生时才进行操作：

```
# mac.py

class MovingAverageCrossStrategy(Strategy):
    """
    Carries out a basic Moving Average Crossover strategy with a
    short/long simple weighted moving average. Default short/long
    windows are 100/400 periods respectively.
    """

    def __init__(
        self, bars, events, short_window=100, long_window=400
    ):
        """
        Initialises the Moving Average Cross Strategy.

        Parameters:
        bars - The DataHandler object that provides bar information
        events - The Event Queue object.
        short_window - The short moving average lookback.
        long_window - The long moving average lookback.
        """
        self.bars = bars
        self.symbol_list = self.bars.symbol_list
        self.events = events
        self.short_window = short_window
        self.long_window = long_window

        # Set to True if a symbol is in the market
        self.bought = self._calculate_initial_bought()
```

由于策略无仓位开始，我们将每个品种的初始“bought”值设置为“OUT”

```
# mac.py

def _calculate_initial_bought(self):
    """
    Adds keys to the bought dictionary for all symbols
    and sets them to 'OUT'.
    """
    bought = {}
    for s in self.symbol_list:
        bought[s] = 'OUT'
    return bought
```

该策略的核心是 `calculate_signals`。它对 `MarketEvent` 做出反应，并且获得交易的每个交易品种的最新 `N` bar 的收盘价，其中 `N` 等于最大

的回测期。

然后计算短期和长期简单移动平均线。 策略的规则是当短移动平均值超过长移动平均值时进入市场（做多股票）。 相反，如果长移动平均值超过短移动平均值，则告知策略平仓。

通过每个相应情况下将 `SignalEvent` 对象放置在 `Event Queue` 上，然后分别将“bought”属性（每个符号）更新为“LONG”或“OUT”来处理该逻辑。 由于这是一项单边策略，我们不会考虑“做空”市场：

```
# mac.py
```

```
def calculate_signals(self, event):
    """
    Generates a new set of signals based on the MAC
    SMA with the short window crossing the long window
    meaning a long entry and vice versa for a short entry.

    Parameters
    event - A MarketEvent object.
    """
    if event.type == 'MARKET':
        for s in self.symbol_list:
            bars = self.bars.get_latest_bars_values(
                s, "adj_close", N=self.long_window
            )
            bar_date = self.bars.get_latest_bar_datetime(s)
            if bars is not None and bars != []:
                short_sma = np.mean(bars[-self.short_window:])
                long_sma = np.mean(bars[-self.long_window:])

                symbol = s
                dt = datetime.datetime.utcnow()
                sig_dir = ""

                if short_sma > long_sma and self.bought[s] == "OUT":
                    print("LONG: %s" % bar_date)
                    sig_dir = 'LONG'
```

```
                signal = SignalEvent(1, symbol, dt, sig_dir, 1.0)
                self.events.put(signal)
                self.bought[s] = 'LONG'
            elif short_sma < long_sma and self.bought[s] == "LONG":
                print("SHORT: %s" % bar_date)
                sig_dir = 'EXIT'
                signal = SignalEvent(1, symbol, dt, sig_dir, 1.0)
                self.events.put(signal)
                self.bought[s] = 'OUT'
```

结束 `MovingAverageCrossStrategy` 对象实现。整个回测系统的最后一项任务是在 `mac.py` 中使用一个 `__main__` 方法来实际执行回测。

首先,确保将 `csv_dir` 的值更改为金融数据的 CSV 文件目录的路径。您还需要下载 AAPL 股票的 CSV 文件（来自雅虎财经），该文件由以下链接（1990 年 1 月 1 日至 2002 年 1 月 1 日）给出，因为这是我们将测试策略的股票：

<http://ichart.finance.yahoo.com/table.csv?s=AAPL&a=00&b=1&c=1990&d=00&e=1&f=2002&g=d&ignore=.csv>

确保将此文件放在 `csv_dir` 里 `main` 函数指向的路径。 `__main__` 函数只是实例化一个新的 `backtest` 对象，然后在其上调用 `simulate_trading` 方法来执行它：

```
# mac.py

if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['AAPL']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(1990, 1, 1, 0, 0, 0)

    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandler, SimulatedExecutionHandler,
        Portfolio, MovingAverageCrossStrategy
    )
    backtest.simulate_trading()
```

要运行代码，请确保已经设置了 Python 环境（如前面章节中所述），然后命令行打开存储代码的目录。然后你应该运行：

```
python mac.py
```

您将看到以下列表（由于 `bar` 打印输出而被截断）：

```

..
..
3029
3030
Creating summary stats...
Creating equity curve...

```

	AAPL	cash	commission	total	returns	equity_curve	drawdown
datetime							
2001-12-18	0	99211	13	99211	0	0.99211	0.025383
2001-12-19	0	99211	13	99211	0	0.99211	0.025383
2001-12-20	0	99211	13	99211	0	0.99211	0.025383
2001-12-21	0	99211	13	99211	0	0.99211	0.025383
2001-12-24	0	99211	13	99211	0	0.99211	0.025383
2001-12-26	0	99211	13	99211	0	0.99211	0.025383
2001-12-27	0	99211	13	99211	0	0.99211	0.025383
2001-12-28	0	99211	13	99211	0	0.99211	0.025383
2001-12-31	0	99211	13	99211	0	0.99211	0.025383
2001-12-31	0	99211	13	99211	0	0.99211	0.025383

```

[('Total Return', '-0.79%'),
 ('Sharpe Ratio', '-0.09'),
 ('Max Drawdown', '2.56%'),
 ('Drawdown Duration', '2312')]
Signals: 10
Orders: 10
Fills: 10

```

该策略的绩效如图 15.1 所示

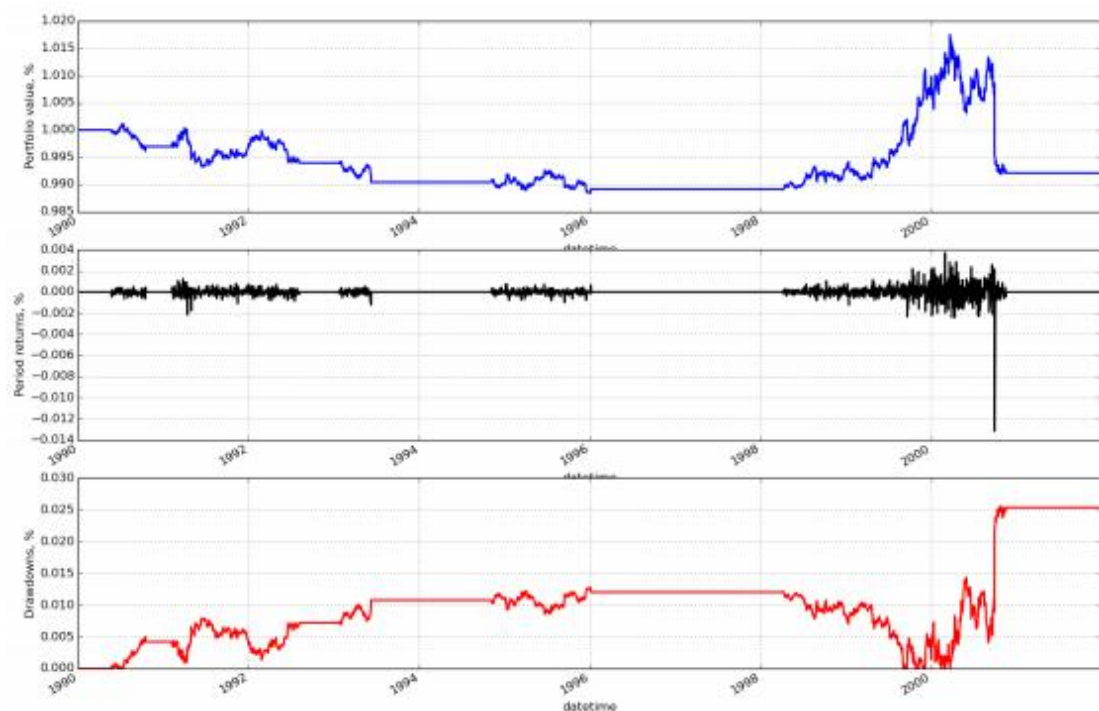


图 15.1: 均线交叉策略的净值曲线, 每日收益和回撤

显然，对于这一特定的技术指标，AAPL 股票的回报和夏普比率并不是很好！显然，我们在下一组策略中有一些工作要做，以找到一个可以产生积极表现的策略。

15.2 标准普尔 500 指数交易预测

在这个部分中，我们将考虑围绕前面章节中讨论的预测引擎构建的交易策略。我们将尝试用机器学习预测股票市场。

我们将尝试预测 SPY，这是跟踪标准普尔 500 指数价值的 ETF。最后，我们想回答一个问题，即使用基于历史价格数据的预测算法（具有轻微的预测性能）是否为我们提供了超越买入持有策略的更好的绩效回报。

该策略的规划如下：

- 1.将预测模型拟合到 S&P500 数据的集合。使用 Logistic 回归，判别分析器，支持向量机或随机森林。预测章节中概述了执行此操作的过程。
- 2.使用调整后的收盘回报数据的两个历史数据作为明天回报的预测因子。如果预测收益为正，那就买入。如果预测收益为负，则平仓。我们依然不打算考虑这种策略的卖空。

Implementation

对于此策略，我们将创建 `snp_forecast.py` 文件并导入以下必需的库：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# snp_forecast.py

from __future__ import print_function

import datetime

import pandas as pd
from sklearn.qda import QDA

from strategy import Strategy
from event import SignalEvent
from backtest import Backtest
from data import HistoricCSVDataHandler
from execution import SimulatedExecutionHandler
from portfolio import Portfolio
from create_lagged_series import create_lagged_series
```

我们导入了 Pandas 和 Scikit-Learn，以便执行监督分类器模型的拟合程序。我们还从事件驱动的回测系统中导入了必要的类。最后，我们导入了 `create_lagged_series` 函数，我们在预测章节中使用了该函数。

下一步是将 `SPYDailyForecastStrategy` 创建为 `Strategy` 抽象基类的子类。由于我们将策略的参数直接“hardcode”到类中，为简单起见，`__init__` 构造函数所需的唯一参数是 `barsdata` 处理程序和事件队列。

我们将 `self.model` 设置为 `datetime` 对象，然后设置 `self.long_market = False`。最后，我们将 `self.model` 设置为来自以下 `create_symbol_forecast_model` 的训练模型：

```
# snp_forecast.py

class SPYDailyForecastStrategy(Strategy):
    """
```



```

S&P500 forecast strategy. It uses a Quadratic Discriminant
Analyser to predict the returns for a subsequent time
period and then generated long/exit signals based on the
prediction.
"""
def __init__(self, bars, events):
    self.bars = bars
    self.symbol_list = self.bars.symbol_list
    self.events = events
    self.datetime_now = datetime.datetime.utcnow()

    self.model_start_date = datetime.datetime(2001,1,10)
    self.model_end_date = datetime.datetime(2005,12,31)
    self.model_start_test_date = datetime.datetime(2005,1,1)

    self.long_market = False
    self.short_market = False
    self.bar_index = 0

    self.model = self.create_symbol_forecast_model()

```

这里我们定义 `create_symbol_forecast_model`。它本质上调用 `create_lagged_series` 函数，该函数生成一个 Pandas DataFrame，每天得到五个历史数据返回值。然后我们只考虑两个最近的历史数据。因为越早的历史数据的预测能力是越小的（译者按：信息熵在扩散越早越无序）。

在这个阶段，我们创建训练和测试数据组。我选择不输出测试数据，因为我们之前已经在预测章节中训练了模型。最后，我们将训练数据拟合到二次判别分析器，然后返回模型。

请注意，我们可以使用随机森林，支持向量机或 Logistic 回归轻松替换模型。我们需要做的就是从 ScikitLearn 导入正确的库，只需替换 `model = QDA()` 行：

```
# snp_forecast.py

def create_symbol_forecast_model(self):
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        self.symbol_list[0], self.model_start_date,
        self.model_end_date, lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1", "Lag2"]]
    y = snpret["Direction"]

    # Create training and test sets
    start_test = self.model_start_test_date
    X_train = X[X.index < start_test]
    X_test = X[X.index >= start_test]
    y_train = y[y.index < start_test]
    y_test = y[y.index >= start_test]

    model = QDA()
    model.fit(X_train, y_train)
    return model
```

在此阶段，我们准备覆盖 Strategy 类的 calculate_signals。我们首先计算一些进入 SignalEvent 的便捷参数，然后只有在我们收到 MarketEvent 对象（基本的健全性检查）时才生成一组信号。

我们等待五个 bar 已经过去，然后获得历史数据的返回值。然后我们将这些值包装在 Pandas 中，以便模型的预测方法能够正常运行。然后我们计算一个预测，它表现为+1 或-1。

我们创建了一个 SignalEvent，让类知道我们要在市场上做多还是做空，通过+1 或-1 来进行判断做多做空


```

# snp_forecast.py

def calculate_signals(self, event):
    """
    Calculate the SignalEvents based on market data.
    """
    sym = self.symbol_list[0]
    dt = self.datetime_now

    if event.type == 'MARKET':
        self.bar_index += 1
        if self.bar_index > 5:
            lags = self.bars.get_latest_bars_values(
                self.symbol_list[0], "returns", N=3
            )
            pred_series = pd.Series(
                {
                    'Lag1': lags[1]*100.0,
                    'Lag2': lags[2]*100.0
                }
            )
            pred = self.model.predict(pred_series)
            if pred > 0 and not self.long_market:
                self.long_market = True
                signal = SignalEvent(1, sym, dt, 'LONG', 1.0)
                self.events.put(signal)

            if pred < 0 and self.long_market:
                self.long_market = False
                signal = SignalEvent(1, sym, dt, 'EXIT', 1.0)
                self.events.put(signal)

```

要运行该策略，您需要从 Yahoo Finance for SPY 下载 CSV 文件并将其放在合适的目录中（请注意，您需要在下面更改路径！）。然后我们通过 Backtest 类包装回测系统，并通过调用 simulate_trading 进行测试：

```

# snp_forecast.py

if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['SPY']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(2006,1,3)

    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandler, SimulatedExecutionHandler,
        Portfolio, SPYDailyForecastStrategy
    )
    backtest.simulate_trading()

```

扣除交易成本后该策略的输出如下：

```
..
..
2209
2210
Creating summary stats...
Creating equity curve...
      SPY      cash  commission      total  returns  equity_curve \
datetime
2014-09-29  19754   90563.3       349.7  110317.3 -0.000326    1.103173
2014-09-30  19702   90563.3       349.7  110265.3 -0.000471    1.102653
2014-10-01  19435   90563.3       349.7  109998.3 -0.002421    1.099983
2014-10-02  19438   90563.3       349.7  110001.3  0.000027    1.100013
2014-10-03  19652   90563.3       349.7  110215.3  0.001945    1.102153
2014-10-06  19629   90563.3       349.7  110192.3 -0.000209    1.101923
2014-10-07  19326   90563.3       349.7  109889.3 -0.002750    1.098893
2014-10-08  19664   90563.3       349.7  110227.3  0.003076    1.102273
2014-10-09  19274   90563.3       349.7  109837.3 -0.003538    1.098373
2014-10-09      0  109836.0       351.0  109836.0 -0.000012    1.098360

      drawdown
datetime
2014-09-29  0.003340
2014-09-30  0.003860
2014-10-01  0.006530
2014-10-02  0.006500
2014-10-03  0.004360
2014-10-06  0.004590
2014-10-07  0.007620
2014-10-08  0.004240
2014-10-09  0.008140
2014-10-09  0.008153
[('Total Return', '9.84%'),
 ('Sharpe Ratio', '0.54'),
 ('Max Drawdown', '5.99%'),
 ('Drawdown Duration', '811')]
Signals: 270
Orders: 270
Fills: 270
```

图 15.2 中的以下可视化显示了净值曲线，每日回报和回撤：

请立即注意绩效不是很好！ 夏普比率<1，但回撤幅度比较合理不到 6%。 事实证明，如果我们在这段时间内只是简单地购买并持有 SPY，表现和策略相似。

因此，一旦包含交易成本，我们实际上并未从预测策略中获得太多收益。 我特别想用这个例子，因为它使用了策略“端到端”的实现，

其中考虑了实际的交易成本。可以看出，在日间数据上做出预测预测并不容易产生良好的绩效！

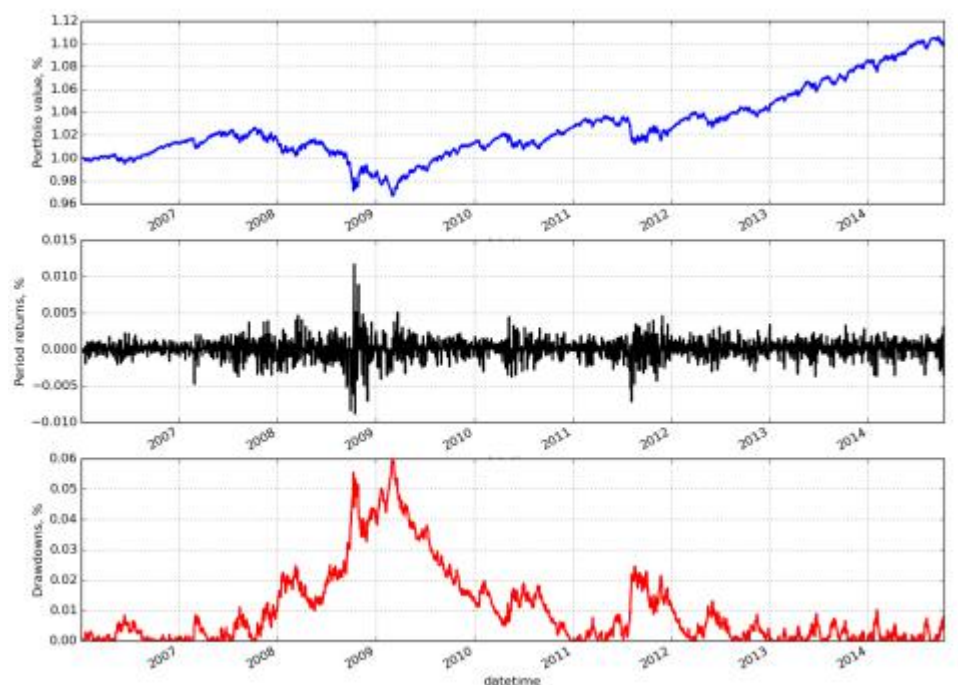


图 15.2: SPY 预测策略的净值曲线，每日收益和回撤

我们的最终策略将利用其他时间序列和更高的频率。我们将看到在修改算法的某些方面后，绩效可以得到显著改善。

15.3 均值回归配对交易

为了寻求更高的夏普比率，我们需要考虑更高频率的策略。

第一个主要问题是获取数据明显不那么简单，因为高质量的日内数据通常不是免费的。如上所述，我使用 DTN IQFeed 的日内分钟行情，因此您需要自己的 DTN 帐户才能获得此策略所需的数据。

第二个问题是回测需要更长的时间，尤其是我们在此构建的事件驱动模型。一旦我们开始考虑对跨越多年的微小数据的多样化投资组合进行回测，然后执行任何参数优化，我们就会很快发现到在 PC 上进行

回测可能需要数小时甚至数天。这需要考虑到您的研究时间。

第三个问题是现在，由于我们用更高频率的交易，执行将需要完全自动化。这意味着此类执行环境和代码必须高度可靠且无错误，否则可能发生重大损失。该策略扩展了上面的上一个日间策略，以利用日内数据。

特别是我们将使用细粒度更高的 **OHLCV bar**，而不是日 **OHLCV bar**。

该策略的规划很简单：

- 1.确定具有残差时间序列的一对股票，其已被统计识别为均值回归。

我发现了两个能源部门的美国股票，其中包括 **STX** 和 **WLL**。

- 2.通过普通最小二乘法（**OLS**）对特定窗口执行滚动线性回归，创建对的时间序列。该窗口期是要优化的参数。

- 3.创建相同回顾期的残差时间序列的滚动 **z-score**，并使用它来确定交易信号的买入/卖出阈值。

- 4.4.如果超过上限阈值时无仓位是开仓进入市场（多空或多空取决于阈过量的方向）。如果在市场上超过下限阈值，平仓。上阈值和下阈值也是要优化的参数。

实际上，我们可以使用 **Cointegrated Augmented Dickey-Fuller (CADF)** 测试来确定更准确的对冲参数。这是该策略的扩展部分。

Implementation

与往常一样，首先是导入必要的库。 我们需要调用 **rolling_apply** 的 **pandas** 库，该方法用于在滚动的基础上应用窗口的 **z-score** 计算。 我们导入 **statsmodels**，因为它提供了一种计算线性回归的普通最小二乘

（OLS）算法的函数，这是获得对冲构造的多空比所必需的。

我们还需要稍加修改的 `DataHandler` 和 `Portfolio`，以便在 `DTN IQFeed` 数据上进行细粒度更高的交易。为了创建这些文件，您只需将 `portfolio.py` 和 `data.py` 中的所有代码分别复制到新文件 `hft_portfolio.py` 和 `hft_data.py` 中，然后修改必要的部分，我将在下面演示。

以下是 `intraday_mr.py` 的导入库：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# intraday_mr.py

from __future__ import print_function

import datetime

import numpy as np
import pandas as pd
import statsmodels.api as sm

from strategy import Strategy
from event import SignalEvent
from backtest import Backtest
from hft_data import HistoricCSVDataHandlerHFT
from hft_portfolio import PortfolioHFT
from execution import SimulatedExecutionHandler
```

在下面的代码片段中，我们创建了从 `Strategy` 抽象基类派生的 `IntradayOLSMRStrategy` 类。构造函数 `__init__` 访问历史数据 `bar` 提供程序，`events queue`，`zscore_low` 阈值和 `zscore_high` 阈值，用于确定两对之间的价差序列何时是均值序列。

此外，我们指定 `OLS` 窗口（此处设置为 100），这是一个可进行优化的参数。在模拟开始时，我们既不多也不空，所以我们将 `self.long_market` 和 `self.short_market` 设置为等于 `False`：


```
# intraday_mr.py
```

```
class IntradayOLSMRStrategy(Strategy):
    """
    Uses ordinary least squares (OLS) to perform a rolling linear
    regression to determine the hedge ratio between a pair of equities.
    The z-score of the residuals time series is then calculated in a
    rolling fashion and if it exceeds an interval of thresholds
    (defaulting to [0.5, 3.0]) then a long/short signal pair are generated
    (for the high threshold) or an exit signal pair are generated (for the
    low threshold).
    """

    def __init__(
        self, bars, events, ols_window=100,
        zscore_low=0.5, zscore_high=3.0
    ):
        """
        Initialises the stat arb strategy.

        Parameters:
        bars - The DataHandler object that provides bar information
        events - The Event Queue object.
        """
        self.bars = bars
        self.symbol_list = self.bars.symbol_list
        self.events = events
        self.ols_window = ols_window
        self.zscore_low = zscore_low
        self.zscore_high = zscore_high

        self.pair = ('AREX', 'WLL')
        self.datetime = datetime.datetime.utcnow()

        self.long_market = False
        self.short_market = False
```

以下方法 `calculate_xy_signals` 获取当前 `zscore`（来自下面执行的滚动计算）并确定是否需要生成新的交易信号。

然后返回这些信号。

我们可能感兴趣的有四种潜在状态。它们是：

- 1 上涨市场，低于负 `zscore` 阈值
2. 上涨市场，`zscore` 的绝对值为下限（较低）阈值
3. 下跌市场，超过正 `zscore` 阈值
4. 下跌市场，`zscore` 的绝对值为下限（较低）阈值

在任何一种情况下，都必须产生两个信号，一个用于该对的第一个分量（AREX），另一个用于该对的第二个分量（WLL）。如果未达到这些条件，则返回一对 None 值：

```
# intraday_mr.py
```

```
def calculate_xy_signals(self, zscore_last):  
    """  
    Calculates the actual x, y signal pairings  
    to be sent to the signal generator.
```

```
Parameters
```

```
zscore_last - The current zscore to test against  
"""
```

```
y_signal = None  
x_signal = None  
p0 = self.pair[0]  
p1 = self.pair[1]  
dt = self.datetime  
hr = abs(self.hedge_ratio)
```

```
# If we're long the market and below the  
# negative of the high zscore threshold  
if zscore_last <= -self.zscore_high and not self.long_market:  
    self.long_market = True  
    y_signal = SignalEvent(1, p0, dt, 'LONG', 1.0)  
    x_signal = SignalEvent(1, p1, dt, 'SHORT', hr)  
  
# If we're long the market and between the  
# absolute value of the low zscore threshold  
if abs(zscore_last) <= self.zscore_low and self.long_market:  
    self.long_market = False  
    y_signal = SignalEvent(1, p0, dt, 'EXIT', 1.0)  
    x_signal = SignalEvent(1, p1, dt, 'EXIT', 1.0)  
  
# If we're short the market and above  
# the high zscore threshold  
if zscore_last >= self.zscore_high and not self.short_market:  
    self.short_market = True  
    y_signal = SignalEvent(1, p0, dt, 'SHORT', 1.0)  
    x_signal = SignalEvent(1, p1, dt, 'LONG', hr)  
  
# If we're short the market and between the  
# absolute value of the low zscore threshold  
if abs(zscore_last) <= self.zscore_low and self.short_market:  
    self.short_market = False  
    y_signal = SignalEvent(1, p0, dt, 'EXIT', 1.0)  
    x_signal = SignalEvent(1, p1, dt, 'EXIT', 1.0)  
  
return y_signal, x_signal
```


以下方法 `calculate_signals_for_pairs` 获得该对的每个组成部分的最新 bar（100 条）并使用它们构建基于普通最小二乘的线性回归。 识别构造差价时间序列所必需的多空比率。

一旦构建了多空比率，就构建了一系列差价序列。 下一步是通过减去其平均值并除以回测期间的标准差来计算差价序列产生的最新 zscore。

最后，`y_signal` 和 `x_signal` 是根据此 zscore 计算的。 如果信号不是 `None`，则 `SignalEvent` 实例将被发送回事件队列：

```
# intraday_mr.py

def calculate_signals_for_pairs(self):
    """
    Generates a new set of signals based on the mean reversion
    strategy.

    Calculates the hedge ratio between the pair of tickers.
    We use OLS for this, although we should ideall use CADF.
    """
    # Obtain the latest window of values for each
    # component of the pair of tickers
    y = self.bars.get_latest_bars_values(
        self.pair[0], "close", N=self.ols_window
    )
    x = self.bars.get_latest_bars_values(
        self.pair[1], "close", N=self.ols_window
    )

    if y is not None and x is not None:
        # Check that all window periods are available
        if len(y) >= self.ols_window and len(x) >= self.ols_window:
            # Calculate the current hedge ratio using OLS
            self.hedge_ratio = sm.OLS(y, x).fit().params[0]

            # Calculate the current z-score of the residuals
            spread = y - self.hedge_ratio * x
            zscore_last = ((spread - spread.mean())/spread.std())[-1]

            # Calculate signals and add to events queue
            y_signal, x_signal = self.calculate_xy_signals(zscore_last)
            if y_signal is not None and x_signal is not None:
                self.events.put(y_signal)
                self.events.put(x_signal)
```

最后 `calculate_signals` 被覆盖，用于检查队列中接收的事件是否实际上是 `MarketEvent`，在这种情况下执行新信号的计算：

```
# intraday_mr.py

def calculate_signals(self, event):
    """
    Calculate the SignalEvents based on market data.
    """
    if event.type == 'MARKET':
        self.calculate_signals_for_pairs()
```

`__main__` 部分将所有部分连接在一起，以生成策略的回测。我们告诉模拟存储自动收报机 `tick` 的位置。因为使用 `DTN IQFeed` 格式，我截断了两个文件，以便它们在相同的分钟开始和结束。这对特殊的 `AREX` 和 `WLL`，开始日期交易是 2007 年 11 月 8 日上午 10:41:00。

最后，我们构建了 `backtest` 对象并开始模拟交易：

```
# intraday_mr.py

if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['AREX', 'WLL']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(2007, 11, 8, 10, 41, 0)

    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,

        start_date, HistoricCSVDataHandlerHFT, SimulatedExecutionHandler,
        PortfolioHFT, IntradayOLSMRStrategy
    )
    backtest.simulate_trading()
```

但是，在我们执行此文件之前，我们需要对数据处理程序和项目组合对象进行一些修改。特别是，有必要创建新文件 `hft_data.py` 和 `hft_portfolio.py`，它们分别是 `data.py` 和 `portfolio.py` 的副本。在 `hft_data.py` 中，我们需要将 `HistoricCSVDataHandler` 重命名为 `HistoricCSVDataHandlerHFT` 并替换 `_open_convert_csv_files` 方法中的名

称列表。

```
names=[
    'datetime', 'open', 'high',
    'low', 'close', 'volume', 'adj_close'
]
```

必须替换为:

```
names=[
    'datetime', 'open', 'low',
    'high', 'close', 'volume', 'oi'
]
```

这是为了确保 DTN IQFeed 的格式能与回测一起使用。

另一个变化是在 `hft_portfolio.py` 中将 `Portfolio` 重命名为 `PortfolioHFT`。

然后我们必须修改几行以便实现 DTN 数据的更高频率。

特别是，在 `update_timeindex` 方法中，我们必须更改以下代码：

```
for s in self.symbol_list:
    # Approximation to the real value
    market_value = self.current_positions[s] * \
        self.bars.get_latest_bar_value(s, "adj_close")
    dh[s] = market_value
    dh['total'] += market_value
```

To:

```
for s in self.symbol_list:
    # Approximation to the real value
    market_value = self.current_positions[s] * \
        self.bars.get_latest_bar_value(s, "close")
    dh[s] = market_value
    dh['total'] += market_value
```

这确保我们获得收盘价，而不是 `adj_close` 价格。 后者适用于雅虎财经，而前者适用于 DTN IQFeed。

我们还必须在 `update_holdings_from_fill` 中进行类似的调整。 我们需要更改以下代码：

```
# Update holdings list with new quantities
fill_cost = self.bars.get_latest_bar_value(
    fill.symbol, "adj_close"
)
```

To:

```
# Update holdings list with new quantities
fill_cost = self.bars.get_latest_bar_value(
    fill.symbol, "close"
)
```

最后的更改发生在文件底部的 `output_summary_stats`。 我们需要修改夏普比率的计算方式。

```
sharpe_ratio = create_sharpe_ratio(returns)
```

Must be changed to:

```
sharpe_ratio = create_sharpe_ratio(returns, periods=252*6.5*60)
```

这样就完成了必要的更改。 执行 `intraday_mr.py` 后，我们从 `backtest` 中获得以下输出：

```
..
..
375072
375073
Creating summary stats...
Creating equity curve...

```

	AREX	WLL	cash	commission	total	returns	\
datetime							
2014-03-11 15:53:00	2098	-6802	120604.3	9721.4	115900.3	-0.000052	
2014-03-11 15:54:00	2101	-6799	120604.3	9721.4	115906.3	0.000052	
2014-03-11 15:55:00	2100	-6802	120604.3	9721.4	115902.3	-0.000035	
2014-03-11 15:56:00	2097	-6810	120604.3	9721.4	115891.3	-0.000095	
2014-03-11 15:57:00	2098	-6801	120604.3	9721.4	115901.3	0.000086	
2014-03-11 15:58:00	2098	-6800	120604.3	9721.4	115902.3	0.000009	
2014-03-11 15:59:00	2099	-6800	120604.3	9721.4	115903.3	0.000009	
2014-03-11 16:00:00	2100	-6801	120604.3	9721.4	115903.3	0.000000	
2014-03-11 16:01:00	2100	-6801	120604.3	9721.4	115903.3	0.000000	
2014-03-11 16:01:00	2100	-6801	120604.3	9721.4	115903.3	0.000000	

datetime	equity_curve	drawdown
2014-03-11 15:53:00	1.159003	0.003933
2014-03-11 15:54:00	1.159063	0.003873
2014-03-11 15:55:00	1.159023	0.003913
2014-03-11 15:56:00	1.158913	0.004023
2014-03-11 15:57:00	1.159013	0.003923
2014-03-11 15:58:00	1.159023	0.003913
2014-03-11 15:59:00	1.159033	0.003903
2014-03-11 16:00:00	1.159033	0.003903
2014-03-11 16:01:00	1.159033	0.003903
2014-03-11 16:01:00	1.159033	0.003903
[('Total Return', '15.90%'),		
('Sharpe Ratio', '1.89'),		
('Max Drawdown', '3.03%'),		
('Drawdown Duration', '120718')]		
Signals: 7594		
Orders: 7478		
Fills: 7478		

您可以看到在此期间该策略表现良好。虽然它的总回报率不到 16%。夏普比率是合理的（与典型的每日策略相比），但考虑到策略的高频特性，我们应该期待更多。 该策略的主要吸引力在于最大回撤较低（约 3%）。

这表明我们可以应用更多杠杆来获得更多回报。

该策略的绩效如图 15.3 所示：

请注意，这些数字基于交易总共 100 股。 您可以通过简单地调整 Portfolio 类的 `generate_naive_order` 方法来调整杠杆。

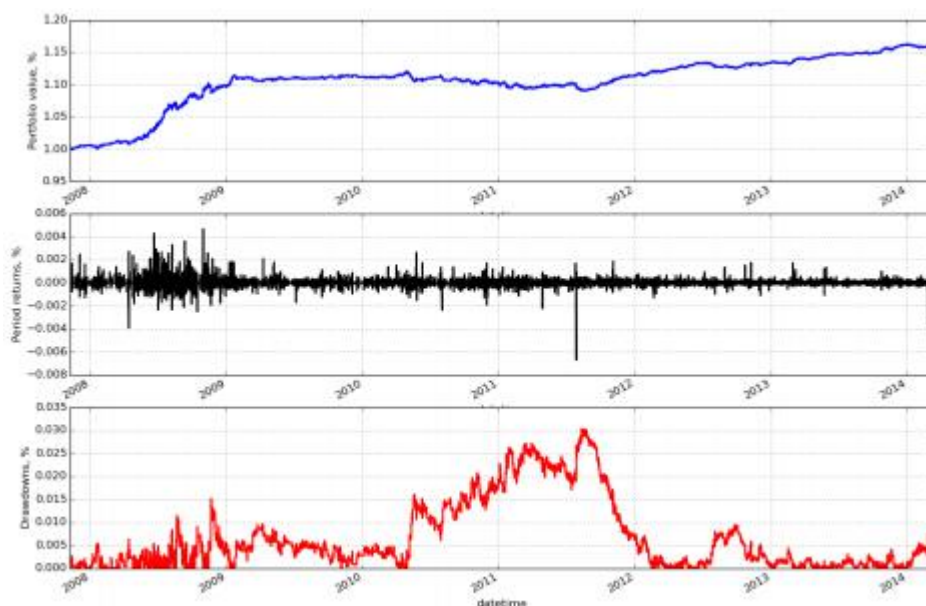


图 15.3: 日内均值回归策略的净值曲线, 每日收益和回撤

mkt_quantity 将其更改为 2000 得到以下结果:

```
..  
..  
[('Total Return', '392.85%'),  
 ('Sharpe Ratio', '2.29'),  
 ('Max Drawdown', '45.69%'),  
 ('Drawdown Duration', '102150')]  
..  
..
```

显然, 夏普比率和总回报更具吸引力, 但在此期间我们必须承受 45% 的最大亏损!

15.4 绘制绩效图

上面显示的两个数字都是使用 plot_performance.py 脚本创建的。为了完整性, 我已经写了代码, 以便您可以将其用作创建自己的绩效图的基础。

必须与 backtest 的输出文件相同的目录中运行它, 即 equity.csv 所在的位置。

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
```

```
# plot_performance.py
```

```
import os.path
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
if __name__ == "__main__":
```

```
    data = pd.io.parsers.read_csv(
        "equity.csv", header=0,
        parse_dates=True, index_col=0
    ).sort()
```

```
    # Plot three charts: Equity curve,
```

```
    # period returns, drawdowns
```

```
    fig = plt.figure()
```

```
    # Set the outer colour to white
```

```
    fig.patch.set_facecolor('white')
```

```
    # Plot the equity curve
```

```
    ax1 = fig.add_subplot(311, ylabel='Portfolio value, %')
```

```
    data['equity_curve'].plot(ax=ax1, color="blue", lw=2.)
```

```
    plt.grid(True)
```

```
    # Plot the returns
```

```
    ax2 = fig.add_subplot(312, ylabel='Period returns, %')
```

```
    data['returns'].plot(ax=ax2, color="black", lw=2.)
```

```
    plt.grid(True)
```

```
    # Plot the returns
```

```
    ax3 = fig.add_subplot(313, ylabel='Drawdowns, %')
```

```
    data['drawdown'].plot(ax=ax3, color="red", lw=2.)
```

```
    plt.grid(True)
```

```
    # Plot the figure
```

```
    plt.show()
```


第 16 章

策略优化

在前面的章节中，我们已经考虑了如何创建基础预测模型以及基于它的交易策略。在此过程中，我们已经看到这些模型有很多参数。在 **SVM** 的情况下，我们具有“调整”参数 γ 和 **C**。在均线交叉交易策略中，我们能调整均值时间的两个窗口的参数。

在本章中，我们将描述优化方法，通过以系统方式调整参数来改善交易策略的绩效。为此，我们将使用统计模型中的一些方法，例如交叉验证和网格搜索。

关于模型选择和参数优化的文献很多，大多数方法都超出了本书的范围。我想在这里介绍这个主题，以便您可以按照自己的情况探索更复杂的技术。

16.1 参数优化

在这个阶段，几乎所有的交易策略和统计模型都需要一个或多个参数才能被使用。在使用技术指标的动量策略中，例如移动平均线，需要指定一个指标。许多均值回归策略也是如此，它需要一个（滚动）指标来计算两个时间序列之间的回归。特定的统计机器学习模型，例如逻辑回归，**SVM** 或随机森林也需要参数才能被计算。

考虑参数优化时最大的危险是过度拟合。当模型在样本内的训练数据上训练并且被优化以良好地执行时会发生此问题，但是当应用于样本外时性能显著降低。例如，交易策略在回测中可以表现得非常好（在样本内），但是当部署用于实时交易时，可能完全无利可图。

参数优化的另一个问题是它可能变得非常昂贵。使用现代计算系统，

由于并行化和高频 CPU，这不像以前那样成为问题。然而，多参数优化会数量级的增加计算复杂度。我们必须意识到这是研究和开发过程的一部分。

16.1.1 要优化哪些参数？

基于统计的算法交易模型通常具有许多参数和不同的绩效度量。基础统计学习算法有其自己的一组参数。在多线性或逻辑回归的情况下，这些将是 β_i 系数。在随机森林的情况下，一个这样的参数将是在集合中使用的基础决策树的数量。应用于交易模型的参数可以是买入和买入阈值，例如特定时间序列的 **z-score**。**z-score** 本身有一个隐含的滚动窗口。可以看出，参数的数量可以非常多。

除了参数之外，还有许多方法可以评估统计模型的性能以及基于它的交易策略。我们已经定义了诸如准确度和混淆矩阵之类的概念。此外，还有更多的统计测量，如均方误差（**MSE**）。这些是通过与其域相关的参数在统计模型内优化的性能。

实际交易策略根据不同的标准进行评估，例如复合年增长率（**CAGR**）和最大回撤。我们需要改变买卖和平仓标准，以及与统计模型没有直接关系的其他阈值。因此，这引发了关于要优化哪组参数以及何时优化的问题。

在接下来的部分中，我们将在早期研究和开发阶段优化统计模型参数，以及优化统计模型的交易策略相关参数，对每个绩效指标进行优化。

16.1.2 优化是困难的

利用多个实值参数，优化速度变得非常堪忧，因为每个新参数增加了额外的空间维度。如果我们考虑网格搜索方法（将在下面详细讨论），并且具有单个参数 α ，那么我们可能希望在集合 $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ 内改变 α 。这需要 5 次模拟。

如果我们现在考虑一个附加参数 β ，它可能在 $\{0.2, 0.4, 0.6, 0.8, 1.0\}$ 范围内变化，那么我们将不得不考虑 $5^2 = 25$ 次模拟。另一个参数 γ ，有 5 个值，使其达到 $5^3 = 125$ 次模拟。如果每个参数具有 10 个待测试的值，则将进行 $10^3 = 1000$ 次模拟。可以看出，这种参数搜索模拟非常麻烦。

很明显，在进行穷举参数搜索和维持合理的总模拟时间之间存在折衷。虽然并行性（包括多核 CPU 和图形处理单元（GPU））在某种程度上缓解了这个问题的，但在引入参数时我们仍需要小心。

16.1.3 过度拟合

过度拟合是针对特定数据集优化参数或参数集的过程，使得发现适合的绩效（或不好的绩效）被最大化（或最小化），但是当应用于样本外的数据集时，绩效指标可能大幅下降。该概念与偏差-方差困境的概念密切相关。

偏差 - 方差困境涉及统计模型在作为低偏差模型或低方差模型或两者之间的折衷之间进行权衡的情况。偏差是指模型对参数的估计与参数的真实“总体”值之间的差异，或统计模型中的错误假设。方差是指模型对于训练集中小波动的误差灵敏度（在样本数据内）。

在所有统计模型中，人们同时试图最小化偏差误差和方差误差，

以便提高模型精度。这种情况可能导致模型过度拟合，因为通过引入具有更大灵活性的模型可以大大减少训练误差。然而，这些模型在新（样本外）数据上的表现极差，因为它们基本上“适合”样本内数据。

高偏差，低方差模型的一个常见例子是应用于非线性数据集的线性回归。新点的添加不会显著影响回归斜率（假设它们与其他数据不太远），但由于问题本质上是非线性的，因此使用线性模型在结果中存在系统性偏差。

低偏差，高方差模型的一个常见示例是应用于非线性数据集的多项式拟合。可以调整模型的参数（多项式的次数）以非常精确地拟合这样的模型（即，对训练数据的低偏差），但是新点的增加几乎肯定会导致模型必须修改其多项式的拟合以适应新数据。这将使其成为样本数据中有非常高的方差。这样的模型可能具有非常差的可预测性或对样本外数据的推断能力。

过度拟合也可以在交易策略上体现出来，而不仅仅是统计模型。例如，我们可以通过改变多空和平仓阈值参数来优化夏普比率。虽然这可以提高回测中的盈利能力（或者实质上最小化风险），但是当策略实时部署时，它不是简单复制，因为我们可能在历史数据中已经对噪声进行了这样的优化。

我们将讨论以下技术，以尽可能减少过度拟合。然而，人们必须意识到，在算法交易和统计分析中，它总是存在的。

16.2 模型选择

在本节中，我们将考虑如何优化将作为交易策略的统计模型。在

统计学和机器学习领域，这被称为模型选择。虽然我不会对各种模型选择技术进行详尽的讨论，但我将介绍一些适用于交易策略基本机制，如交叉验证和网格搜索。

16.2.1 交叉验证

交叉验证是一种用于评估统计模型如何推演到之前未曾暴露过的新数据的方法。这种方法通常用于预测模型，例如用于预测资产价格序列次日回报的监督学习。从根本上说，交叉验证的目标是最小化样本数据之外的错误，而不会导致过度拟合模型。

在本节中，我们将描述训练/测试拆分和 **k-fold** 交叉验证，以及 **Scikit-Learn** 中的使用技术，以便在我们已经开发的统计模型上自动执行这些程序。

训练/测试拆分

最简单的交叉验证示例称为训练/测试拆分或 **2 分交叉验证**。一旦有先前的历史数据集（例如资产价格的每日时间序列），它就分成两个部分。拆分的比例通常在 **0.5** 和 **0.8** 之间变化。在后一种情况下，这意味着 **80%** 的数据用于培训，**20%** 用于测试。所有感兴趣的统计数据，例如准确度，混淆矩阵或均方误差都是在测试集上计算的，该测试集不会在训练过程中使用。

要使用 **Scikit-Learn** 在 **Python** 中执行此过程，我们可以使用 **sklearn.cross_validation.train_test_split** 方法。我们将继续使用我们的模型，如预测章节中所讨论的那样。特别是，我们将修改 **forecast.py** 并创建一个名为 **train_test_split.py** 的新文件。我们需要将导入添加到导

入列表中：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# train_test_split.py

from __future__ import print_function

import datetime

import sklearn

from sklearn.cross_validation import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.lda import LDA
from sklearn.metrics import confusion_matrix
from sklearn.qda import QDA
from sklearn.svm import LinearSVC, SVC

from create_lagged_series import create_lagged_series
```

在 `forecast.py` 中，我们根据时间序列中的时间进行拆分数据：

```
# forecast.py

..

# The test data is split into two parts: Before and after 1st Jan 2005.
start_test = datetime.datetime(2005,1,1)

# Create training and test sets
X_train = X[X.index < start_test]
X_test = X[X.index >= start_test]
y_train = y[y.index < start_test]
y_test = y[y.index >= start_test]

..
```



```

# train_test_split.py

if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.8, random_state=42
    )

    # Create the (parametrised) models
    print("Hit Rates/Confusion Matrices:\n")
    models = [("LR", LogisticRegression()),
               ("LDA", LDA()),
               ("QDA", QDA()),
               ("LSVC", LinearSVC()),
               ("RSVM", SVC(
                   C=1000000.0, cache_size=200, class_weight=None,
                   coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
                   max_iter=-1, probability=False, random_state=None,
                   shrinking=True, tol=0.001, verbose=False
               )),
               ("RF", RandomForestClassifier(
                   n_estimators=1000, criterion='gini',
                   max_depth=None, min_samples_split=2,
                   min_samples_leaf=1, max_features='auto',
                   bootstrap=True, oob_score=False, n_jobs=1,
                   random_state=None, verbose=0
               ))]

    # Iterate through the models
    for m in models:

        # Train each of the models on the training set
        m[1].fit(X_train, y_train)

        # Make an array of predictions on the test set
        pred = m[1].predict(X_test)

        # Output the hit-rate and the confusion matrix for each model
        print("%s:\n%0.3f" % (m[0], m[1].score(X_test, y_test)))
        print("%s\n" % confusion_matrix(pred, y_test))

```

请注意，我们已选择训练集的比率为数据的 80%，测试数据为 20%。

此外，我们已经指定了 `random_state` 来随机化数据选择采样。这意味着数据不是按时间顺序按顺序划分，而是随机采样。

模型上的交叉验证结果如下（由于拟合程序的不同，您结果可能会略有不同）：

Hit Rates/Confusion Matrices:

LR:
0.511
[[70 70]
 [419 441]]

LDA:
0.513
[[69 67]
 [420 444]]

QDA:
0.503
[[83 91]
 [406 420]]

LSVC:
0.513
[[69 67]
 [420 444]]

RSVM:
0.506
[[8 13]
 [481 498]]

RF:
0.490
[[200 221]
 [289 290]]

可以看出，准确度大大低于上述预测章节中的确度。因此，我们可以得出结论，训练/测试拆分的特定选择导致对回测的预测能力过度乐观的看法。

下一步是增加执行交叉验证的次数，以便最大限度地减少任何潜在的过度拟合。为此，我们将使用 `k-fold` 交叉验证。

K-Cross 交叉验证

我们可以使用 **k-fold** 交叉验证将该集随机分割为 **k** 个等大小的子样本，而不是将该集合划分为单个训练和测试集。对于每次迭代（其中有 **k** 个），**k** 个子样本中的一个被保留作为测试集，而剩余的 **k-1** 个子样本一起形成训练集。然后在 **k** 个集合中上训练统计模型，并且在其特定的第 **k** 个测试集上评估其性能。

其目的是通过平均预测结果（或其他方式）将每个模型的结果组合成一个集合，以产生预测结果。使用 **k-fold** 交叉验证的主要好处是原始数据集中的每个分类器仅用于训练和测试一次。

这引发了一个关于如何选择 **k** 的问题，这产生另一个参数。通常，使用 **k = 10**，但是也可以执行其他分析以选择 **k** 的最佳值。

我们现在将使用 Scikit-Learn 的 `cross_validation` 模块来获取 **k-fold** 交叉验证对象。我们创建一个名为 `k_fold_cross_val.py` 的新文件，它是 `train_test_split.py` 的副本，并通过添加以下行：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# k_fold_cross_val.py

from __future__ import print_function

import datetime

import pandas as pd
import sklearn
from sklearn import cross_validation
from sklearn.metrics import confusion_matrix
from sklearn.svm import SVC

from create_laqqed_series import create_laqqed_series
```

然后，我们需要通过删除 `train_test_split` 并将其替换为 `K-Fold` 更改 `__main__` 函数。

第一个是数据集的长度，在这种情况下为 1250 天。第二个值是表示拆分数 `K`，我们设置为 10。第三个值是 `indices`，我将其设置为 `False`。这意味着实际的索引值用于迭代器返回的数组。第四和第五用于随机化样本的顺序。

和之前的 `forecast.py` 和 `train_test_split.py` 一样，我们获得了 S&P500 的历史序列。然后，我们创建一组预测变量 (`X`) 和响应 (`Y`)。然后我们使用 `K-Fold` 对象并迭代它。在每次迭代期间，我们为每个 `X` 和 `Y` 创建训练和测试集。然后将它们送入径向支持向量机。最后输出 `SVM` 的每个实例的准确度和混淆矩阵。

```
# k_fold_cross_val.py

if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Create a k-fold cross validation object
    kf = cross_validation.KFold(
        len(snpret), n_folds=10, indices=False,
        shuffle=True, random_state=42
    )
```

```

# Use the kf object to create index arrays that
# state which elements have been retained for training
# and which elements have been retained for testing
# for each k-element iteration
for train_index, test_index in kf:
    X_train = X.ix[X.index[train_index]]
    X_test = X.ix[X.index[test_index]]
    y_train = y.ix[y.index[train_index]]
    y_test = y.ix[y.index[test_index]]

    # In this instance only use the
    # Radial Support Vector Machine (SVM)
    print("Hit Rate/Confusion Matrix:")
    model = SVC(
        C=1000000.0, cache_size=200, class_weight=None,
        coef0=0.0, degree=3, gamma=0.0001, kernel='rbf',
        max_iter=-1, probability=False, random_state=None,
        shrinking=True, tol=0.001, verbose=False
    )

    # Train the model on the retained training data
    model.fit(X_train, y_train)

    # Make an array of predictions on the test set
    pred = model.predict(X_test)

    # Output the hit-rate and the confusion matrix for each model
    print("%.3f" % model.score(X_test, y_test))
    print("%s\n" % confusion_matrix(pred, y_test))

```

代码输出如下：

```

Hit Rate/Confusion Matrix:
0.528
[[11 10]
 [49 55]]

```



```
Hit Rate/Confusion Matrix:  
0.400  
[[ 2  5]  
 [70 48]]
```

```
Hit Rate/Confusion Matrix:  
0.528  
[[ 8  8]  
 [51 58]]
```

```
Hit Rate/Confusion Matrix:  
0.536  
[[ 6  3]  
 [55 61]]
```

```
Hit Rate/Confusion Matrix:  
0.512  
[[ 7  5]  
 [56 57]]
```

```
Hit Rate/Confusion Matrix:  
0.480  
[[11 11]  
 [54 49]]
```

```
Hit Rate/Confusion Matrix:  
0.608  
[[12 13]  
 [36 64]]
```

```
Hit Rate/Confusion Matrix:  
0.440  
[[ 8 17]  
 [53 47]]
```

```
Hit Rate/Confusion Matrix:  
0.560  
[[10  9]  
 [46 60]]
```

```
Hit Rate/Confusion Matrix:  
0.528  
[[ 9 11]  
 [48 57]]
```

很明显，命中率和混淆矩阵在不同的集合中变化很大。这表明该模型在该特定数据集上易于过度拟合。对此的补救措施是以更高的频率或更长的时间使用更多的数据。

为了在交易策略中利用该模型，有必要将这些经过单独训练的分类器（即每个 K 个对象）组合成一个整体平均值，然后将该组合模型用于

策略内的分类。

注意，从技术上讲，对时间有序的数据（即时间序列）使用简单的交叉验证技术是不合适的。 有更复杂的机制以这种方式处理自相关，但我想强调做法。

16.2.2 网格搜索

到目前为止，我们已经看到 **k-fold** 交叉验证有助于我们通过对样本的每个元素执行验证来避免数据过度拟合。我们现在将注意力转向优化特定统计模型的参数。这些参数是模型估计，程序不直接学习的参数。例如，支持向量机的 **C** 和 γ 。实质上，它们是在我们调用每个统计模型的初始化时需要指定的参数。对于此过程，我们将使用称为网格搜索的过程。

基本思想是采用一系列参数并评估范围内每个参数元素的统计模型的性能。为了在 **Scikit-Learn** 中实现这一点，我们可以创建一个 **ParameterGrid**。这样的对象将产生一个 **Python** 字典列表，每个字典包含一个参数组合，用于输入统计模型。

下面给出了一个示例代码，生成支持向量机相关的参数：

```
>>> from sklearn.grid_search import ParameterGrid
>>> param_grid = {'C': [1, 10, 100, 1000], 'gamma': [0.001, 0.0001]}
>>> list(ParameterGrid(param_grid))

[{'C': 1, 'gamma': 0.001},
 {'C': 1, 'gamma': 0.0001},
 {'C': 10, 'gamma': 0.001},
 {'C': 10, 'gamma': 0.0001},
 {'C': 100, 'gamma': 0.001},
 {'C': 100, 'gamma': 0.0001},
 {'C': 1000, 'gamma': 0.001},
 {'C': 1000, 'gamma': 0.0001}]
```


现在我们有了一种生成 `ParameterGrid` 的方法，我们需要迭代统计模型以搜索最佳绩效得分。 在这种情况下，我们将寻求最大化分类器的准确度。

Scikit-Learn 的 `GridSearchCV` 机制允许我们执行实际的网格搜索。 实际上，它允许我们不仅可以同时执行标准网格搜索，还可以执行交叉验证方案。

我们现在要创建一个新文件 `grid_search.py`，它再次使用 `create_lagged_series.py` 和支持向量机来执行交叉验证的参数网格搜索。 首先我们必须导入正确的库：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# grid_search.py

from __future__ import print_function

import datetime

import sklearn
from sklearn import cross_validation
from sklearn.cross_validation import train_test_split
from sklearn.grid_search import GridSearchCV
from sklearn.metrics import classification_report
from sklearn.svm import SVC

from create_lagged_series import create_lagged_series
```

和之前的 `k_fold_cross_val.py` 一样，我们创建了一个滞后历史序列，然后使用前两天的返回值作为预测变量。我们最初创建一个训练/测试分组，使得 50% 的数据可以用于训练和交叉验证，而剩余的数据可以“保留”用于评估。

随后我们创建了 `tuned_parameters` 列表，其中包含一个字典，表示我

们希望测试的参数。创建所有参数列表的笛卡尔积，即每个可能参数组合的列表。 创建参数列表后，我们将它传递给 `GridSearchCV` 类，以及我们感兴趣的分类器类型（即径向支持向量机），`k-fold` 交叉验证 `k` 值为 10。

最后，我们训练模型并输出最佳估计量及其相关的准确度分数。 通过这种方式，我们不仅通过交叉验证优化了模型参数，而且我们还通过参数化网格搜索优化了模型的参数，所有这些都在一个类中！ 代码允许进行大量实验而不会被过度的“data wrangling”所困扰。

```
if __name__ == "__main__":
    # Create a lagged series of the S&P500 US stock market index
    snpret = create_lagged_series(
        "^GSPC", datetime.datetime(2001,1,10),
        datetime.datetime(2005,12,31), lags=5
    )

    # Use the prior two days of returns as predictor
    # values, with direction as the response
    X = snpret[["Lag1","Lag2"]]
    y = snpret["Direction"]

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.5, random_state=42
    )

    # Set the parameters by cross-validation
    tuned_parameters = [
        {'kernel': ['rbf'], 'gamma': [1e-3, 1e-4], 'C': [1, 10, 100, 1000]}
    ]

    # Perform the grid search on the tuned parameters
    model = GridSearchCV(SVC(C=1), tuned_parameters, cv=10)
    model.fit(X_train, y_train)

    print("Optimised parameters found on training set:")
    print(model.best_estimator_, "\n")

    print("Grid scores calculated on training set:")
    for params, mean_score, scores in model.grid_scores_:
        print("%0.3f for %r" % (mean_score, params))
```

网格搜索交叉验证程序的输出如下：

```

Optimised parameters found on training set:
SVC(C=1, cache_size=200, class_weight=None, coef0=0.0, degree=3, gamma=0.001,
    kernel='rbf', max_iter=-1, probability=False, random_state=None,
    shrinking=True, tol=0.001, verbose=False)

Grid scores calculated on training set:
0.541 for {'kernel': 'rbf', 'C': 1, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 1, 'gamma': 0.0001}
0.541 for {'kernel': 'rbf', 'C': 10, 'gamma': 0.001}

0.541 for {'kernel': 'rbf', 'C': 10, 'gamma': 0.0001}
0.541 for {'kernel': 'rbf', 'C': 100, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 100, 'gamma': 0.0001}
0.538 for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.001}
0.541 for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.0001}

```

我们可以看到 $\gamma = 0.001$ 和 $C = 1$ 在验证集上为这个特定的径向核支持向量机最佳准确度。正如我们之前在前一章中的那样，这个模型现在可以构成基于预测的交易策略的基础。

16.3 优化策略

到目前为止，我们一直专注于模型选择和优化基础统计模型构成交易策略的基础。然而，预测模型各种绩效，有利可图的算法策略是两个不同的方面。我们现在将注意力转向优化对盈利能力和风险指标有直接影响的参数。

为实现这一目标，我们将使用前一章中描述的事件驱动的回测工具。将考虑具有三个与其相关的参数的特定策略，并使用网格搜索机制搜索由参数的笛卡尔积形成的空间。然后，我们将尝试最大化特定指标，例如夏普比率最大化或最小化最大回撤等。

16.3.1 日内均值回复策略

本章中我们采用的策略是使用能源股票 AREX 和 WLL 的“日内均值回归配对交易”。它包含我们能够优化的三个参数：线性回归时间，价差 z-score 开仓阈值和 z-score 分数平仓阈值。

我们将考虑每个参数的一系列值，然后计算每个范围内的策略回测，将每个模拟的总回报，夏普比率和回撤特征输出到 CSV 文件。这将使我们能够优化夏普或最小化我们的交易策略的最大回撤。

16.3.2 参数调整

由于事件驱动的回测是 CPU-intensive 的(译者按：Python 通常为单线程运行（多进程无法利用多核）)，我们将参数范围限制为每个参数三个值。这将给我们总共 $3^3 = 27$ 个单独的模拟来执行。参数范围如下：

- OLS Lookback Window - $w_l \in \{50, 100, 200\}$
- Z-Score Entry Threshold - $z_h \in \{2.0, 3.0, 4.0\}$
- Z-Score Exit Threshold - $z_l \in \{0.5, 1.0, 1.5\}$

为了执行该组仿真，将计算所有三个范围的笛卡尔积，然后对每个参数组合进行模拟。

第一项任务是修改 `intraday_mr.py` 文件：

```
# intraday_mr.py
..
from itertools import product
..
```

然后我们可以修改 `__main__`，以包括为上面讨论的所有三个参数生成参数列表。

第一项任务是为 OLS 窗口值，zscore 开仓阈值和 zscore 平仓阈值创建实际参数范围。这些中的每一个都有三个独立的变体，总共有 27 组模拟。

创建范围后，`itertools.product` 方法用于创建所有变体的笛卡尔积，然后将其输入到字典列表中，以确保将正确的关键字参数传递给 `Strategy` 对象。

最后实现回测：

```
if __name__ == "__main__":
    csv_dir = '/path/to/your/csv/file' # CHANGE THIS!
    symbol_list = ['AREX', 'WLL']
    initial_capital = 100000.0
    heartbeat = 0.0
    start_date = datetime.datetime(2007, 11, 8, 10, 41, 0)

    # Create the strategy parameter grid
    # using the itertools cartesian product generator
    strat_lookback = [50, 100, 200]
    strat_z_entry = [2.0, 3.0, 4.0]
    strat_z_exit = [0.5, 1.0, 1.5]
    strat_params_list = list(product(
        strat_lookback, strat_z_entry, strat_z_exit
    ))

    # Create a list of dictionaries with the correct
    # keyword/value pairs for the strategy parameters
    strat_params_dict_list = [
        dict(ols_window=sp[0], zscore_high=sp[1], zscore_low=sp[2])
        for sp in strat_params_list
    ]

    # Carry out the set of backtests for all parameter combinations
    backtest = Backtest(
        csv_dir, symbol_list, initial_capital, heartbeat,
        start_date, HistoricCSVDataHandlerHFT, SimulatedExecutionHandler,
        PortfolioHFT, IntradayOLSMRStrategy,
        strat_params_list=strat_params_dict_list
    )
    backtest.simulate_trading()
```

下一步是修改 `backtest.py` 中的 `Backtest` 对象，以便能够处理多个参数集。 我们需要修改 `_generate_trading_instances`，使其具有一个参数，该参数表示创建新 `Strategy` 对象时的特定参数集：

```
# backtest.py

..
def _generate_trading_instances(self, strategy_params_dict):
    """
    Generates the trading instance objects from
    their class types.
    """
    print("Creating DataHandler, Strategy, Portfolio and ExecutionHandler for")
    print("strategy parameter list: %s..." % strategy_params_dict)
    self.data_handler = self.data_handler_cls(
        self.events, self.csv_dir, self.symbol_list, self.header_strings
    )

    self.strategy = self.strategy_cls(
        self.data_handler, self.events, **strategy_params_dict
    )
    self.portfolio = self.portfolio_cls(
        self.data_handler, self.events, self.start_date,
        self.num_strats, self.periods, self.initial_capital
    )
    self.execution_handler = self.execution_handler_cls(self.events)
..
```

现在，在策略参数列表循环中调用此方法，而不是构建 **Backtest** 对象。虽然为每个参数集重新创建所有数据处理程序，事件队列和组合对象可能看起来很浪费，但它确保所有迭代器都已重置，使得我们每个模拟创建的都是一个“clean slate”。

下一个任务是修改 `simulate_trading` 方法以循环遍历策略参数。创建一个输出 **CSV** 文件，用于存储参数组合及其特定的性能指标。这将允许我们稍后在各参数之间绘制绩效特征。循环遍历所有策略参数，并在每次模拟时生成新的交易实例。然后执行回测并计算统计数据。这些存储并输出到 **CSV** 文件中。模拟结束后，输出文件将关闭：

```
# backtest.py

..
def simulate_trading(self):
    """
    Simulates the backtest and outputs portfolio performance.
    """
    out = open("output/opt.csv", "w")

    spl = len(self.strat_params_list)
    for i, sp in enumerate(self.strat_params_list):
        print("Strategy %s out of %s..." % (i+1, spl))
        self._generate_trading_instances(sp)
        self._run_backtest()
        stats = self._output_performance()
        pprint.pprint(stats)

        tot_ret = float(stats[0][1].replace("%", ""))
        cagr = float(stats[1][1].replace("%", ""))
        sharpe = float(stats[2][1])
        max_dd = float(stats[3][1].replace("%", ""))
        dd_dur = int(stats[4][1])

        out.write(
            "%s,%s,%s,%s,%s,%s,%s,%s\n" % (
                sp["ols_window"], sp["zscore_high"], sp["zscore_low"],
                tot_ret, cagr, sharpe, max_dd, dd_dur
            )
        )

    out.close()
```

在我的 PC 上，此过程需要一些时间， 每次模拟超过 600,000 个数据的 27 个参数，模拟大约需要 3 个小时。 在此阶段，回测系统没有进行并行运算，并行运行模拟将使该过程更快。 参数的输出如下：


```
50,2.0,0.5,213.96,20.19,1.63,42.55,255568
50,2.0,1.0,264.9,23.13,2.18,27.83,160319
50,2.0,1.5,167.71,17.15,1.63,60.52,293207
50,3.0,0.5,298.64,24.9,2.82,14.06,35127
50,3.0,1.0,324.0,26.14,3.61,9.81,33533
50,3.0,1.5,294.91,24.71,3.71,8.04,31231
50,4.0,0.5,212.46,20.1,2.93,8.49,23920
50,4.0,1.0,222.94,20.74,3.5,8.21,28167
50,4.0,1.5,215.08,20.26,3.66,8.25,22462
100,2.0,0.5,378.56,28.62,2.54,22.72,74027
100,2.0,1.0,374.23,28.43,3.0,15.71,89118
100,2.0,1.5,317.53,25.83,2.93,14.56,80624
100,3.0,0.5,320.1,25.95,3.06,13.35,66012
100,3.0,1.0,307.18,25.32,3.2,11.57,32185
100,3.0,1.5,306.13,25.27,3.52,7.63,33930
100,4.0,0.5,231.48,21.25,2.82,7.44,29160
100,4.0,1.0,227.54,21.01,3.11,7.7,15400
100,4.0,1.5,224.43,20.83,3.33,7.73,18584
200,2.0,0.5,461.5,31.97,2.98,19.25,31024
200,2.0,1.0,461.99,31.99,3.64,10.53,64793
200,2.0,1.5,399.75,29.52,3.57,10.74,33463
200,3.0,0.5,333.36,26.58,3.07,19.24,56569
200,3.0,1.0,325.96,26.23,3.29,10.78,35045
200,3.0,1.5,284.12,24.15,3.21,9.87,34294
200,4.0,0.5,245.61,22.06,2.9,12.52,51143
200,4.0,1.0,223.63,20.78,2.93,9.61,40075
200,4.0,1.5,203.6,19.55,2.96,7.16,40078
```

我们可以看到， $w1 = 50$ ， $zh = 3.0$ 和 $z1 = 1.5$ 的参数值处夏普比率最佳。对于夏普比率，我们的总回报率为 294.91%，最大亏损率为 8.04%。

最佳总回报率为 461.99%，尽管最大亏损为 10.53%，但由 $w1 = 200$ ， $zh = 2.0$ 和 $z1 = 1.0$ 的参数集给出。

16.3.3 可视化

作为策略优化的最后一项任务，我们现在将使用 Matplotlib 可视化后测量绩效特征，这是进行初始策略研究时非常有用的一步。不幸的是，我们遇到了三维问题，因此绩效可视化并不简单。但是，对这种情况有一些措施。

首先，我们可以固定一个参数的值，并通过“数据立方体”的其余部分获取“参数切片”。例如，我们可以将窗口固定为 100，然后查

看相应的 z-score 开仓和平仓阈值的变化对夏普比率或最大亏损的影响。为此，我们将使用 Matplotlib。我们将读取输出 CSV 并重新整数据，以便我们可以看到结果。

Sharpe / Drawdown 热图

我们将固定 $wl = 100$ 的窗口期，然后生成一个 3×3 网格和夏普比率的“热图”以及 z-score 阈值变化的最大值。在以下代码中，我们导入输出 CSV 文件。第一个任务是过滤掉不用的窗口期($wl \in \{50, 200\}$)。然后我们将剩余的绩效数据重新整形为两个 3×3 矩阵。第一个表示每个 z-score 阈值组合的夏普比率，第二个表示最大回撤。

以下是创建夏普比率热图的代码。我们首先导入 Matplotlib 和 NumPy。然后我们定义一个名为 `create_data_matrix` 的函数，它将夏普比率数据重新整形为 3×3 网格。在 `__main__` 函数中，我们打开 CSV 文件（记得更改系统上的路径！）并排除非 $wl=100$ 的行。

然后，我们创建一个蓝色阴影热图，并使用 z-score 阈值正确的行/列标签。随后我们将夏普比率的实际值放在热图上。

最后，我们设置刻度线，标签，标题，然后绘制热图：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# plot_sharpe.py

import matplotlib.pyplot as plt
import numpy as np

def create_data_matrix(csv_ref, col_index):
    data = np.zeros((3, 3))
    for i in range(0, 3):
        for j in range(0, 3):
            data[i][j] = float(csv_ref[i*3+j][col_index])
    return data
```

```

if __name__ == "__main__":
    # Open the CSV file and obtain only the lines
    # with a lookback value of 100
    csv_file = open("/path/to/opt.csv", "r").readlines()
    csv_ref = [
        c.strip().split(",")
        for c in csv_file if c[:3] == "100"
    ]
    data = create_data_matrix(csv_ref, 5)

    fig, ax = plt.subplots()
    heatmap = ax.pcolor(data, cmap=plt.cm.Blues)
    row_labels = [0.5, 1.0, 1.5]
    column_labels = [2.0, 3.0, 4.0]

    for y in range(data.shape[0]):
        for x in range(data.shape[1]):
            plt.text(x + 0.5, y + 0.5, '%.2f' % data[y, x],
                    horizontalalignment='center',
                    verticalalignment='center',
                    )

    plt.colorbar(heatmap)

    ax.set_xticks(np.arange(data.shape[0])+0.5, minor=False)
    ax.set_yticks(np.arange(data.shape[1])+0.5, minor=False)
    ax.set_xticklabels(row_labels, minor=False)
    ax.set_yticklabels(column_labels, minor=False)

    plt.suptitle('Sharpe Ratio Heatmap', fontsize=18)
    plt.xlabel('Z-Score Exit Threshold', fontsize=14)

    plt.ylabel('Z-Score Entry Threshold', fontsize=14)
    plt.show()

```

最大跌幅我们使用一个红色阴影热图并更改 `create_data_matrix` 函数中的列索引以计算最大下跌百分比数据。

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

# plot_drawdown.py

import matplotlib.pyplot as plt
import numpy as np

def create_data_matrix(csv_ref, col_index):
    data = np.zeros((3, 3))
    for i in range(0, 3):
        for j in range(0, 3):
            data[i][j] = float(csv_ref[i*3+j][col_index])
    return data

if __name__ == "__main__":
    # Open the CSV file and obtain only the lines
    # with a lookback value of 100
    csv_file = open("/path/to/opt.csv", "r").readlines()
    csv_ref = [
        c.strip().split(",")
        for c in csv_file if c[:3] == "100"
    ]
    data = create_data_matrix(csv_ref, 6)

    fig, ax = plt.subplots()
    heatmap = ax.pcolor(data, cmap=plt.cm.Reds)
    row_labels = [0.5, 1.0, 1.5]
    column_labels = [2.0, 3.0, 4.0]

    for y in range(data.shape[0]):
        for x in range(data.shape[1]):
            plt.text(x + 0.5, y + 0.5, '%.2f%%' % data[y, x],
                    horizontalalignment='center',
                    verticalalignment='center',
                    )

    plt.colorbar(heatmap)

    ax.set_xticks(np.arange(data.shape[0])+0.5, minor=False)
    ax.set_yticks(np.arange(data.shape[1])+0.5, minor=False)
    ax.set_xticklabels(row_labels, minor=False)
    ax.set_yticklabels(column_labels, minor=False)

    plt.suptitle('Maximum Drawdown Heatmap', fontsize=18)
    plt.xlabel('Z-Score Exit Threshold', fontsize=14)
    plt.ylabel('Z-Score Entry Threshold', fontsize=14)
    plt.show()
```

上述产生的热图如图 16.3.3 和图 16.3.3 所示

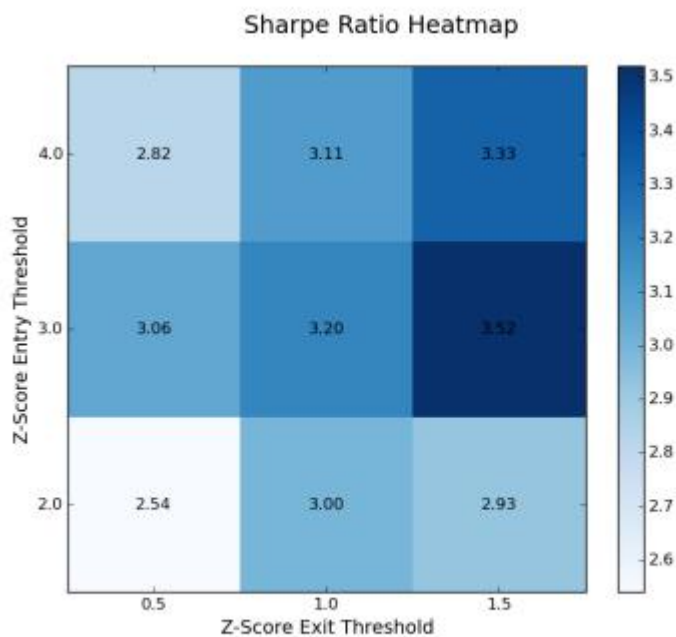


Figure 16.1: Sharpe Ratio heatmap for z-score entry/exit thresholds

Figure 16.1: Sharpe Ratio heatmap for z-score entry/exit thresholds

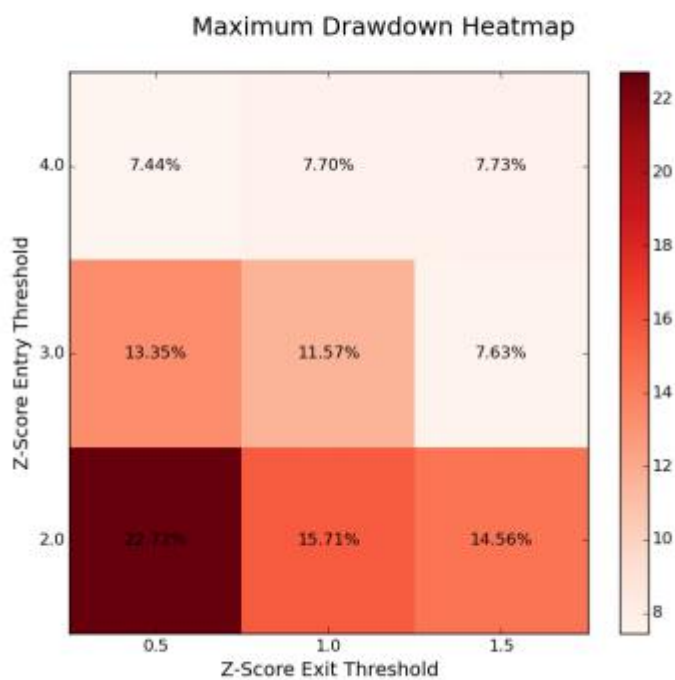


Figure 16.2: Maximum Drawdown heatmap for z-score entry/exit thresholds

在 $w1 = 100$ 时，最小和最大的夏普比率以及最小和最大的最大回撤之间的差异是显而易见的。夏普比率针对较大的开仓和平仓阈值进

行了优化，同时在同一区域内最小化回撤。 当开仓和平仓阈值都很低时，夏普比率和最大跌幅最差。

这显然促使我们考虑在部署到实时交易时使用相对较高的开仓和平仓阈值。